RWCREW · TECHNISCHE DOCUMENTATIE

Ontwikkelaarshandleiding

Opzet, architectuur, gebruikte tools en de manier waarop de applicatie ontwikkeld is, voor iedereen die RWCrew mee gaat onderhouden. Versie 2.1 · met schermafbeeldingen.

- **Deel A**: Opzet en architectuur: hoe RWCrew in elkaar zit en waar het draait
- **Deel B**: Gebruikte tools: elke technologie, wat ze doet en waarom ze gekozen is
- **Deel C**: Hoe de applicatie ontwikkeld is: code, patronen, recepten en werkwijze
- **Deel D**: Dagelijks beheer: wat je kunt doen zonder te programmeren

Versie	2.1
Datum	25 oktober 2026
Codebasis	branch *develop*, stand 25 oktober 2026
Status	Vrijgegeven

INTERN DOCUMENT

Documentbeheer

Over dit document

Deze handleiding beschrijft de webapplicatie **RWCrew**: een beheerplatform met losse modules (Scans, KarTracker, Interventie Aanvragen, Altsien Select, MasterData en de telefoon-app) voor de werking van de Altsien-ploegen. RWCrew werd tot nu toe door één persoon gebouwd. Het doel van dit document is dat een team van meerdere mensen de applicatie kan begrijpen, beheren en verder ontwikkelen zonder dat alle kennis in één hoofd zit.

Documentgegevens

Titel	RWCrew · Ontwikkelaarshandleiding
Versie	2.1 (met schermafbeeldingen)
Datum	25 oktober 2026
Oorspronkelijke ontwikkelaar	Frederik Verstraeten
Beschreven codebasis	Git-repository `RWCrew`, branch `develop`, stand van 25 oktober 2026
Bron van dit document	`docs/handleiding/` in dezelfde repository (zie bijlage C)
Classificatie	Intern: bevat geen wachtwoorden of sleutels, wel interne adressen en werkwijzen

Versiegeschiedenis

Versiegeschiedenis

Versie	Datum	Auteur	Wijziging
1.0	3 oktober 2026	F. Verstraeten	Eerste volledige versie: architectuur, tools, ontwikkelwerkwijze en beheer.
2.0	8 oktober 2026	F. Verstraeten	Schermafbeeldingen van website en telefoon-app; StockMaster (module 4); verwerking van Scans-acties en automatisch scannen; doorsturen van CSV's tussen test en productie; versielabel; Belgische tijd.
2.1	25 oktober 2026	F. Verstraeten	Atelier (module 5: masterdata, reservaties, stock, balie, polsbandscans via webhook met `WRISTBAND_WEBHOOK_TOKEN`); Walkies (module 6) als lege module met eigen rechten; KarTracker: Leveringsvolgorde, Planningsanalyses, Speciale levertoeren, Karcode Simulator en Instellingen; productstamgegevens verhuisd van MasterData naar StockMaster; productenstap in Altsien Select; nieuwe en vernieuwde schermafbeeldingen.

Voor wie is deze handleiding?

Doelgroepen en aanbevolen hoofdstukken

Lezer	Lees zeker
Iedereen die RWCrew mee onderhoudt	Hoofdstukken 1 tot en met 5 en de woordenlijst (bijlage A)
Beheerders (gebruikers, rechten, server, data) zonder programmeerachtergrond	Daarnaast hoofdstukken 6 tot en met 8 en hoofdstuk 18
Ontwikkelaars	Alles, met nadruk op Deel C (hoofdstukken 9 tot en met 17)

Gebruikte kaders

Doorheen het document helpen vaste kaders je om snel te zien wat voor jou belangrijk is:

> KORT GEZEGD
> Samenvatting bovenaan elk hoofdstuk. Wie geen technische details nodig heeft, kan na dit kader verder naar het volgende hoofdstuk.

> VERGELIJKING
> Een vergelijking uit het dagelijkse leven die een technisch begrip uitlegt.

> TIP
> Een handige werkwijze of snelkoppeling.

> LET OP
> Een valkuil die in het verleden al tijd of fouten gekost heeft.

> VOOR ONTWIKKELAARS
> Technische verdieping. Niet-ontwikkelaars mogen dit kader overslaan.

Hoofdstukken krijgen onder hun titel een label: IEDEREEN, BEHEERDERS of ONTWIKKELAARS.

Schrijfwijze

- Schermafbeeldingen zijn gemaakt op een lokale testomgeving. Alle namen, e-mailadressen, telefoonnummers en IP-adressen erop zijn **fictief**; ploegnamen en karnummers zijn testgegevens.
- `backend/app/main.py` of `backend/app/main.py`: een bestand of map in de repository, altijd vanaf de hoofdmap van de repository.
- `npm run dev`: een commando dat je in een terminal typt.
- Termen in het Engels (zoals *backend*, *endpoint*) worden bij het eerste gebruik uitgelegd en staan ook in de woordenlijst. We vertalen ze niet geforceerd, omdat je ze in de code en online documentatie altijd in het Engels tegenkomt.

Inhoudsopgave

INLEIDING 13

1 Hoe lees je deze handleiding 14

1.1 Waarom deze handleiding 14

1.2 Opbouw van het document 14

1.3 Andere bronnen van informatie 15

1.4 Actualiteit van dit document 15

2 Basisbegrippen: hoe een webapplicatie werkt 17

2.1 Drie lagen: browser, server, database 17

2.2 Vraag en antwoord: HTTP 18

2.3 Frontend en backend 19

2.4 API, endpoints en JSON 19

2.5 Database, tabellen en migraties 19

2.6 Inloggen: wachtwoorden, cookies en tokens 20

2.7 Domeinnamen, HTTPS en de reverse proxy 20

2.8 Broncode, Git en GitHub 21

2.9 Van code naar draaiende site: bouwen, containers en deployen 21

2.10 Omgevingen: lokaal, test en productie 22

2.11 Progressive Web App (PWA) 22

DEEL A · OPZET EN ARCHITECTUUR 23

3 Wat is RWCrew? 24

3.1 Doel en context 24

3.2 Het startscherm en de modules 24

3.3 De modules in meer detail 26

3.4 Gedeelde begrippen 39

3.5 Talen en weergave 40

4 Architectuuroverzicht 41

4.1 Het grote plaatje 41

4.2 De frontend: Next.js 42

4.3 De backend: FastAPI 42

4.4 De database: PostgreSQL 42

4.5 Externe koppelingen 43

4.6 Website en telefoon-app: één applicatie 43

4.7 Ontwerpprincipes 43

5 Omgevingen: lokaal, test en productie 45

5.1 Overzicht 45

5.2 De lokale ontwikkelomgeving 45
5.3 De server (Hostinger VPS) 48
5.4 Instellingen per omgeving 49
5.5 De database bekijken (pgAdmin) 50
5.6 Back-ups: aandachtspunt 50

6 Deployment en CI/CD 51
6.1 Van commit tot live 51
6.2 GitHub Actions 51
6.3 Wat deploy.sh doet 52
6.4 De Docker-images 52
6.5 Een nieuwe instelling toevoegen 52
6.6 Welke versie draait waar? 53
6.7 Handmatig deployen en terugdraaien 53
6.8 Checklist voor een release naar productie 53

7 Beveiliging en toegangsrechten 55
7.1 Inloggen en wachtwoorden 55
7.2 Sessies: toegangstoken en vernieuwingstoken 55
7.3 Twee onafhankelijke toegangsniveaus 57
7.4 Extra regels in Altsien Select 60
7.5 Endpoints zonder login 60
7.6 Overige maatregelen 60

DEEL B · GEBRUIKTE TOOLS 62

8 Tools en technologieën 63
8.1 De technologie in één beeld 63
8.2 Frontend 63
8.3 Backend 64
8.4 Database, infrastructuur en diensten 65
8.5 Ontwikkeltools 65
8.6 De Python 3.14-beperking 66
8.7 Claude Code en CLAUDE.md 66
8.8 Afhankelijkheden bijwerken 67

DEEL C · HOE DE APPLICATIE ONTWIKKELD IS 68

9 Structuur van de repository 69
9.1 De hoofdmap 69
9.2 De backend-map 69
9.3 De frontend-map 70
9.4 Waar vind ik…? 70

10 De backend in detail 72
10.1 Opstart: main.py 72
10.2 Instellingen en databaseverbinding 72
10.3 Modellen: tabellen als Python-klassen 72
10.4 Schema's: de vorm van invoer en uitvoer 73
10.5 Het pakket van een module 73
10.6 Dependencies: wie roept aan en mag dat? 74
10.7 Het rechtensysteem in code 75
10.8 De gedeelde fabriek voor eenvoudige modules 76
10.9 Gedeelde logica tussen modules 76
10.10 StockMaster: grootboek en saldo 77
10.11 Atelier: stock, balie en polsbandscans 77
10.12 PDF's, Excel en QR-codes 78
10.13 Uitgaande mail 78
10.14 Databasemigraties met Alembic 78
10.15 Automatische tests 79

11 De frontend in detail 81
11.1 Mappen zijn adressen 81
11.2 De lagen van layouts 81
11.3 Servercomponenten en clientcomponenten 82
11.4 Gegevens ophalen: server en browser 82
11.5 Componenten 83
11.6 Vertalingen 83
11.7 Thema en modulekleuren 84
11.8 Eigenaardigheden van shadcn op Base UI 84
11.9 Configuratie van de frontend 84

12 De telefoon-app (PWA) 85
12.1 Waarom een PWA 85
12.2 Installeerbaar maken: het manifest 86
12.3 Structuur 86
12.4 Rechten en routes per module 86
12.5 Code die website en telefoon delen 88
12.6 Camera, GPS en HTTPS 88
12.7 Website of telefoon in de inloghistoriek 88

13 Eén verzoek van begin tot eind 89
13.1 Overzicht 89
13.2 Stap 1–2: in de browser 89
13.3 Stap 3–5: Nginx en de controles in de backend 90

13.4 Stap 6–8: opslaan en antwoorden 91
13.5 Stap 9–10: terug in de browser, en de mail 91
13.6 Varianten van hetzelfde pad 91

14 Recepten: veelvoorkomende uitbreidingen 92
14.1 Een nieuw scherm in een bestaande module 92
14.2 Een kolom of tabel toevoegen of wijzigen 93
14.3 Een nieuwe module bouwen (bv. module 7 echt maken) 93
14.4 Een stap toevoegen aan de Ploeg Wizard 93
14.5 Een module toevoegen aan de telefoon-app 94
14.6 Een schermtekst toevoegen of wijzigen 94
14.7 Een nieuwe instelling 94
14.8 Een Excel-import toevoegen 94
14.9 Een nieuwe mail toevoegen 94

15 Codeerafspraken 96
15.1 Commentaar in gewone taal 96
15.2 Alles per module 96
15.3 Naamgeving 96
15.4 Lijstschermen: titels en kolomkoppen 97
15.5 Tabellen met vaste kop en Acties-kolom 97
15.6 Belgische tijd overal 98
15.7 Afspraken in de backend 98
15.8 Afspraken in de frontend 98
15.9 Commits en branches 99

16 Samenwerken als team 100
16.1 De huidige situatie 100
16.2 Aanbevolen branchmodel 100
16.3 Branches beschermen op GitHub 101
16.4 Aanbevolen: automatische tests bij elke pull request 101
16.5 Code review 101
16.6 Definition of done 101
16.7 Rollen in het team 102
16.8 Toegangen en geheimen beheren 102

17 Bekende valkuilen 103
17.1 De backend reageert met oude code (--reload op Windows) 103
17.2 Taal wisselen laat de hele pagina flitsen 103
17.3 Overige valkuilen 103

DEEL D · DAGELIJKS BEHEER 106

18 Beheer zonder te programmeren 107

18.1 De superbeheerder 107
18.2 Een nieuwe gebruiker toevoegen 107
18.3 Toegang intrekken, wachtwoord resetten, verwijderen 108
18.4 Rollen beheren binnen een module 108
18.5 De inloghistoriek bekijken 109
18.6 Seizoenen en stamgegevens 109
18.7 De mailinglijst van Interventie Aanvragen 111
18.8 Een Scans-scanner toevoegen 111
18.9 De polsbandscans van Atelier koppelen 112
18.10 De telefoon-app laten installeren 113
18.11 Controleren of alles draait 113
18.12 Gegevens bekijken 114
18.13 Wat te doen bij een storing 114

BIJLAGEN 115

Bijlage AWoordenlijst 116

Bijlage BSchermen en rechtensleutels per module 122

Bijlage CDeze handleiding bijwerken 125

C.1 Werkwijze 125
C.2 Schermafbeeldingen vernieuwen 125
C.3 Opmaakregels voor auteurs 126

Lijst van figuren en tabellen

Figuren

Figuur 2.1De drie lagen van een webapplicatie. De browser praat nooit rechtstreeks met de database. 17

Figuur 3.1Het startscherm voor een gebruiker met toegang tot alle modules. Rechtsboven de kleine tegel van de Mobile App; bovenaan de seizoenkiezer, taal, thema en gebruikersmenu. 26

Figuur 3.2Scans · Tag Regelgegevens: één rij per gescande tag. De kleur van de eerste kolom toont de verwerkingsstatus (groen = verwerkt). 27

Figuur 3.3Scans · Instellingen: de ontvangstmap, het automatisch scannen met zijn interval en het adres waarop apparaten hun CSV-bestanden sturen. 27

Figuur 3.4KarTracker · KarManagement: het karrenregister met status, ploeg, transporttype en laatst gekende locatie. 29

Figuur 3.5KarTracker · Kar Map: karren, afleverlocaties en distributiepunten op de kaart (OpenStreetMap), met een tabel van de getoonde punten eronder. 29

Figuur 3.6KarTracker · Leveringsvolgorde: links de levertoer (slepen om te ordenen, met de vorige positie ernaast), rechts de actieve afleverlocaties zonder positie. 30

Figuur 3.7KarTracker · Initiële leverplanning: per distributiepunt en afleverlocatie (in tourvolgorde) de karren die bij de start geleverd worden, met filters en selectie om af te drukken. 30

Figuur 3.8Interventie Aanvragen: de lijst van aanvragen, gegroepeerd per dag, met de status in kleur en per regel PDF, wijzigen en verwijderen. 31

Figuur 3.9StockMaster · Voorraadoverzicht: per product de vrije voorraad, wat in karren zit en het totaal, met snelle acties per regel. 33

Figuur 3.10StockMaster · Karren: één kaart per kar met de status (in het magazijn of onderweg), de ploeg en hoe ver ze geladen is. 33

Figuur 3.11StockMaster · detail van één kar: inhoud tegenover de benodigdheden, de acties (laden, vertrekt, uitladen, tellen, laadlijst) en de geschiedenis. 34

Figuur 3.12Altsien Select · Ploeg Wizard voor één ploeg: bovenaan de stappen met hun voortgang, eronder de huidige stap (hier: de festivals). 35

Figuur 3.13Altsien Select · Ploegfiche: alle keuzes van één ploeg op één scherm, ook als PDF af te drukken. 35

Figuur 3.14MasterData · Ploegen: de stamgegevens van alle ploegen, met locatie, leveringswijze en of ze actief zijn. 36

Figuur 3.15Atelier · Uitgifte & inname: festival, dag en de scanner van deze balie, en de twee grote knoppen waarmee elke transactie begint. 37

Figuur 3.16Atelier · Reservaties: per festival alle actieve ploegen, met het aantal personen, een opmerking en het pakket. 38

Figuur 3.17Atelier · Stock: per artikel en maat beschikbaar, te wassen en uitgegeven, met de minimumstock. 38

Figuur 3.18Atelier · Instellingen: polsband verplicht of niet, het webhookadres en of de token ingesteld is, de scanners en alle ontvangen berichten. 39

Figuur 4.1Overzicht van alle onderdelen van RWCrew en hun verbindingen (productie; test is een tweede, identieke set containers op dezelfde server). 41

Figuur 5.1De lokale opstelling: database in Docker, backend en frontend rechtstreeks, optioneel een tunnel voor een telefoon via HTTPS. 46

Figuur 6.1De weg van een wijziging: eerst naar test via develop, na controle naar productie via main. 51
Figuur 7.1Levensloop van een inlogsessie: korte toegangstokens, een vernieuwingstoken dat bij elk gebruik vervangen wordt, en een harde grens van 6 uur. 56
Figuur 7.2De twee toegangsniveaus. Een gebruiker heeft eerst moduletoegang nodig; pas dan telt zijn rol binnen de module. 57
Figuur 7.3Manage Access (Toegangsbeheer): per gebruiker de vlaggen Super admin en Altsien Kernlid, en één vinkje per module. 57
Figuur 7.4Access Rights > Rollen: de rechtenmatrix van een rol, met per scherm de vinkjes Bekijken, Aanmaken, Wijzigen en Verwijderen. 59
Figuur 7.5Access Rights > Gebruikers: per gebruiker met moduletoegang één rol in deze module. 59
Figuur 8.1Alle tools van RWCrew, gegroepeerd per laag. 63
Figuur 11.1De geneste layouts rond een scherm van module 3. Elke laag voegt één verantwoordelijkheid toe. 82
Figuur 12.1De telefoon-app: inloggen, de moduletegels met de jaarkeuze, en het menu van KarTracker. 85
Figuur 12.2Telefoonschermen: een kar en haar bewegingen (KarScan), de lijst van interventie-aanvragen (Akties) en een nieuwe aanvraag. 87
Figuur 12.3Kar Planning en het KPI-overzicht van Interventie Aanvragen op de telefoon. 88
Figuur 13.1Het pad van "nieuwe interventie-aanvraag opslaan" door het systeem. De mail vertrekt pas nadat de gebruiker zijn antwoord kreeg. 89
Figuur 13.2Het dialoogvenster ‘Nieuwe interventieaanvraag aanmaken’ waarin de medewerker het formulier invult. 90
Figuur 16.1Aanbevolen branchmodel: werk op korte feature-branches, merge via pull requests in develop, en breng develop via een release-PR naar main. 100
Figuur 18.1Manage Access: gebruikers aanmaken en per gebruiker de modules aanvinken. Rechtsboven de knop ‘Nieuwe gebruiker’. 108
Figuur 18.2Login History: elke inlogpoging met tijdstip, gebruiker, resultaat, bron (website of telefoon-app), toestel en IP-adres. 109
Figuur 18.3MasterData · Jaartallen: de seizoenen en of hun periode open is. 110
Figuur 18.4MasterData · Data Upload/Download: per tabel een sjabloon downloaden, invullen en weer uploaden. 110
Figuur 18.5Interventie Aanvragen · Mailinglijst: wie een mail krijgt bij elke nieuwe aanvraag. 111
Figuur 18.6Scans · Scanners: de geregistreerde scanners; via het sleutelpictogram maak je een nieuwe API-sleutel aan. 112
Figuur 18.7Mobile App (module 10): het adres van de telefoon-app met de installatiestappen. Lokaal staat er ‘niet geconfigureerd’ omdat APP_PUBLIC_URL daar leeg is; op test en productie staat hier de QR-code. 113

Tabellen

Tabel 1.1De vier delen van deze handleiding 14
Tabel 1.2Bestaande documentatie in de repository 15
Tabel 2.1De vier HTTP-methodes die RWCrew gebruikt 18
Tabel 2.2Veelvoorkomende HTTP-statuscodes 18

Tabel 3.1De modules van RWCrew 24
Tabel 3.2De stappen van de Ploeg Wizard 34
Tabel 3.3De blokken van Atelier 36
Tabel 3.4Begrippen uit het domein die in meerdere modules terugkomen 39
Tabel 4.1Groepen van endpoints in de backend 42
Tabel 4.2Voorvoegsels van de databasetabellen 42
Tabel 4.3Diensten buiten de eigen server 43
Tabel 5.1De drie omgevingen naast elkaar 45
Tabel 5.2Benodigde software op een ontwikkel-pc 46
Tabel 5.3Hoe Nginx verzoeken verdeelt 48
Tabel 5.4Belangrijkste instellingen 49
Tabel 5.5Instellingen in pgAdmin 50
Tabel 6.1GitHub-secrets voor de deployment 51
Tabel 7.1De twee inlogcookies 55
Tabel 7.2Voorbeeld van een rechtenmatrix voor de rol "Logistiek" in Interventie Aanvragen 58
Tabel 7.3Publieke endpoints en hun beveiliging 60
Tabel 8.1Tools van de frontend 63
Tabel 8.2Tools van de backend 64
Tabel 8.3Database, hosting en externe diensten 65
Tabel 8.4Tools voor het dagelijkse ontwikkelwerk 65
Tabel 8.5Bewuste keuzes door de Python 3.14-beperking 66
Tabel 9.1Snel de juiste plaats vinden 70
Tabel 10.1Bestanden in backend/app/modules/module_N/ 73
Tabel 10.2De belangrijkste dependencies 74
Tabel 10.3Onderdelen van het rechtensysteem per module (voorbeeld: module 3) 75
Tabel 10.4Gedeelde services 76
Tabel 11.1Voorbeelden van map naar adres 81
Tabel 11.2De drie manieren om de backend aan te spreken 82
Tabel 11.3Radix-gewoonten die hier niet werken 84
Tabel 13.1Dezelfde handeling via andere ingangen 91
Tabel 15.1Naamgevingsregels 96
Tabel 16.1Voorgestelde rollen (één persoon kan meerdere rollen hebben) 102
Tabel 16.2Systemen waarvoor toegang beheerd moet worden 102
Tabel 17.1Symptoom, oorzaak en oplossing van bekende problemen 103
Tabel 18.1Veelvoorkomende wijzigingen aan gebruikers 108
Tabel 18.2Snelle controles 113
Tabel 18.3Eerste hulp bij storingen 114
Tabel B.1Scans (module 1) 122
Tabel B.2KarTracker (module 2) 122
Tabel B.3Interventie Aanvragen (module 3) 123

Tabel B.4Altsien Select (module 8) 123
Tabel B.5MasterData (module 9) 123
Tabel B.6StockMaster (module 4) 123
Tabel B.7Atelier (module 5) 124
Tabel B.8Walkies (module 6) 124
Tabel C.1Bestanden van de handleiding 125

INLEIDING

Voor je begint

Waarom deze handleiding bestaat, hoe je ze leest en de basisbegrippen die je nodig hebt om de rest te volgen, ook als je zelf nooit een webapplicatie hebt gebouwd.

Doelgroep: iedereen

1 Hoe lees je deze handleiding

IEDEREEN

> **KORT GEZEGD**
> RWCrew is door één persoon gebouwd en wordt voortaan door een team onderhouden. Deze handleiding legt vast wat tot nu toe vooral in één hoofd zat: hoe de applicatie in elkaar zit, met welke tools ze gemaakt is, hoe je ze lokaal opstart, hoe ze op de server terechtkomt en welke afspraken er in de code gelden. Ze is geschreven voor IT-minded lezers die niet allemaal ontwikkelaar zijn: elk technisch begrip wordt uitgelegd voordat het gebruikt wordt.

1.1 Waarom deze handleiding

Een applicatie die door één persoon gebouwd is, heeft een groot risico: de kennis over *waarom* iets zo gebouwd is, zit niet in de code maar in het hoofd van die persoon. Zolang die persoon beschikbaar is, is dat geen probleem. Zodra meerdere mensen de applicatie moeten beheren, wordt het dat wel. Dan moet iemand anders een fout kunnen oplossen, een nieuwe gebruiker kunnen toevoegen of een nieuwe module kunnen bouwen zonder telkens te moeten vragen hoe het zit.

Deze handleiding heeft daarom drie doelen:

1. **Overzicht geven**: in een paar uur lezen moet je begrijpen uit welke onderdelen RWCrew bestaat, hoe die met elkaar praten en waar ze draaien.
2. **Keuzes verklaren**: bij elke technologie en elk patroon staat waarom het gekozen is. Zo kun je later een bewuste beslissing nemen om iets te veranderen, in plaats van per ongeluk een zorgvuldig opgeloste valkuil opnieuw open te breken.
3. **Zelfstandig laten werken**: met de recepten in hoofdstuk 14 en de beheertaken in hoofdstuk 18 kun je de meest voorkomende taken uitvoeren zonder hulp.

1.2 Opbouw van het document

Het document is opgedeeld in vier delen, die van algemeen naar specifiek gaan:

Tabel 1.1 – De vier delen van deze handleiding

Deel	Inhoud	Voor wie
Inleiding	Deze leeswijzer en een uitleg van de basisbegrippen van webapplicaties.	Iedereen
A. Opzet en architectuur	Wat RWCrew doet, uit welke onderdelen het bestaat, op welke omgevingen het draait, hoe een nieuwe versie online komt en hoe beveiliging en toegangsrechten werken.	Iedereen
B. Gebruikte tools	Elke technologie, bibliotheek en dienst die RWCrew gebruikt: wat ze doet en waarom ze gekozen is.	Iedereen; details voor ontwikkelaars
C. Hoe de applicatie ontwikkeld is	De code zelf: mappenstructuur, backend, frontend, telefoon-app, een volledige doorloop van één verzoek, recepten, codeerafspraken, samenwerken en valkuilen.	Ontwikkelaars

D. Dagelijks beheer	Taken die je zonder programmeren uitvoert: gebruikers en rechten, stamgegevens, servercontrole, data bekijken, wat te doen bij een storing.	Beheerders
Bijlagen	Woordenlijst, overzicht van alle schermen en rechten, en hoe je deze handleiding zelf bijwerkt.	Iedereen

1.3 Andere bronnen van informatie

Deze handleiding is het startpunt, maar niet de enige documentatie. De volgende bronnen bestaan al in de repository en blijven geldig:

Tabel 1.2 – Bestaande documentatie in de repository

Bestand	Wat staat erin
`CLAUDE.md`	Het "werkgeheugen" van het project: architectuur, valkuilen en afspraken in compacte vorm. Wordt automatisch gelezen door de AI-assistent Claude Code (zie §8.7), maar is ook voor mensen de beknoptste samenvatting.
`README.md`, `backend/README.md`, `frontend/README.md`, `deploy/README.md`	Korte opstartinstructies per onderdeel.
`docs/module-custom-roles-pattern.md`	Stap-voor-stap gids om het rollen- en rechtensysteem in een module te bouwen (zie §7.3.2).
`docs/deploying-to-hostinger.pdf`	De volledige installatie van de server (VPS), Docker, Nginx, HTTPS en GitHub Actions.
`docs/tagscan-raspi-setup-guide.pdf`, `docs/tagscan-raspi-to-vps-beginner-guide.pdf`	Installatie van de Raspberry Pi die Scans-bestanden doorstuurt.
`docs/RW-Crew-Codebase-Guide.pdf`	Een oudere, Engelstalige rondleiding door de code (augustus 2026). Nuttig voor de basisprincipes, maar van vóór de server, de telefoon-app en modules 8 tot 10.
Commentaar in de code zelf	Elk bestand is rijkelijk becommentarieerd in gewone taal (zie §15.1). Het commentaar bovenaan een bestand vertelt meestal waarom het bestaat.

> LET OP
>
> Er bestaat op dezelfde ontwikkel-pc een **ander, los project** dat ook "RWCrew" heet: een SaaS-applicatie bij Combell (map `c:\Fre Software\RWCrew.be`) met modules als Kar Tracker, Kar Scanner en Walkies. Dat project heeft niets met deze repository te maken. Deze handleiding gaat uitsluitend over de repository `RWCrew` op GitHub, die draait op `rwcrew.eu`.

1.4 Actualiteit van dit document

Software verandert voortdurend; documentatie veroudert. Deze versie beschrijft de stand van de code op 25 oktober 2026 (branch `develop`). Versie 1.0 van deze handleiding (zonder schermafbeeldingen) blijft beschikbaar als `docs/RWCrew-Ontwikkelaarshandleiding.pdf`. Wanneer je iets verandert dat in dit document

beschreven staat, pas dan ook de handleiding aan. Dat is eenvoudig: de bron is HTML in dezelfde repository en de PDF wordt met één commando opnieuw gemaakt (zie bijlage C). Spreek in het team af dat een wijziging pas "klaar" is als ook de documentatie klopt (zie §16.6).

2 Basisbegrippen: hoe een webapplicatie werkt

IEDEREEN

KORT GEZEGD

Een webapplicatie bestaat uit drie lagen: de **browser** van de gebruiker (wat je ziet), een **server** die de regels toepast (wie mag wat, wat wordt bewaard) en een **database** waarin alle gegevens blijven staan. De lagen praten met elkaar via vraag-en-antwoordberichten over het internet. Dit hoofdstuk legt die begrippen uit, samen met wat je moet weten over inloggen, beveiliging, broncode, Git en het online zetten van een nieuwe versie. Als je dit al kent, kun je meteen doorgaan naar hoofdstuk 3.

2.1 Drie lagen: browser, server, database

Bijna elke moderne zakelijke applicatie, en zeker RWCrew, volgt dezelfde basisopbouw:

- **De browser** (Edge, Chrome, Safari) op de pc of telefoon van de gebruiker toont schermen, knoppen en formulieren. De browser zelf bewaart geen bedrijfsgegevens: hij vraagt ze op wanneer hij ze nodig heeft en vergeet ze weer zodra je het venster sluit.
- **De server** is een computer die altijd aan staat, ergens in een datacenter. Op die server draaien programma's die verzoeken van browsers ontvangen, controleren of de gebruiker het verzoek mag doen, de juiste gegevens ophalen of wegschrijven en een antwoord terugsturen.
- **De database** is het geheugen van de applicatie: een programma dat gegevens netjes in tabellen bewaart, ook als de server herstart. Alleen de server praat met de database; een browser kan nooit rechtstreeks in de database kijken.



Figuur 2.1 – De drie lagen van een webapplicatie. De browser praat nooit rechtstreeks met de database.

VERGELIJKING

Denk aan een restaurant. De **browser** is de klant aan tafel met de menukaart. De **server** is de ober en de keuken: die neemt de bestelling op, controleert of het gerecht bestaat en of de klant het mag bestellen, en brengt het bord. De **database** is de voorraadkamer. Een klant loopt nooit zelf de voorraadkamer in.

2.2 Vraag en antwoord: HTTP

Browser en server praten met elkaar via **HTTP** (*HyperText Transfer Protocol*), de "taal" van het web. Elk gesprek bestaat uit precies één **verzoek** (*request*) van de browser en één **antwoord** (*response*) van de server. Een verzoek bevat:

- een **adres** (URL), bijvoorbeeld `https://rwcrew.eu/api/modules/module-3/intervention-requests`;
- een **methode** die zegt wat je wilt doen (zie tabel 2.1);
- eventueel **gegevens**, bijvoorbeeld de ingevulde velden van een formulier;
- **cookies**: kleine stukjes tekst die de browser automatisch meestuurt, bijvoorbeeld het bewijs dat je ingelogd bent (zie §2.6).

Tabel 2.1 – De vier HTTP-methodes die RWCrew gebruikt

Methode	Betekenis	Voorbeeld in RWCrew
`GET`	Gegevens opvragen (leest alleen)	De lijst van interventie-aanvragen ophalen
`POST`	Iets nieuws aanmaken of een actie starten	Een nieuwe aanvraag indienen, inloggen
`PUT` / `PATCH`	Iets bestaands wijzigen	De status van een aanvraag aanpassen
`DELETE`	Iets verwijderen	Een rol verwijderen

Elk antwoord heeft een **statuscode**: een getal dat in één oogopslag zegt hoe het verzoek afgelopen is. Je ziet deze codes terug in foutmeldingen en in de ontwikkelaarstools van de browser, dus het loont om de belangrijkste te kennen:

Tabel 2.2 – Veelvoorkomende HTTP-statuscodes

Code	Naam	Betekenis in RWCrew
`200`	OK	Alles in orde; het antwoord bevat de gevraagde gegevens.
`201`	Created	Er is iets nieuws aangemaakt (bv. een aanvraag).
`204`	No Content	Gelukt, maar er is niets terug te sturen (bv. na verwijderen).
`400`	Bad Request	De gegevens kloppen niet (bv. een rol toekennen aan iemand zonder moduletoegang).
`401`	Unauthorized	Je bent niet (meer) ingelogd. Na 15 minuten verloopt het toegangsbewijs; de app vernieuwt het dan automatisch (zie §7.2).
`403`	Forbidden	Je bent ingelogd, maar je hebt geen recht op deze module of dit scherm.
`404`	Not Found	Het gevraagde bestaat niet (of je mag niet weten dat het bestaat, zoals een ploeg van iemand anders in Altsien Select).
`409` / `422`	Conflict / Unprocessable	De invoer botst met bestaande gegevens of heeft een verkeerd formaat.
`500`	Internal Server Error	Er ging iets mis in de server zelf: een fout in de code of een onbereikbare database. Altijd onderzoeken.

2.3 Frontend en backend

De software die op de server draait, wordt in twee grote stukken verdeeld:

- De **frontend** is alles wat met *tonen* te maken heeft: de schermen, menu's, tabellen en formulieren. In RWCrew is dat een programma geschreven met **Next.js** (JavaScript/TypeScript). Het maakt de webpagina's die naar de browser gaan.
- De **backend** is alles wat met *regels en gegevens* te maken heeft: wie mag inloggen, wie mag wat zien, wat wordt er in de database bewaard, welke mail wordt verstuurd. In RWCrew is dat een programma geschreven met **FastAPI** (Python).

Die scheiding is bewust. De frontend mag nooit zelf beslissen of iemand iets mag. Wie technisch handig is, kan in zijn browser alles aanpassen wat de frontend toont, inclusief verborgen knoppen zichtbaar maken. Daarom controleert de **backend bij elk verzoek opnieuw** of de gebruiker het recht heeft. Een verborgen knop in de frontend is gebruiksgemak; de echte beveiliging zit altijd in de backend.

2.4 API, endpoints en JSON

De backend biedt een **API** aan (*Application Programming Interface*): een vaste lijst van adressen waarop de frontend vragen kan stellen. Elk zo'n adres, samen met zijn methode, heet een **endpoint**. Voorbeelden uit RWCrew:

- `POST /api/auth/login`: inloggen met e-mailadres en wachtwoord;
- `GET /api/auth/me`: "wie ben ik en welke modules mag ik openen?";
- `GET /api/modules/module-3/intervention-requests`: de lijst van interventie-aanvragen.

De gegevens die heen en weer gaan, staan in **JSON**-formaat (*JavaScript Object Notation*): een eenvoudig, leesbaar tekstformaat van namen en waarden. Zo ziet het antwoord van `/api/auth/me` er bijvoorbeeld uit:

Antwoord van GET /api/auth/me (JSON)

```
{
  "id": 1,
  "email": "beheerder@voorbeeld.be",
  "display_name": "Beheerder",
  "is_super_admin": true,
  "language_preference": "nl",
  "accessible_module_keys": ["module-2", "module-3", "module-9"]
}
```

> **TIP**
> De backend van RWCrew documenteert zichzelf. Als de backend lokaal draait, opent `http://localhost:8020/docs` een interactieve pagina met *alle* endpoints, de gegevens die ze verwachten en die ze teruggeven. Je kunt er ook meteen verzoeken mee uitproberen.

2.5 Database, tabellen en migraties

RWCrew gebruikt **PostgreSQL**, een veelgebruikte, gratis en zeer betrouwbare *relationele* database. "Relationeel" betekent dat gegevens in **tabellen** staan (zoals werkbladen in Excel) die naar elkaar verwijzen:

- Een **tabel** bevat één soort gegevens, bv. `Landing_users` (gebruikers) of `InterventionRequests_request` (aanvragen).
- Een **rij** is één exemplaar (één gebruiker, één aanvraag); een **kolom** is één eigenschap (e-mailadres, aanmaakdatum).
- Elke rij heeft een unieke **sleutel** (`id`). Een kolom die naar de sleutel van een andere tabel verwijst, heet een **vreemde sleutel** (*foreign key*). Zo verwijst een aanvraag via `team_id` naar een ploeg in `MasterData_team`.

De taal waarmee je een database bevraagt, heet **SQL**. In RWCrew schrijf je zelden rechtstreeks SQL: de bibliotheek **SQLAlchemy** vertaalt Python-code naar SQL (zie §10.3).

Wanneer de structuur van de database verandert, bijvoorbeeld een nieuwe tabel of een extra kolom, gebeurt dat via een **migratie**: een klein script dat de wijziging beschrijft (en hoe je ze terugdraait). Migraties worden genummerd en in volgorde uitgevoerd, op elke omgeving. Zo heeft de database lokaal, op test en op productie altijd exact dezelfde structuur, zonder dat iemand met de hand tabellen aanmaakt. RWCrew gebruikt daarvoor de tool **Alembic** (zie §10.14).

> VERGELIJKING
> Een migratie is als een verbouwingsplan voor een huis: "voeg een raam toe in de keuken". Elk plan bouwt voort op het vorige. Door alle plannen in volgorde uit te voeren, ziet elk huis (elke omgeving) er uiteindelijk hetzelfde uit, en de bewoners (de gegevens) blijven gewoon wonen tijdens de verbouwing.

2.6 Inloggen: wachtwoorden, cookies en tokens

HTTP heeft geen geheugen: elk verzoek staat op zichzelf. De server weet dus niet vanzelf dat het verzoek van daarnet en dat van nu van dezelfde persoon komen. Daarom werkt inloggen zo:

1. Je typt je e-mailadres en wachtwoord. De backend vergelijkt het wachtwoord met de opgeslagen **hash**: een onomkeerbare "vingerafdruk" van het wachtwoord. Het wachtwoord zelf wordt nooit bewaard.
2. Klopt het, dan maakt de backend een **token**: een digitaal ondertekend bewijsje met daarin "dit is gebruiker 12, geldig tot 14u15". Niemand kan dat bewijsje vervalsen zonder de geheime sleutel van de server.
3. De backend geeft het token mee als **cookie**. De browser bewaart die cookie en stuurt hem vanaf dan automatisch mee met elk verzoek naar dezelfde website.
4. Bij elk volgend verzoek leest de backend de cookie, controleert de handtekening en weet zo wie je bent.

RWCrew gebruikt twee tokens: een kortlevend **toegangstoken** (15 minuten) en een **vernieuwingstoken** dat de sessie tot maximaal 6 uur na het inloggen verlengt. Hoe dat precies werkt en waarom, staat in §7.2. Het type token heet **JWT** (*JSON Web Token*), een open standaard.

2.7 Domeinnamen, HTTPS en de reverse proxy

Een **domeinnaam** zoals `rwcrew.eu` is een leesbare naam voor het IP-adres van een server. Het systeem dat namen naar adressen vertaalt, heet **DNS**. Voor RWCrew wordt DNS beheerd bij Cloudflare.

HTTPS is HTTP met versleuteling: niemand onderweg (bv. op een openbaar wifi-netwerk) kan meelezen of iets aanpassen. Daarvoor heeft de server een **certificaat** nodig. RWCrew gebruikt gratis certificaten van **Let's Encrypt**, die automatisch vernieuwd worden door het programma **certbot**. Moderne browsers geven sommige

functies alleen vrij op HTTPS, zoals de **camera** (voor het scannen van QR-codes) en **GPS**. Dat speelt een rol bij de telefoon-app (zie hoofdstuk 12).

Op de server staat een **reverse proxy** (**Nginx**) als portier vooraan. Alle verkeer van buitenaf komt eerst bij Nginx binnen. Nginx regelt de versleuteling en stuurt elk verzoek door naar het juiste programma: adressen die met `/api/` beginnen naar de backend, al de rest naar de frontend.

2.8 Broncode, Git en GitHub

De **broncode** is de tekst die ontwikkelaars schrijven en waaruit de applicatie gebouwd wordt. Ze staat in een **repository** ("repo"): een map met alle bestanden plus hun volledige geschiedenis. Die geschiedenis wordt bijgehouden door **Git**. De centrale kopie van de repository staat op **GitHub**.

Tabel 2.3 – Basisbegrippen van Git

Begrip	Uitleg
Commit	Een bewaarde momentopname van wijzigingen, met een korte beschrijving ("Mailinglijst toegevoegd aan module 3"). Elke commit heeft een unieke code zoals `97cdc36` .
Branch	Een aparte lijn van commits. Je kunt op een branch werken zonder anderen te storen. RWCrew heeft twee vaste branches: `main` (= productie) en `develop` (= test).
Push / pull	Je eigen commits naar GitHub sturen (*push*) of andermans commits ophalen (*pull*).
Merge	De commits van één branch samenvoegen in een andere, bv. `develop` in `main` om een nieuwe versie naar productie te brengen.
Pull request (PR)	Een voorstel op GitHub om een branch samen te voegen, waar collega's de wijzigingen kunnen nalezen en goedkeuren voordat ze doorgaan.
Clone	Een volledige kopie van de repository op je eigen pc maken.

2.9 Van code naar draaiende site: bouwen, containers en deployen

Broncode kan niet zomaar op een server "staan". Ze moet eerst **gebouwd** worden (de frontend wordt omgezet naar geoptimaliseerde bestanden) en dan **gestart** worden met de juiste instellingen. Het hele proces om een nieuwe versie live te zetten, heet **deployen**.

RWCrew gebruikt daarvoor **Docker**. Docker verpakt een programma samen met alles wat het nodig heeft (de juiste Python- of Node.js-versie, bibliotheken, instellingen) in een **container**. Een container draait overal precies hetzelfde: op een ontwikkel-pc en op de server. Het recept voor een container staat in een **Dockerfile**; welke containers samen een applicatie vormen, staat in een **docker-compose**-bestand.

> VERGELIJKING
>
> Een container is als een zeecontainer: wat erin zit maakt voor de haven (de server) niet uit, de afmetingen en de manier van stapelen zijn altijd hetzelfde. Daardoor maakt het niet uit of een programma Python of Node.js nodig heeft. De server hoeft alleen Docker te kennen.

Het automatisch bouwen en deployen bij elke wijziging heet **CI/CD** (*Continuous Integration / Continuous Deployment*). Bij RWCrew doet **GitHub Actions** dat: zodra er een commit op `develop` of `main` binnenkomt, logt GitHub in op de server en zet het de nieuwe versie live (zie hoofdstuk 6).

2.10 Omgevingen: lokaal, test en productie

Dezelfde applicatie draait op meerdere plaatsen, elk met een eigen database en eigen instellingen. Zo'n plaats heet een **omgeving**:

- **Lokaal** (*development*): op de pc van een ontwikkelaar, om te bouwen en te testen. Gegevens zijn proefgegevens.
- **Test** (`test.rwcrew.eu`): een volledige kopie op de server, waar nieuwe functies eerst door echte gebruikers uitgeprobeerd worden. Je herkent ze aan een gekleurde banner "TEST".
- **Productie** (`rwcrew.eu`): de echte applicatie met de echte gegevens.

Instellingen die per omgeving verschillen (databasewachtwoord, geheime sleutels, mailinstellingen) staan niet in de code maar in een **omgevingsbestand** (`.env`). Die bestanden staan nooit in Git, omdat ze geheimen bevatten.

2.11 Progressive Web App (PWA)

Een **Progressive Web App** is een website die zich op een telefoon gedraagt als een app: je zet ze via "Zet op beginscherm" met een eigen icoon op je telefoon, en ze opent schermvullend zonder adresbalk. Er is geen App Store of Play Store nodig, en een nieuwe versie van de website is meteen ook een nieuwe versie van de app. RWCrew gebruikt dit voor de telefoonversie onder het adres `/m` (zie hoofdstuk 12).

DEEL A

Opzet en architectuur

Wat RWCrew doet, uit welke onderdelen het bestaat en hoe die samenwerken, op welke omgevingen het draait, hoe een nieuwe versie online komt en hoe inloggen, beveiliging en toegangsrechten geregeld zijn.

Doelgroep: iedereen. Technische verdieping staat in paarse kaders.

3 Wat is RWCrew?

IEDEREEN

> **KORT GEZEGD**
> RWCrew is een webapplicatie om de logistiek en organisatie van de Altsien-ploegen op festivals te beheren. Na het inloggen zie je een startscherm met **tegels**: één per module waartoe je toegang hebt. Elke module is een afgerond stuk functionaliteit (RFID-tags, karren, interventie-aanvragen, ploegkeuzes, stamgegevens) met een eigen menu en eigen rechten. Een deel van de functies is ook beschikbaar als **telefoon-app**. De applicatie is tweetalig (Nederlands en Engels) en draait op `rwcrew.eu` (productie) en `test.rwcrew.eu` (test).

3.1 Doel en context

Altsien-ploegen zijn ploegen van verenigingen die tijdens festivals meewerken. Daar komt veel logistiek bij kijken: **karren** (rolcontainers met materiaal) moeten op het juiste moment op de juiste plaats geleverd en weer opgehaald worden, materiaal wordt met **RFID-tags** gevolgd, tijdens het festival komen er **interventie-aanvragen** binnen (een herstelling, extra materiaal, een levering), en vóór het seizoen moet elke ploeg haar **keuzes** doorgeven (op welke festivals, waar leveren, wie is verantwoordelijk, speciale vragen).

Vroeger liep dat via losse spreadsheets, Google Forms en mails. RWCrew brengt dat samen in één applicatie met één gebruikersbeheer, één set stamgegevens (ploegen, festivals, seizoenen, producten) en duidelijke rechten per gebruiker.

3.2 Het startscherm en de modules

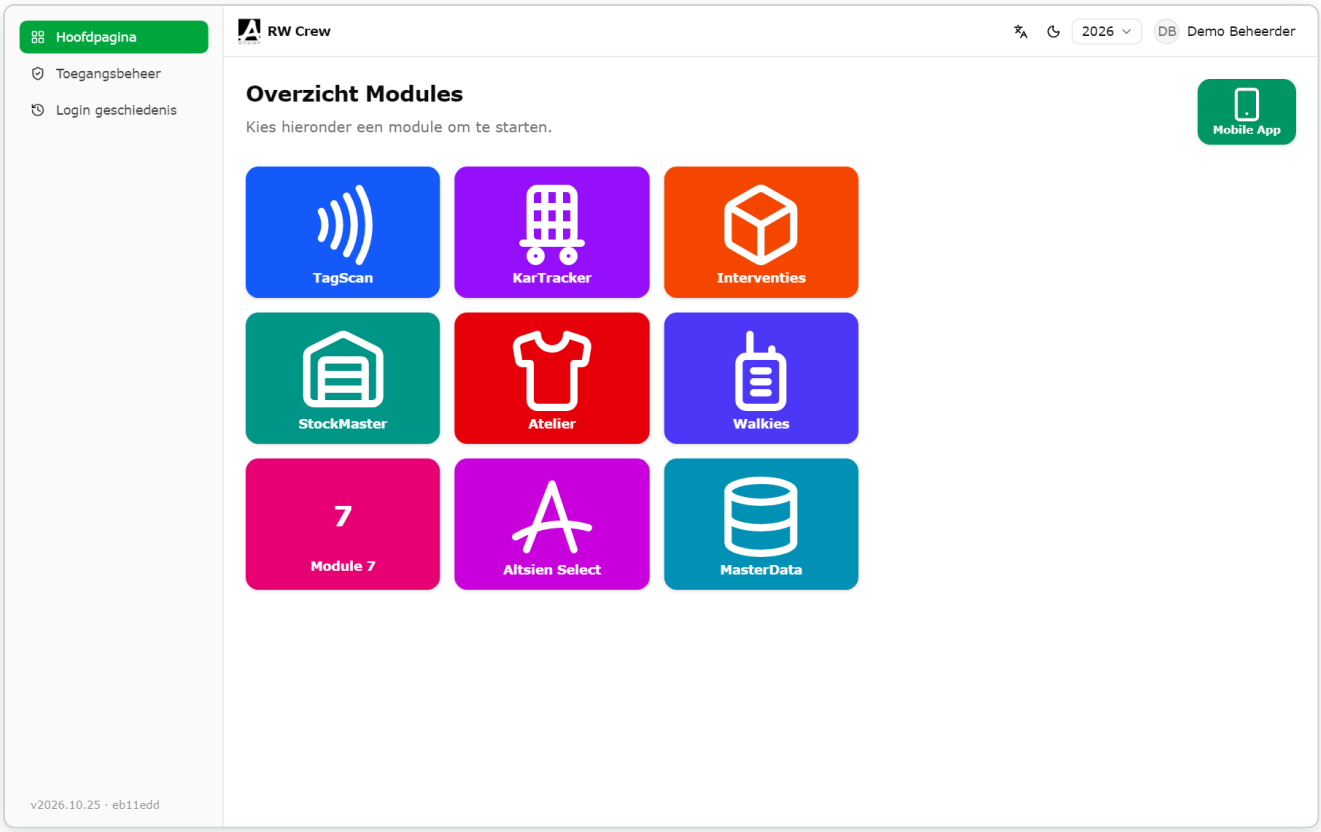
Na het inloggen kom je op het **startscherm** ("Modules Overview"): een raster met een gekleurde tegel per module. Je ziet alleen de tegels van modules waartoe de superbeheerder je toegang gaf. De module "Mobile App" verschijnt als kleine tegel rechtsboven, omdat ze geen echte module is maar een installatiepagina voor de telefoon-app.

Bovenaan elk scherm staat een balk met de naam van de app, de **seizoenkiezer** (alleen de open seizoenen, voor wie toegang heeft tot MasterData), de taalkeuze (NL/EN), de keuze tussen licht en donker thema (standaard donker) en het gebruikersmenu (wachtwoord wijzigen, uitloggen). De **superbeheerder** ziet daarnaast links een beheermenu met "Manage Access" en "Login History" (zie hoofdstuk 18).

Tabel 3.1 – De modules van RWCrew

Sleutel	Naam	Waarvoor	Telefoon
`module-1`	**Scans**	RFID-tags: CSV-bestanden van scanners inlezen, tag-gegevens bekijken, tags beheren, scanners en hun API-sleutels registreren.	Nee

`module-2`	**KarTracker**	Karren (rolcontainers): registratie, statussen, distributiepunten, zones, afleverlocaties, planning per festival, karbladen met QR-code, kaart, grondplan, manuele bewegingen (scannen), leveringsvolgorde, planningsanalyses (initiële levering, verplaatsingen tussen festivals, speciale levertoeren) en de karcode van het slot.	Ja: KarScan, Kar Planning, Kar Map, Karcode Simulator
`module-3`	**Interventie Aanvragen** (Intervention Requests)	Aanvragen van ploegen tijdens het festival: registreren, opvolgen met statussen, toewijzen aan TeamKar, mail met PDF-leveringsbon naar een mailinglijst. Ook een publiek formulier via QR-code, zonder inloggen.	Ja: Akties (lijst, nieuw, wijzigen), KPI
`module-4`	**StockMaster**	Magazijnvoorraad van de producten (de productstamgegevens worden sinds 24 oktober 2026 hier beheerd): vrije voorraad op de vaste plaats van een product en voorraad geladen in karren. Inboeken, uitboeken, karren laden en uitladen, kar vertrekt en kar terug, tellingen, benodigdheden per kar, "Te bestellen", PDF's.	Nee
`module-5`	**Atelier**	Werkkledij (T-shirts, hoodies, …) voor festivalmedewerkers: artikelen en maten, reservaties per festival en ploeg, stock met wascyclus, de balie (ophalen en terugbrengen) en polsbandscans via een webhook van het polsbandbedrijf.	Nee
`module-6`	**Walkies**	Beheer van de walkietalkies. Nu enkel een lege module met KPI-pagina en eigen rechten; de inhoud volgt later.	Nee
`module-7`	Module 7	Plaatshouder voor een toekomstige module: toont een "binnenkort"-pagina.	Nee
`module-8`	**Altsien Select**	De "Ploeg Wizard": per ploeg en per seizoen doorloopt een Altsien-kernlid stappen (festivals, afleverlocaties, ploegverantwoordelijken, producten, speciale vragen, walkietalkies). Resultaat op de "Ploegfiche" (scherm en PDF). Opvolging van speciale vragen.	Nee
`module-9`	**MasterData**	Stamgegevens die alle modules delen: seizoenen (jaren), festivals, ploegen (met hun speciale levertoeren), ploeglocaties, leveringswijzen, ploegtaken, ploegverantwoordelijken. Bulk import/export via Excel. (De producten staan sinds 24 oktober 2026 in StockMaster.)	Nee
`module-10`	**Mobile App**	Installatiepagina voor de telefoon-app: het adres met QR-code en de stappen voor iPhone en Android.	(is de app)



Figuur 3.1 – Het startscherm voor een gebruiker met toegang tot alle modules. Rechtsboven de kleine tegel van de Mobile App; bovenaan de seizoenkiezer, taal, thema en gebruikersmenu.

3.3 De modules in meer detail

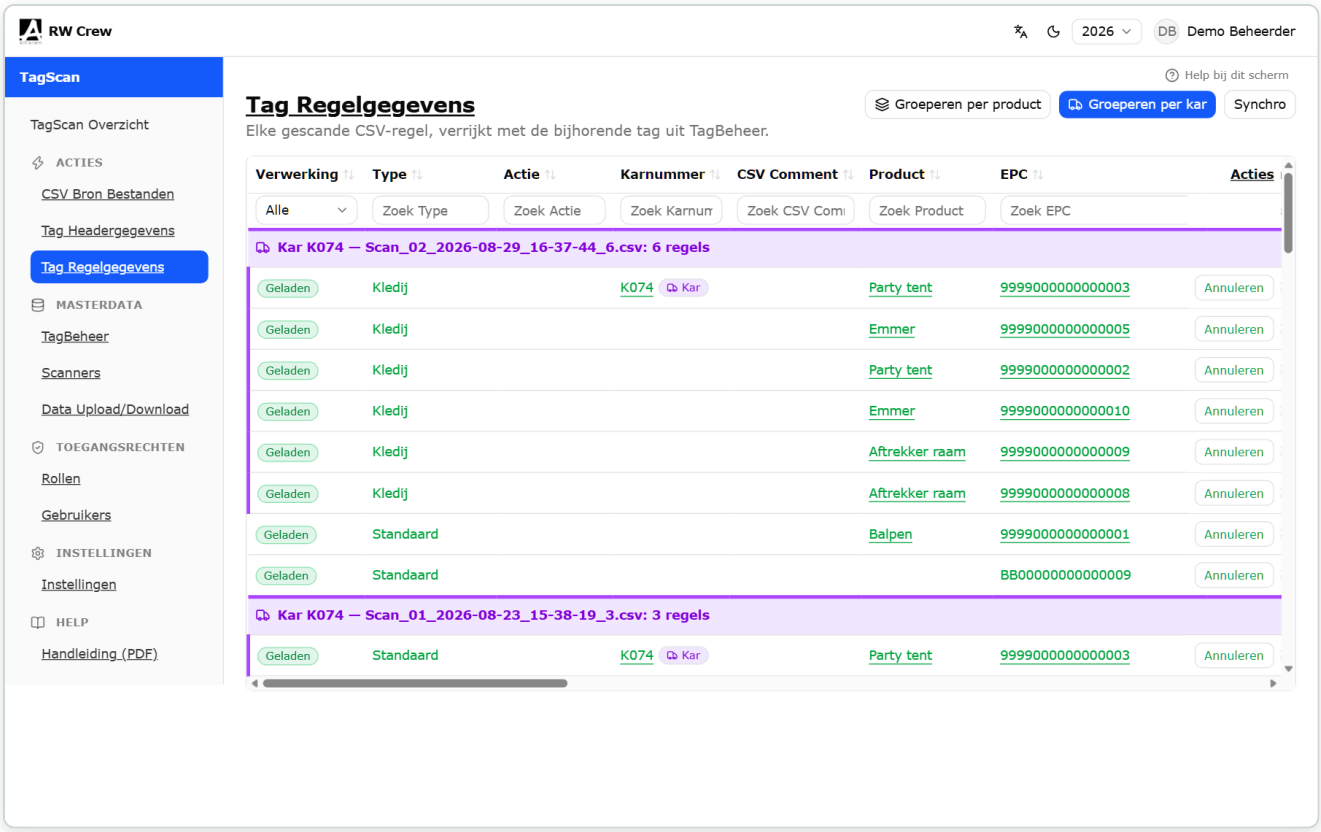
3.3.1 Scans (module 1)

Draagbare RFID-scanners schrijven hun scans weg als CSV-bestanden. Een **Raspberry Pi** bij de scanners stuurt elk afgewerkt bestand via HTTPS naar RWCrew (endpoint `POST /api/public/tagscan-intake`, beveiligd met een API-sleutel per scanner). De bestanden komen terecht in de map "Unreaded Tags" en worden ingelezen wanneer een gebruiker op de knop **Scan** drukt. Daarna zijn ze zichtbaar als *Tag Headerdata* (één rij per bestand) en *Tag Linedata* (één rij per gescande tag). Onder *TagManagement* beheer je het register van tags (ook via Excel-import), onder *Scanners* registreer je scanners en maak je hun API-sleutels aan.

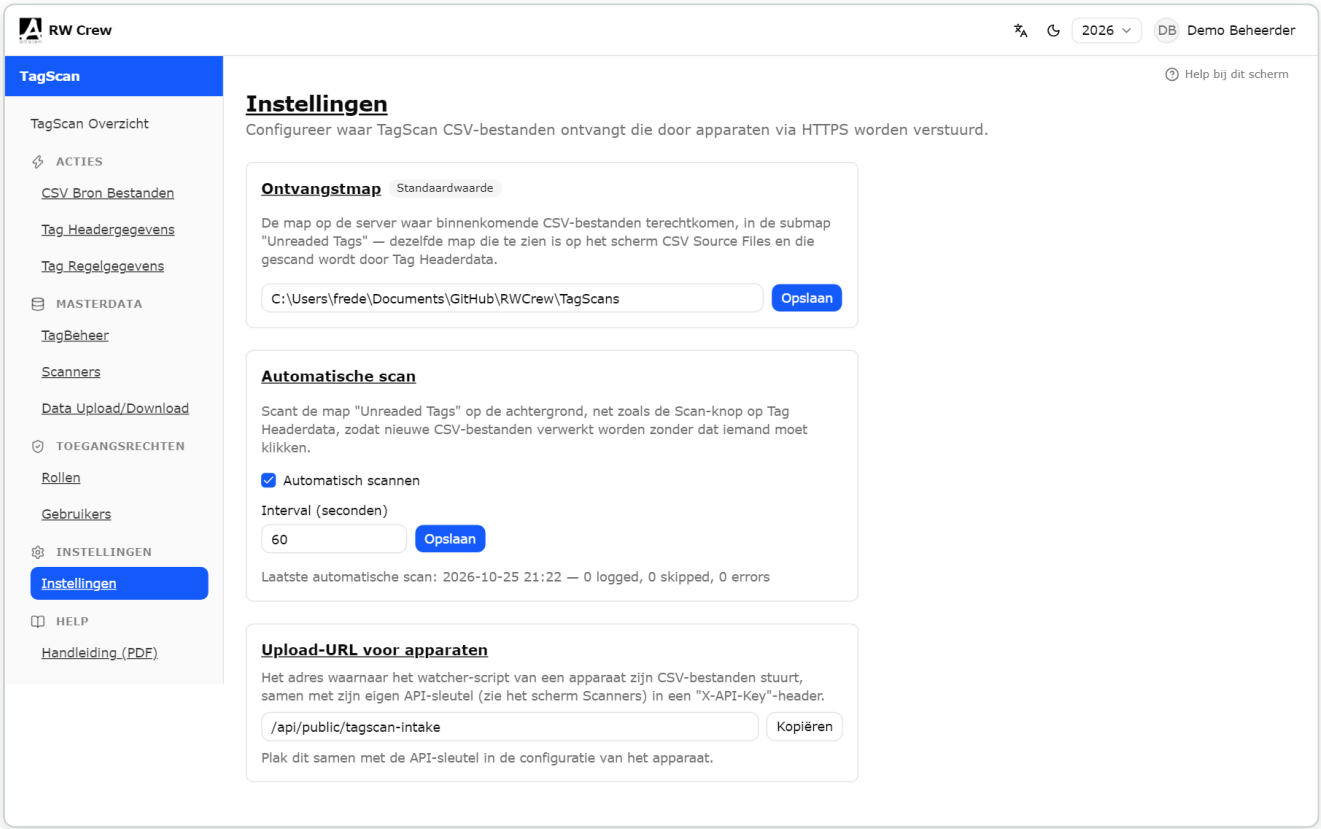
Automatisch scannen. De backend kan zelf op de achtergrond scannen: een taak die bij het opstarten begint, leest elke minuut (instelbaar op het scherm *Instellingen* van Scans) de nieuwe bestanden in, net alsof iemand op **Scan** drukt. Een vergrendeling voorkomt dat de automatische en een manuele scan tegelijk lopen.

Acties verwerken. Elke ingelezen regel heeft naast zijn EPC-status ook een **verwerkingsstatus**: *nieuw* (oranje), *verwerkt* (groen) of *geannuleerd* (rood). Als de kolom `Action` van een regel een actie bevat die RWCrew kent, verschijnt boven elk Scans-scherm een oranje balk "acties wachten op verwerking". Daarin kies je per bestand wat er moet gebeuren en verwerk je alle regels in één keer, of annuleer je een regel met een reden. Vandaag kent RWCrew één actie, `assignment`: tags registreren en aan een **product** koppelen (het product kies je per bestand). Nieuwe acties worden toegevoegd in `module_1/line_processing.py`.

Doorsturen tussen test en productie. Een CSV-bestand heeft een kolom `Mode` (`TEST` of `PROD`). Komt een bestand binnen op de "verkeerde" omgeving, dan stuurt de backend het automatisch door naar de andere (zie tabel 5.4). Zo kan één Raspberry Pi beide omgevingen bedienen.



Figuur 3.2 – Scans · Tag Regelgegevens: één rij per gescande tag. De kleur van de eerste kolom toont de verwerkingsstatus (groen = verwerkt).

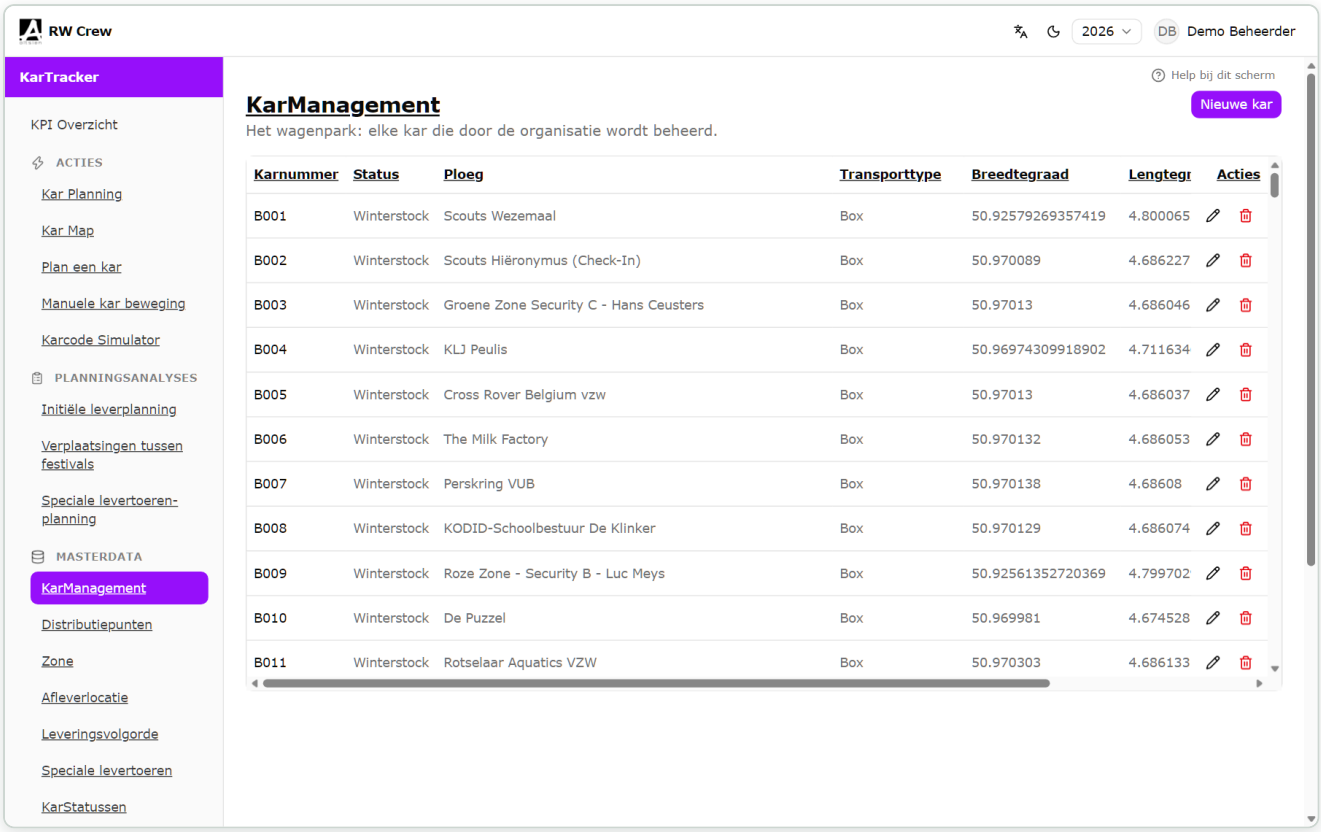


Figuur 3.3 – Scans · Instellingen: de ontvangstmap, het automatisch scannen met zijn interval en het adres waarop apparaten hun CSV-bestanden sturen.

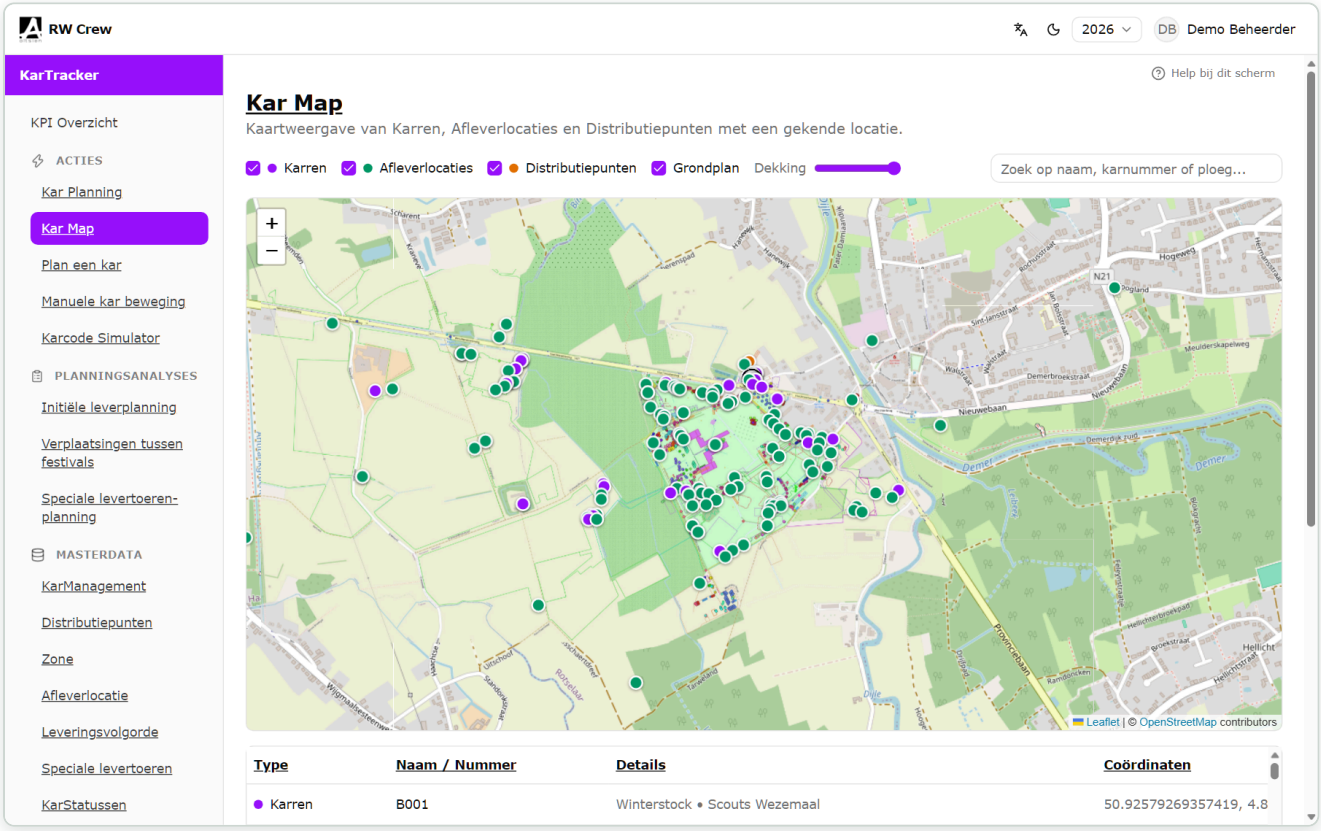
3.3.2 KarTracker (module 2)

KarTracker volgt de **karren**: genummerde rolcontainers die per seizoen aan een ploeg toegewezen worden en op festivals afgeleverd en opgehaald worden. Belangrijkste schermen:

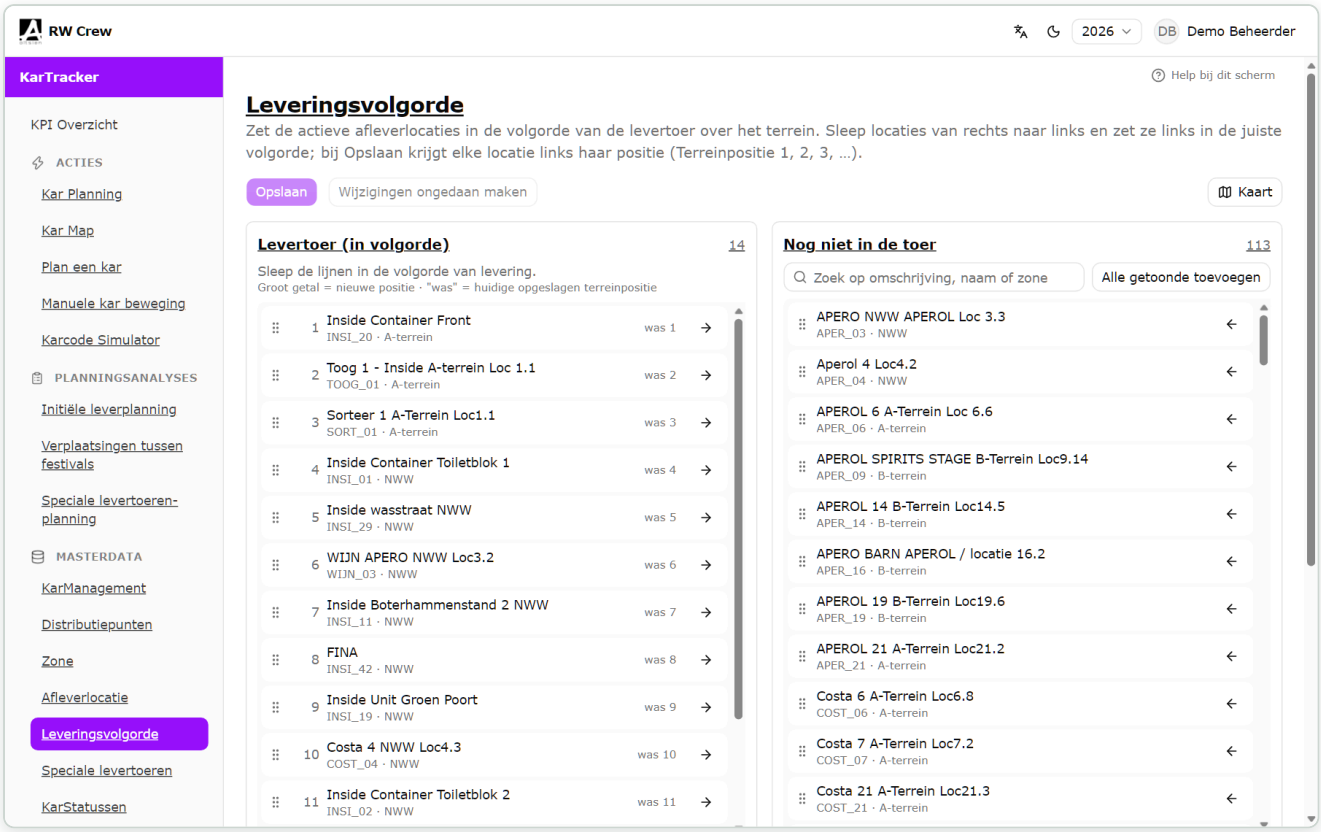
- **KarManagement** en **KarStatussen**: het karrenregister en de mogelijke statussen.
- **Distributiepunten**, **Zones**, **Afleverlocaties** en **Delivery Dates**: de plaatsen en momenten van levering.
- **Plan a kar**: per ploeg, festival en seizoen de afleverlocatie kiezen. Dezelfde gegevens worden ook vanuit Altsien Select ingevuld.
- **Kar Planning**: het overzicht per festival, met het afdrukken van **karbladen**: één A4 per kar met twee QR-codes (het karnummer om te scannen, en een link naar het publieke interventieformulier voor die ploeg).
- **Kar Map** en **Grondplan**: de karren op een kaart (OpenStreetMap via Leaflet) en op een geüpload grondplan.
- **Manuele kar beweging**: een beweging van een kar registreren, op de pc of met de telefooncamera (KarScan).
- **Data Upload/Download**: bulkgegevens via Excel.
- **Leveringsvolgorde**: de volgorde van de ene levertoer over het terrein, door afleverlocaties te slepen (`KarTracker_afleverlocaties.terrein_positie` , een geheel getal; 1 = eerst geleverd), met een kaart van de route.
- **Planningsanalyses**: *Initiële leverplanning* (welke karren bij de start waar geleverd worden, per distributiepunt en in tourvolgorde), *Verplaatsingen tussen festivals* (karren waarvan de afleverlocatie tussen twee festivals verandert) en *Speciale levertoeren-planning* (de stops van bv. de wastoer). Elk scherm drukt een PDF af, optioneel met een kaart op het grondplan die de backend zelf tekent.
- **Speciale levertoeren**: toeren zoals "Kleine wastoer" die regelmatig langs ploegen komen; de koppeling met ploegen gebeurt op de ploegfiche in MasterData.
- **Karcode Simulator** en **Instellingen**: de code van het slot van een kar volgt uit het karnummer via één formule in Excel-stijl (bv. `LEFT(kar*500+3118; 4)`), die enkel beheerders zien en wijzigen. De simulator (ook op de telefoon) toont de code, na het intypen of scannen van het karnummer.



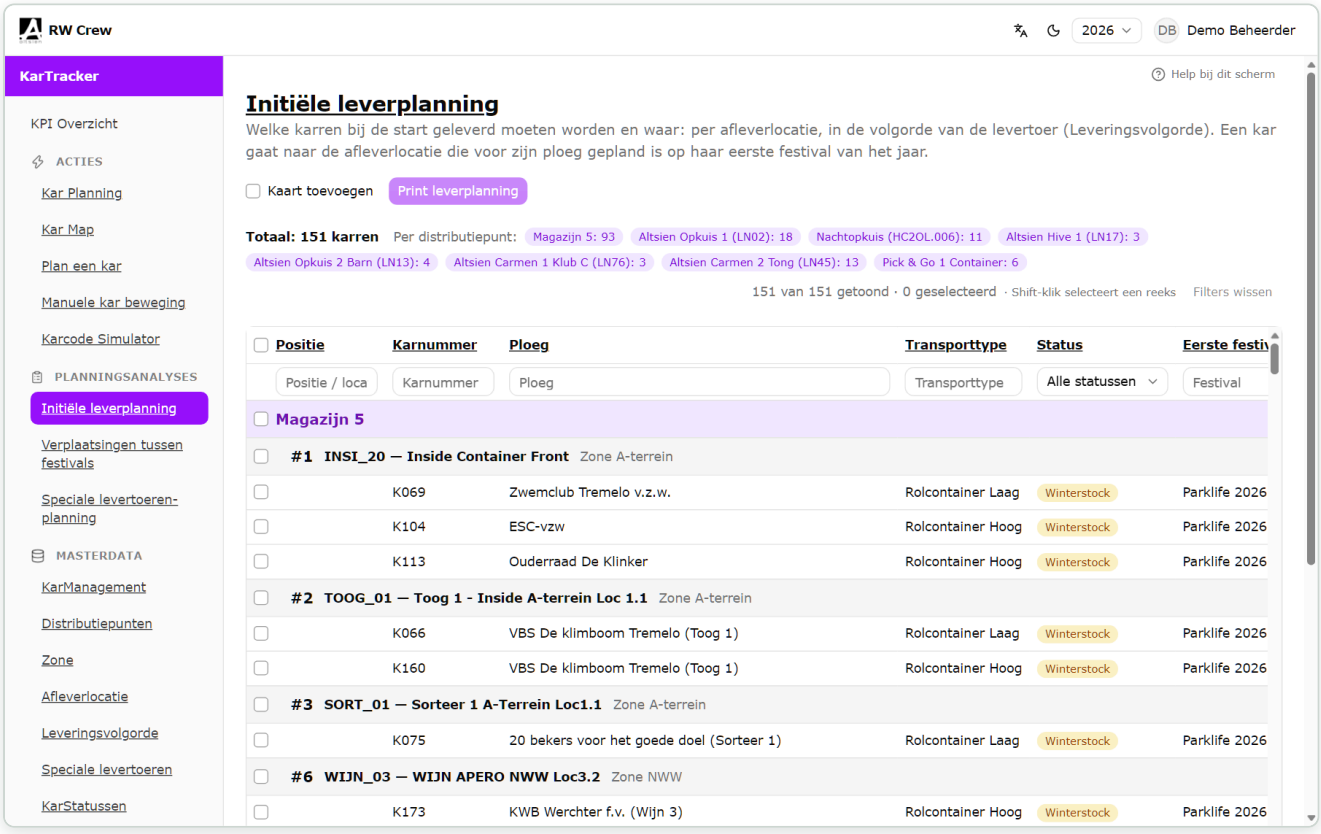
Figuur 3.4 – KarTracker · KarManagement: het karrenregister met status, ploeg, transporttype en laatst gekende locatie.



Figuur 3.5 – KarTracker · Kar Map: karren, afleverlocaties en distributiepunten op de kaart (OpenStreetMap), met een tabel van de getoonde punten eronder.



Figuur 3.6 – KarTracker · Leveringsvolgorde: links de levertoer (slepen om te ordenen, met de vorige positie ernaast), rechts de actieve afleverlocaties zonder positie.



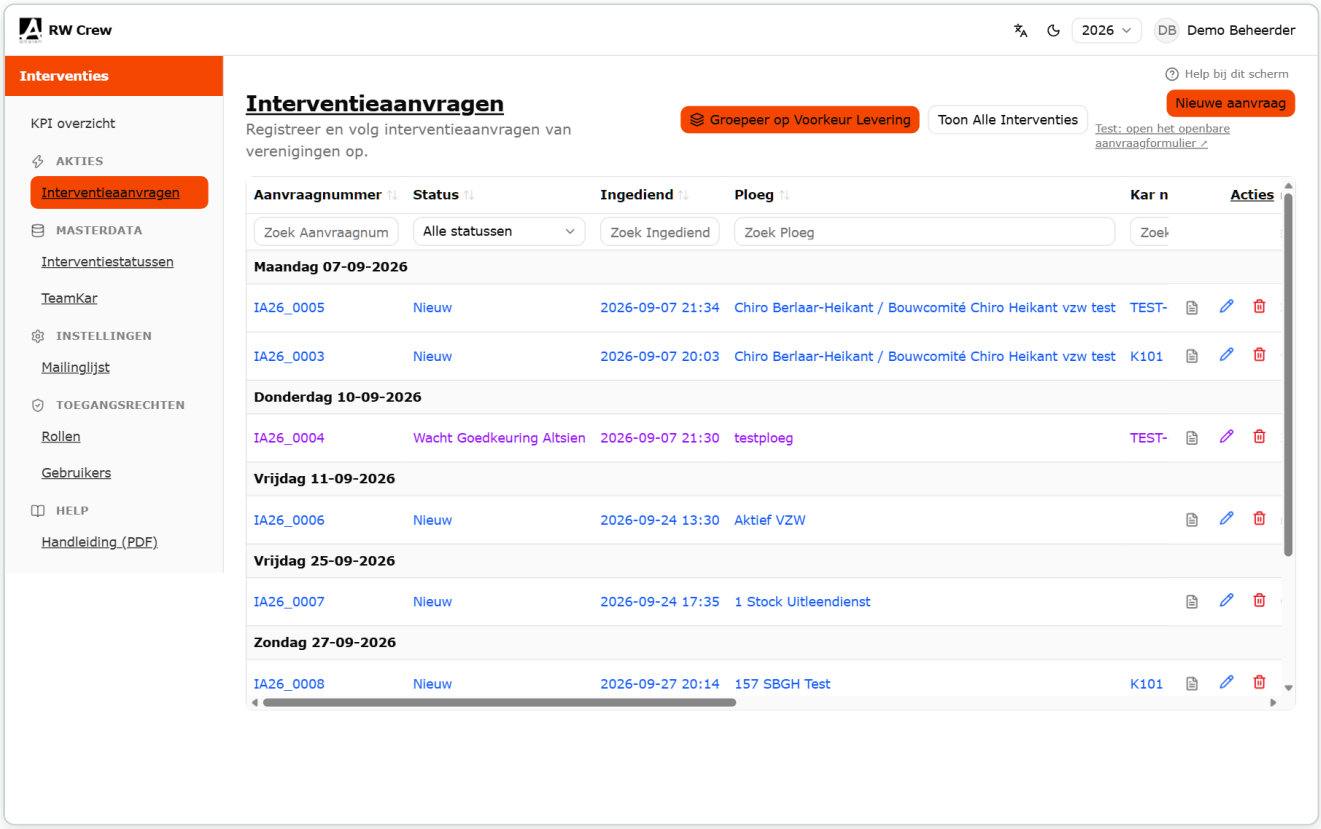
Figuur 3.7 – KarTracker · Initiële leverplanning: per distributiepunt en afleverlocatie (in tourvolgorde) de karren die bij de start geleverd worden, met filters en selectie om af te drukken.

3.3.3 Interventie Aanvragen (module 3)

Tijdens een festival vraagt een ploeg om hulp: een herstelling, bijkomend materiaal, een levering. Zo'n **interventie-aanvraag** krijgt een nummer (`IA26_0001` : "IA", de laatste twee cijfers van het jaar en een volgnummer) en wordt opgevolgd met een **status** (een zelf te beheren lijst met kleuren). Aanvragen komen op drie manieren binnen:

1. via het scherm *Intervention Requests* door een medewerker;
2. via de telefoon-app (tabblad "Akties");
3. via het **publieke formulier** `/nl/intervention-request`, zonder inloggen, bereikbaar via de QR-code op het karblad (ploeg, karnummer, afleverplaats en zone zijn dan al ingevuld).

Bij elke nieuwe aanvraag krijgen alle actieve adressen op de **mailinglijst** (Instellingen > Mailinglijst) een mail met de gegevens en de leveringsbon als PDF in bijlage. *TeamKar* bepaalt welke medewerkers aan een aanvraag gekoppeld kunnen worden als uitvoerder van de leverkar.



Figuur 3.8 – Interventie Aanvragen: de lijst van aanvragen, gegroepeerd per dag, met de status in kleur en per regel PDF, wijzigen en verwijderen.

3.3.4 StockMaster (module 4)

StockMaster houdt de **magazijnvoorraad** bij van de producten uit MasterData, in hele stuks. Een stuk ligt op één van twee plaatsen:

- **vrije voorraad**: op de vaste plaats van het product in het magazijn (magazijn + locatie, ingesteld in MasterData);
- **in een kar**: geladen in een KarTracker-kar, zolang die kar in het magazijn staat.

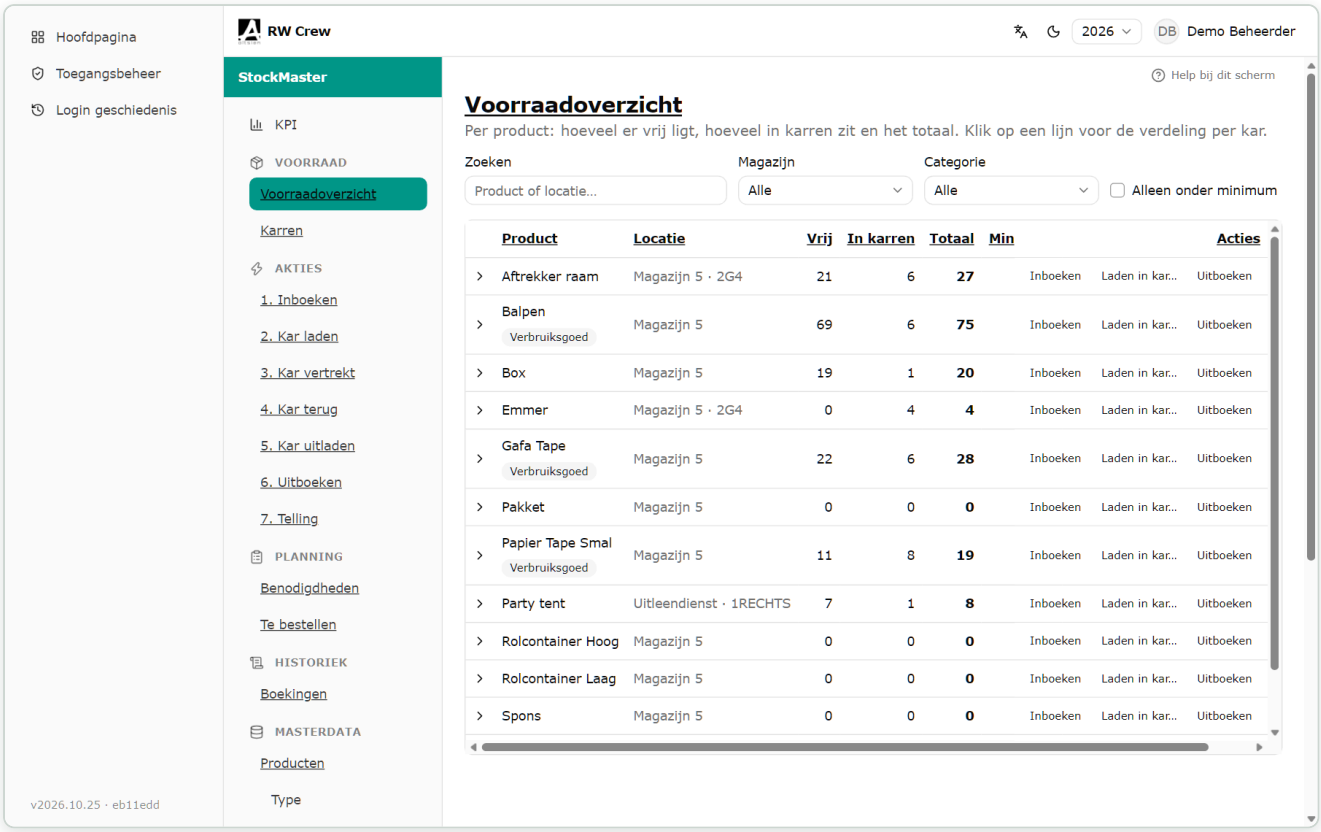
Hetzelfde product kan tegelijk vrij liggen en in meerdere karren zitten. Het voorraadoverzicht toont per product *Vrij · In karren · Totaal* en klapt open tot de verdeling per kar.

Alles wat de voorraad verandert, is een **boeking** met een nummer (bv. `SM-2026-00002`), vanuit één van de acties in het menu: *Inboeken*, *Kar laden*, *Kar vertrekt*, *Kar terug*, *Kar uitladen*, *Uitboeken* en *Telling*. Belangrijke regels:

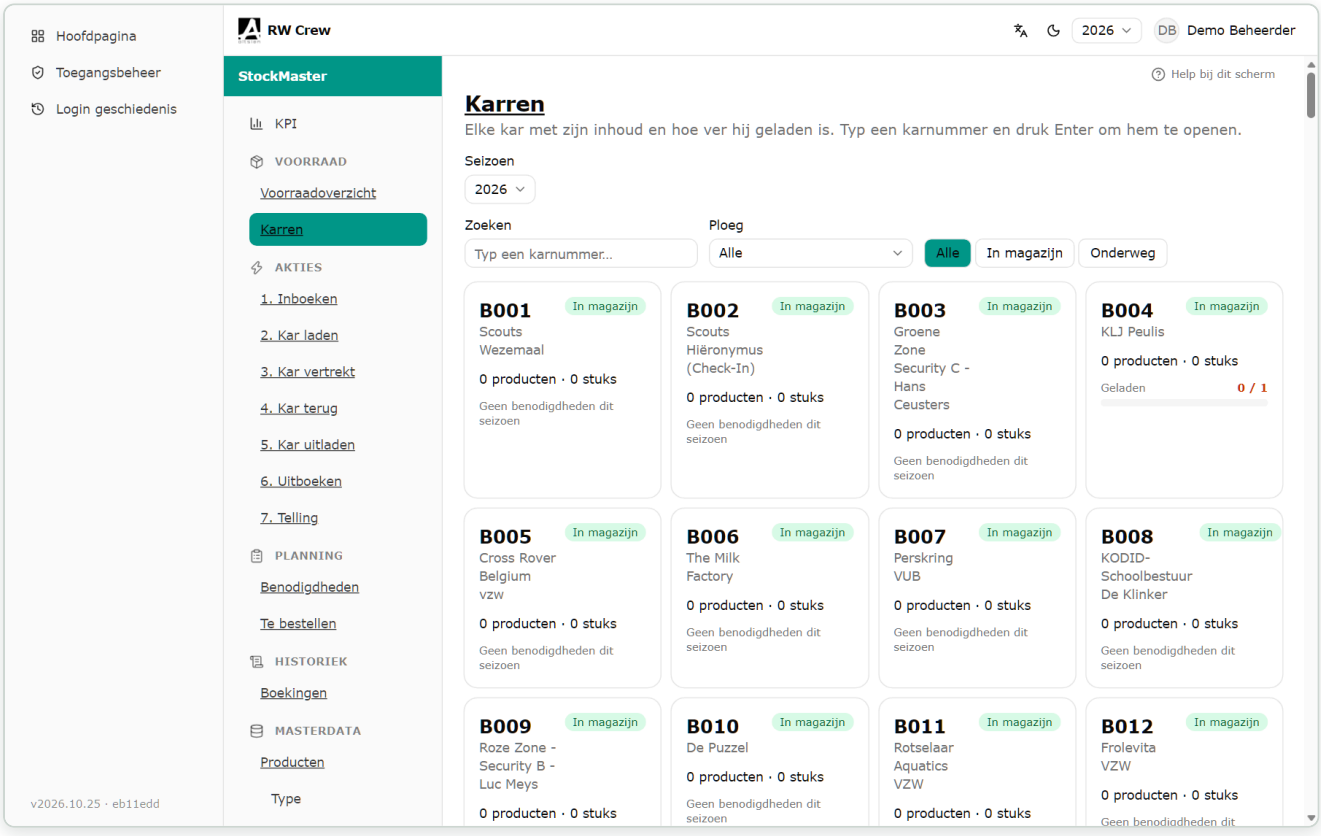
- **Nooit negatief.** Je kunt niet meer uitboeken of laden dan er ligt; twee gebruikers kunnen niet tegelijk het laatste stuk nemen.
- **Niets wordt gewist.** Een fout herstel je met "Ongedaan maken": dat maakt een tegenboeking. Zo blijft altijd zichtbaar wie wat wanneer deed.
- **Altijd onder je eigen naam.** De gebruiker van een boeking is altijd de ingelogde gebruiker, nooit een keuze in het formulier.
- **Karvoorraad bestaat alleen in het magazijn.** "Kar vertrekt" boekt de hele inhoud uit; "Kar terug" boekt wat terugkomt weer in (in de kar of als vrije voorraad). Wat niet terugkwam, telt als verbruik van de ploeg.
- **Gesloten seizoen.** Boeken in een seizoen dat niet meer open is, kan alleen met het recht "Boeken in een gesloten seizoen".

Onder *Benodigdheden* leg je per seizoen en per kar vast wat een ploeg nodig heeft. Dat vult "Kar laden" vooraf in, toont op de karkaarten hoe ver een kar geladen is ("18 / 20") en berekent *Te bestellen*: benodigdheden min wat er al in voorraad is. De benodigdheden worden ook door de Altsien-kernleden ingevuld (stap "Producten" van de Ploeg Wizard). Producten onder hun minimumvoorraad (verbruiksgoederen) geven een oranje waarschuwing bovenaan de module.

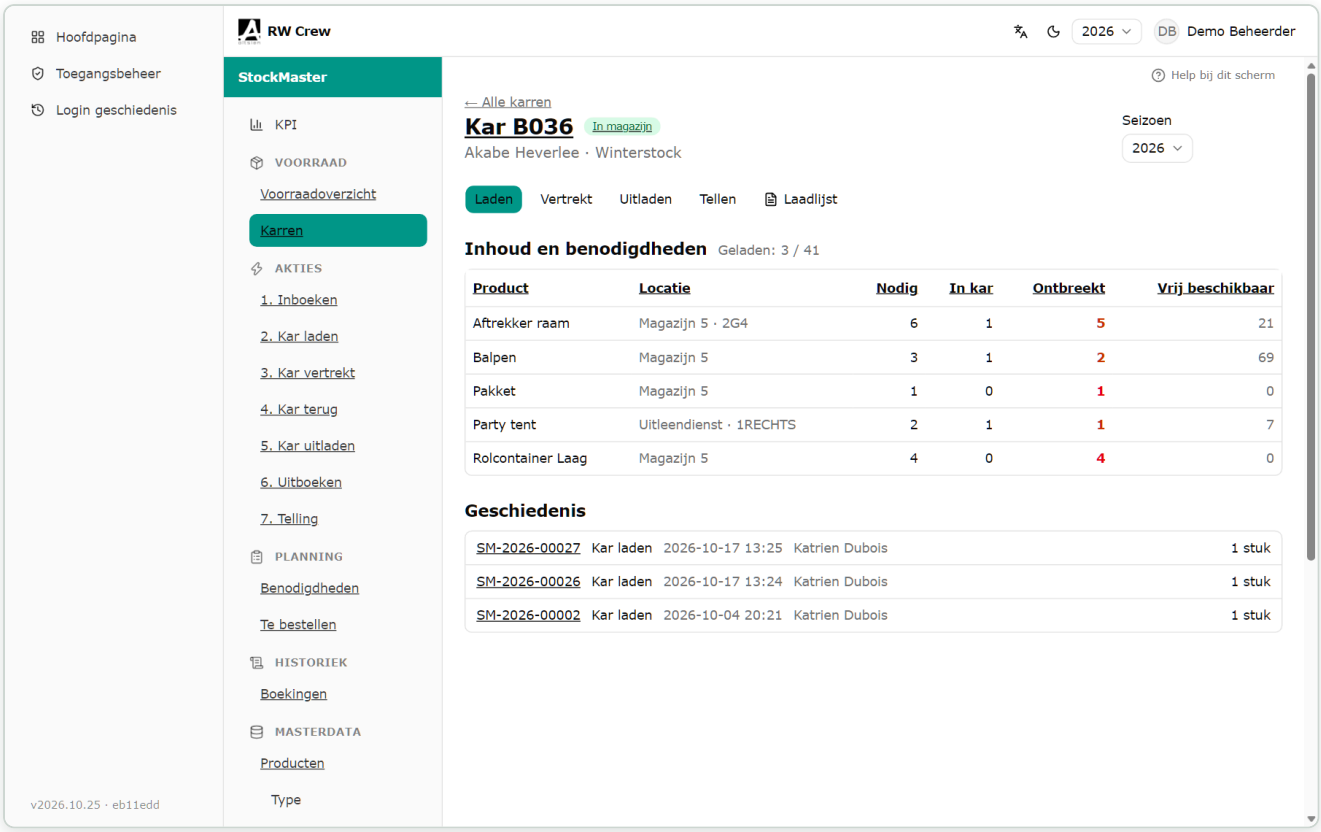
Sinds 24 oktober 2026 beheert StockMaster ook de **productstamgegevens** (menu *Masterdata*: producten met type, magazijn, categorie en limiet, plus Excel-import en -export); de tabellen houden hun naam `MasterData_product…`. Verder: *Inboeken* kan een Scans-scan met actie "Book In" overnemen (een melding bovenaan elk StockMaster-scherm), *Kar laden* toont per product wat "op dit moment beschikbaar is voor levering" volgens het stroomdiagram van de organisatie (limieten, blokkering, logistieke producten), met naast de benodigdheden ook *manuele aanvragen*, en een kar die al vertrok kan een *nalevering* krijgen. Elke actie heeft een PDF: laadlijst, vertrekbon, bestellijst, telblad en boekingsbon. Het volledige ontwerp staat in `docs/stockmaster-design.md`.



Figuur 3.9 – StockMaster · Voorraadoverzicht: per product de vrije voorraad, wat in karren zit en het totaal, met snelle acties per regel.



Figuur 3.10 – StockMaster · Karren: één kaart per kar met de status (in het magazijn of onderweg), de ploeg en hoe ver ze geladen is.



Figuur 3.11 – StockMaster · detail van één kar: inhoud tegenover de benodigdheden, de acties (laden, vertrekt, uitladen, tellen, laadlijst) en de geschiedenis.

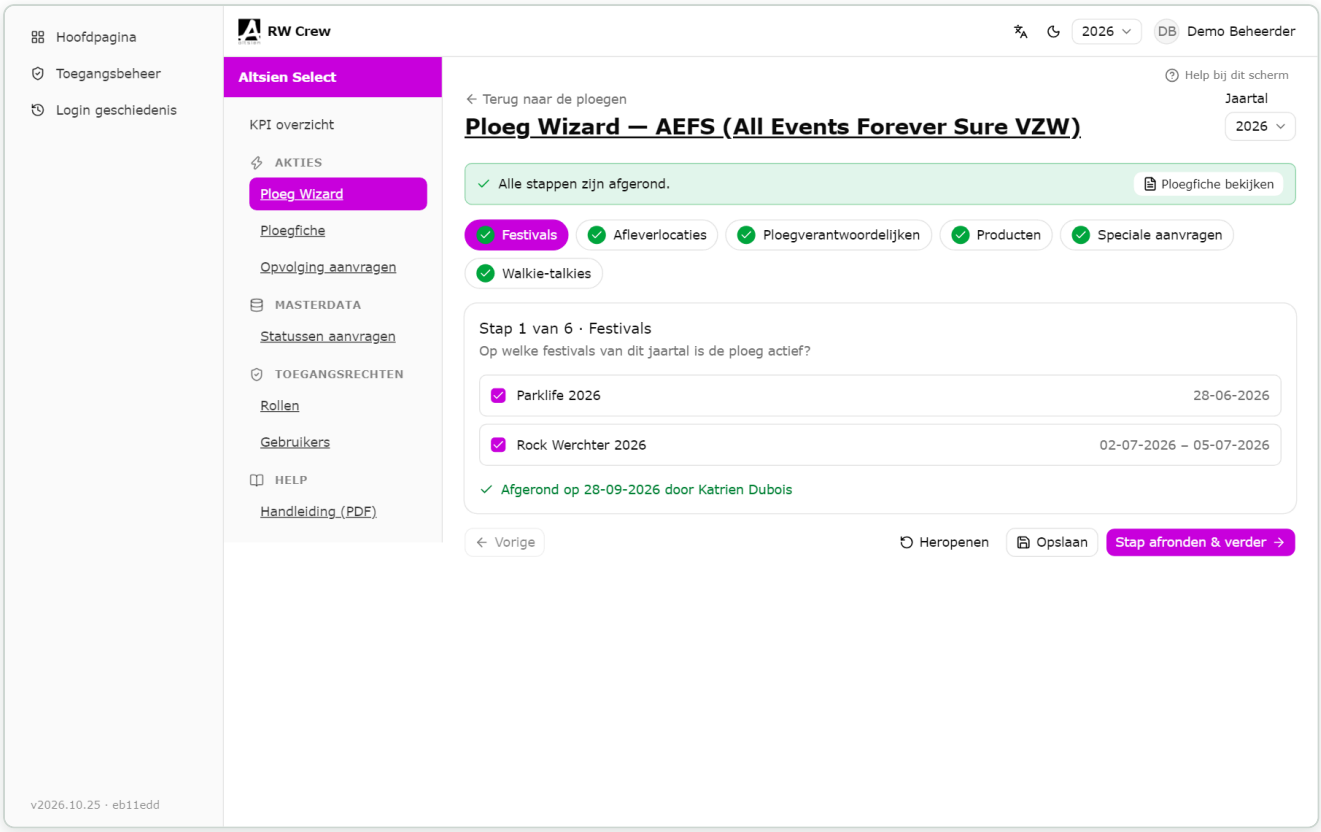
3.3.5 Altsien Select (module 8)

Vóór het seizoen maakt elke ploeg haar keuzes. Een **Altsien-kernlid** (een gebruiker met de vlag "Altsien Kernlid", gekoppeld aan één of meer ploegen) doorloopt daarvoor per ploeg de **Ploeg Wizard**:

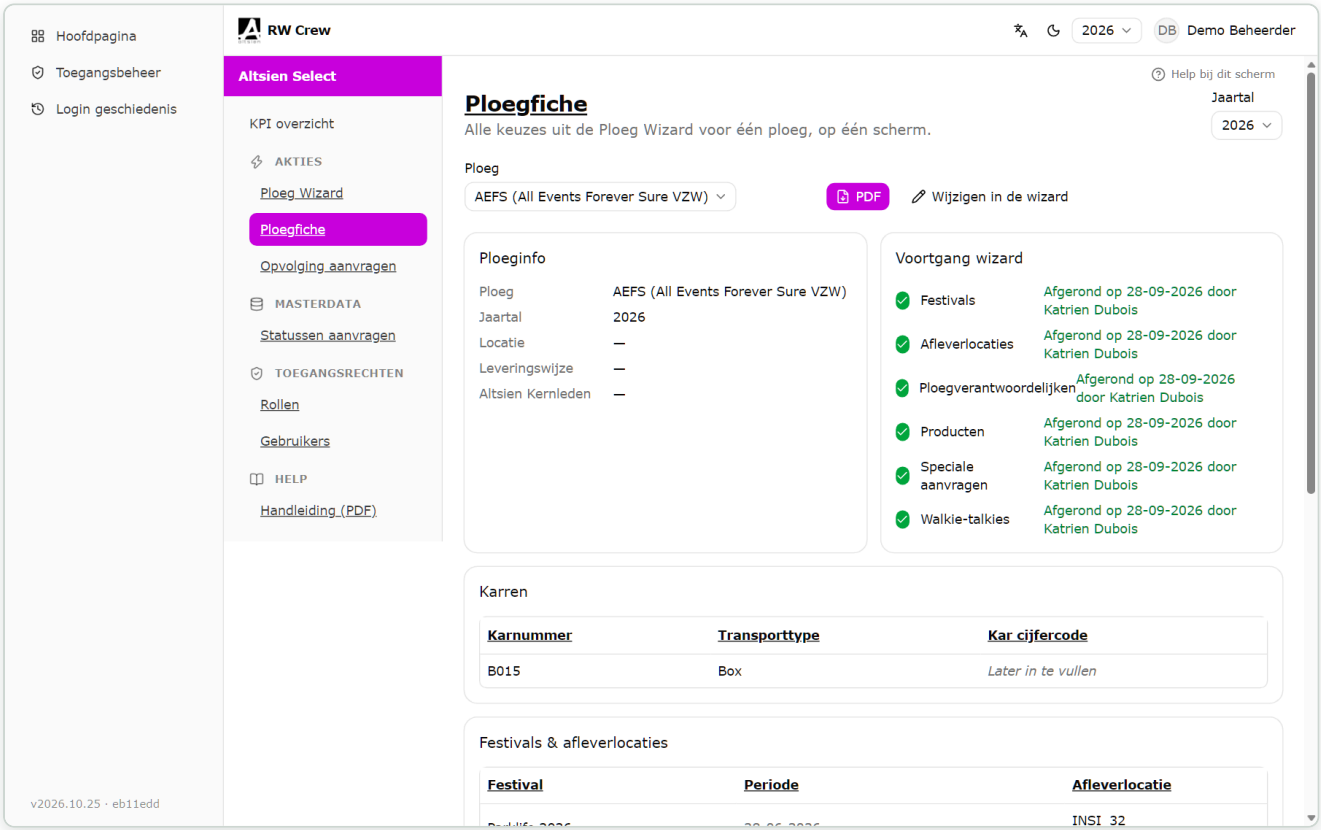
Tabel 3.2 – De stappen van de Ploeg Wizard

#	Stap	Wat wordt er gekozen	Waar opgeslagen
1	Festivals	Op welke festivals is de ploeg actief (minstens één)	MasterData (`MasterData_team_festival`)
2	Afleverlocaties	Per gekozen festival een afleverlocatie	KarTracker (Plan a kar)
3	Ploegverantwoordelijken	Minstens één verantwoordelijke met contactgegevens	MasterData (`MasterData_team_responsible`)
4	Producten	Per kar van de ploeg de benodigde producten	StockMaster (`StockMaster_kar_requirement`)
5	Speciale vragen	Vrije vragen, die daarna met een status opgevolgd worden	Altsien Select
6	Walkietalkies	Plaatshouder tot de walkiemodule bestaat	(nog niet)

Bij enkele stappen toont de wizard de keuzes van **vorig seizoen**, die je regel per regel kunt overnemen. Het resultaat van alle stappen staat op de **Ploegfiche**, die ook als PDF kan worden afgedrukt. Na het afsluiten van een seizoen (vlag *periode open* uit) kan niemand nog wijzigen, behalve wie het recht "organisatie" heeft (zie §7.4).



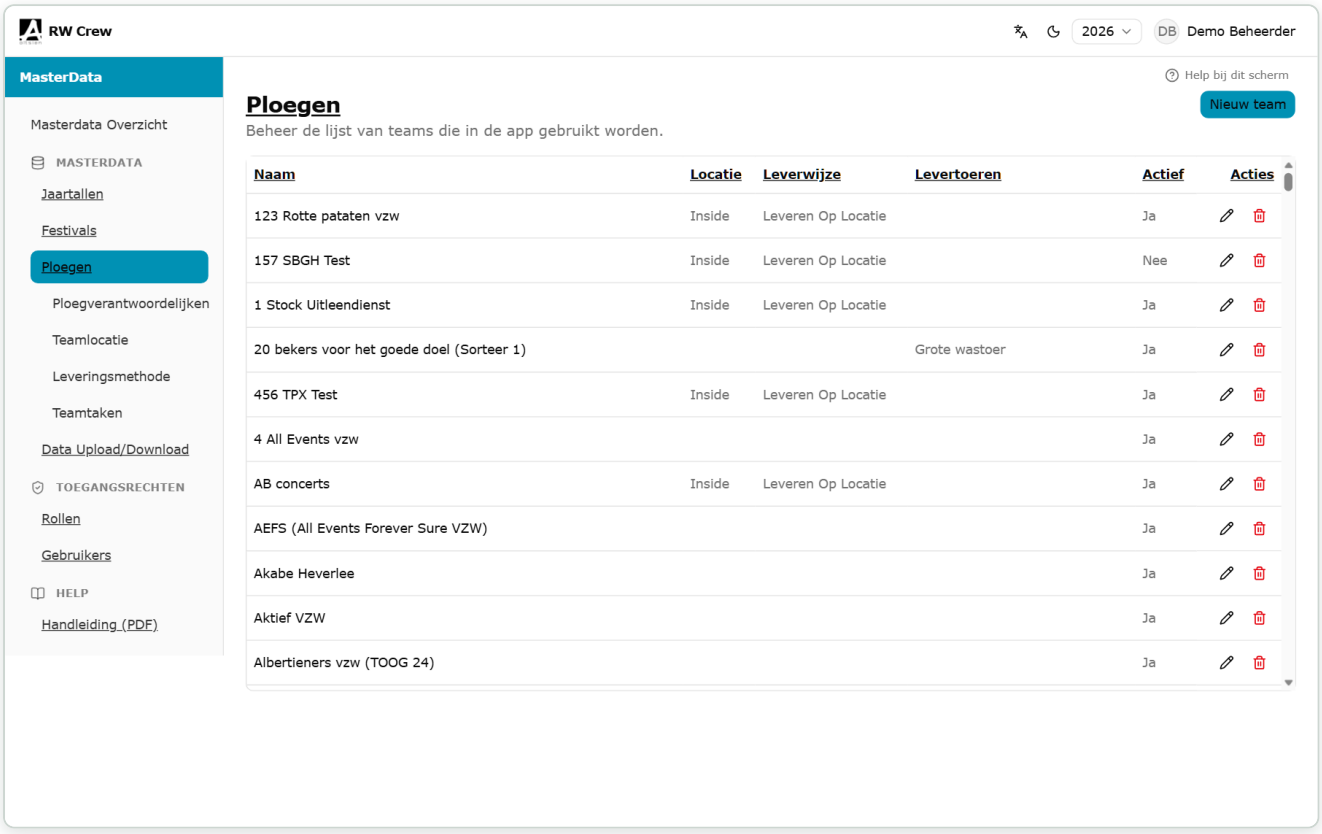
Figuur 3.12 – Altsien Select · Ploeg Wizard voor één ploeg: bovenaan de stappen met hun voortgang, eronder de huidige stap (hier: de festivals).



Figuur 3.13 – Altsien Select · Ploegfiche: alle keuzes van één ploeg op één scherm, ook als PDF af te drukken.

3.3.6 MasterData (module 9)

MasterData beheert de **stamgegevens**: de gegevens die alle andere modules delen en die zelden veranderen. Denk aan de seizoenen (op het scherm "Years"), festivals, ploegen en hun locatie, leveringswijze, taken, verantwoordelijken en speciale levertoeren. (De productcatalogus verhuisde op 24 oktober 2026 naar StockMaster.) Bijna elk scherm heeft een **Excel-import en -export**: je downloadt een sjabloon, vult het in en uploadt het; elke rij wordt apart gecontroleerd zodat één foute rij niet de hele import tegenhoudt.



Figuur 3.14 – MasterData · Ploegen: de stamgegevens van alle ploegen, met locatie, leveringswijze en of ze actief zijn.

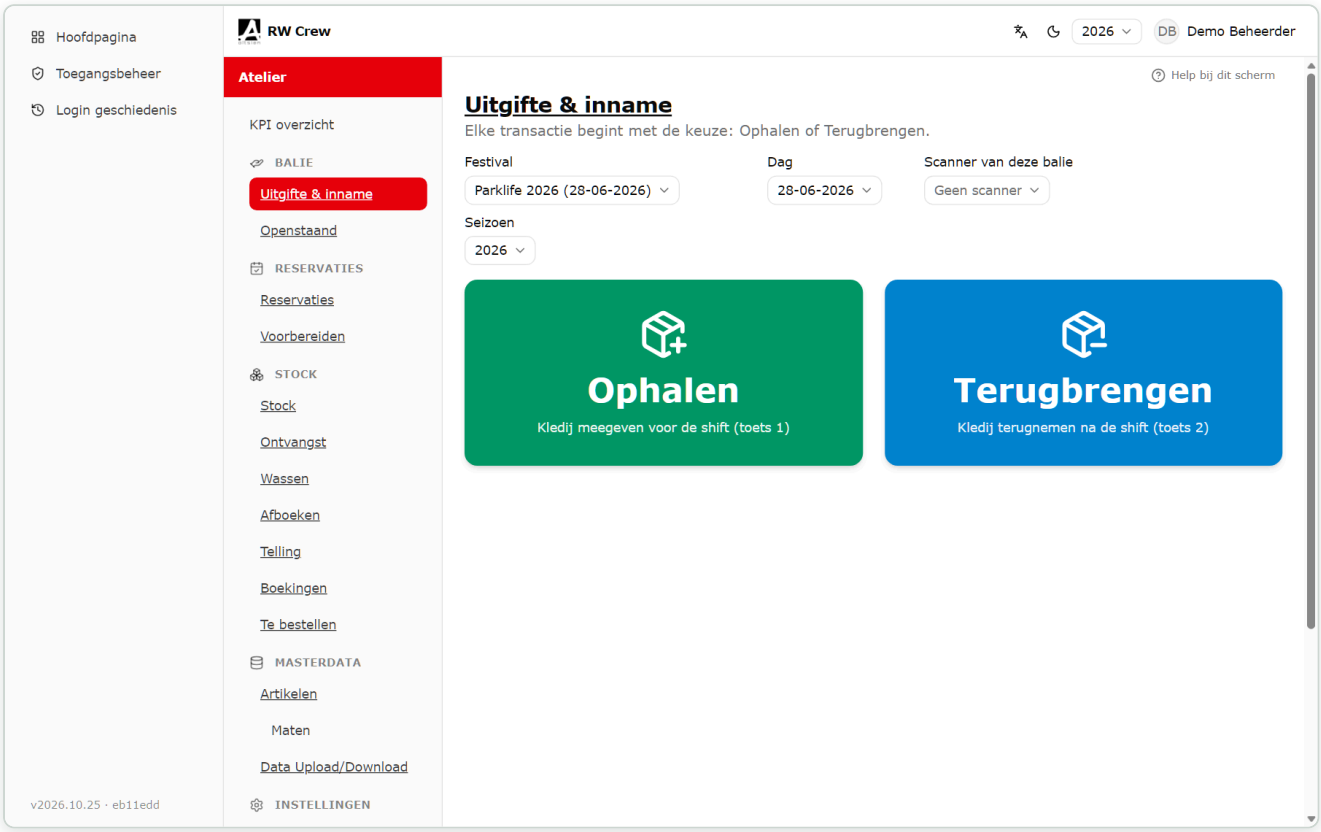
3.3.7 Atelier (module 5)

Medewerkers van ploegen die met eigen kledij werken (VIP, RW terras, …) halen bij het begin van hun shift **werkkledij** op aan de balie van het atelier en brengen **alles** na de shift terug. De medewerkers zijn geen RWCrew-gebruikers: ze horen bij een ploeg, en het pakket van die ploeg bepaalt wat ze krijgen. De module werd in blokken gebouwd:

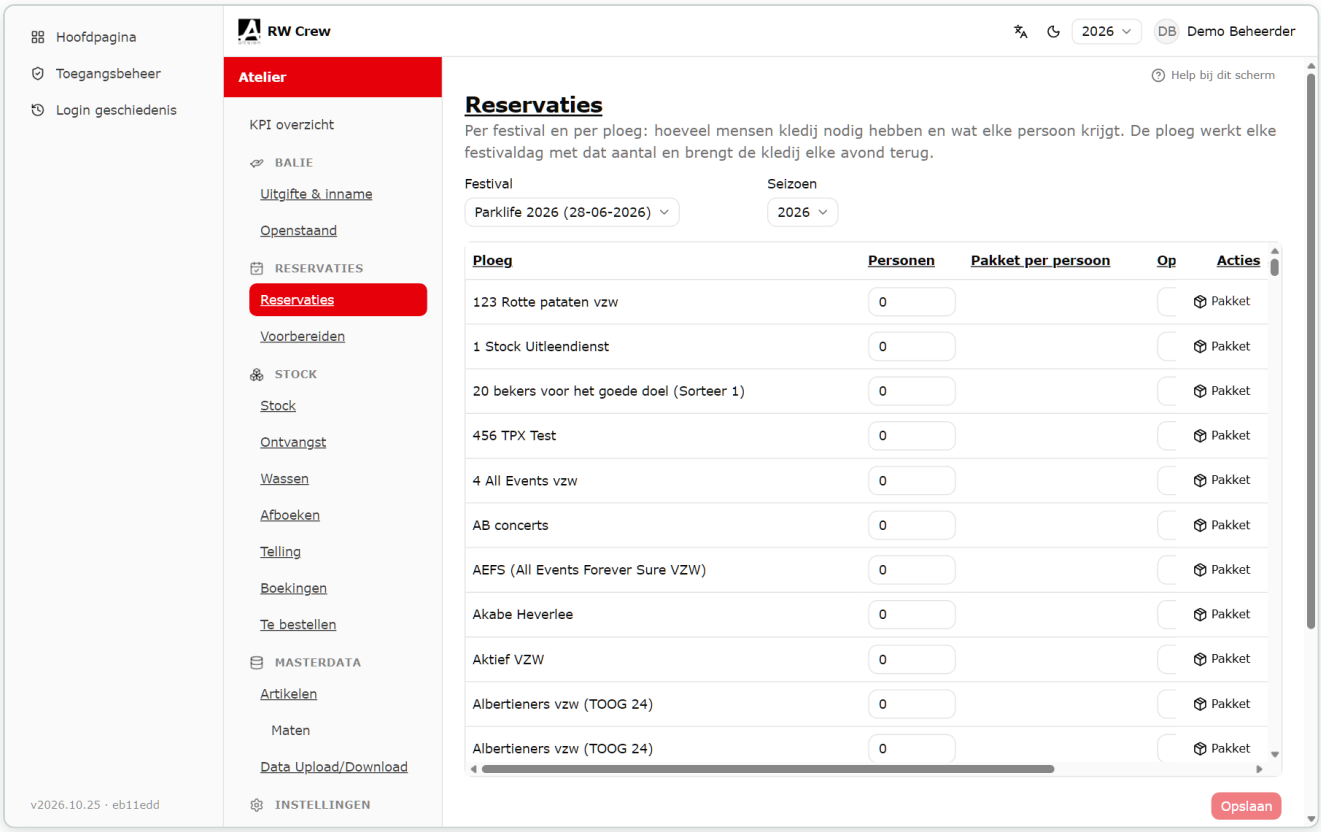
Tabel 3.3 – De blokken van Atelier

Blok	Wat
1 Masterdata	Eigen **artikelen** (geen StockMaster-producten) en **maten** (naam + pasvorm dames/heren/unisex). Een artikelmaat (`Atelier_article_size`) is de eenheid van de stock, met een maatverdeling in % voor de schattingen en een minimumstock.
2 Reservaties	Per festival en per ploeg het **aantal personen** en het **pakket** (artikelen, altijd 1 stuk per persoon; de maat wordt aan de balie gekozen). *Voorbereiden* berekent wat elke dag klaargelegd moet worden, met geschatte maten en een picklijst-PDF. "Kopiëren van vorig jaar" en een waarschuwing voor ploegen die op één festival vergeten werden.
3 Stock	Stuks per artikelmaat met de status *beschikbaar*, *te wassen* of *uitgegeven*: ontvangst, wassen, afboeken, telling, boekingen (met ongedaan maken) en "Te bestellen".

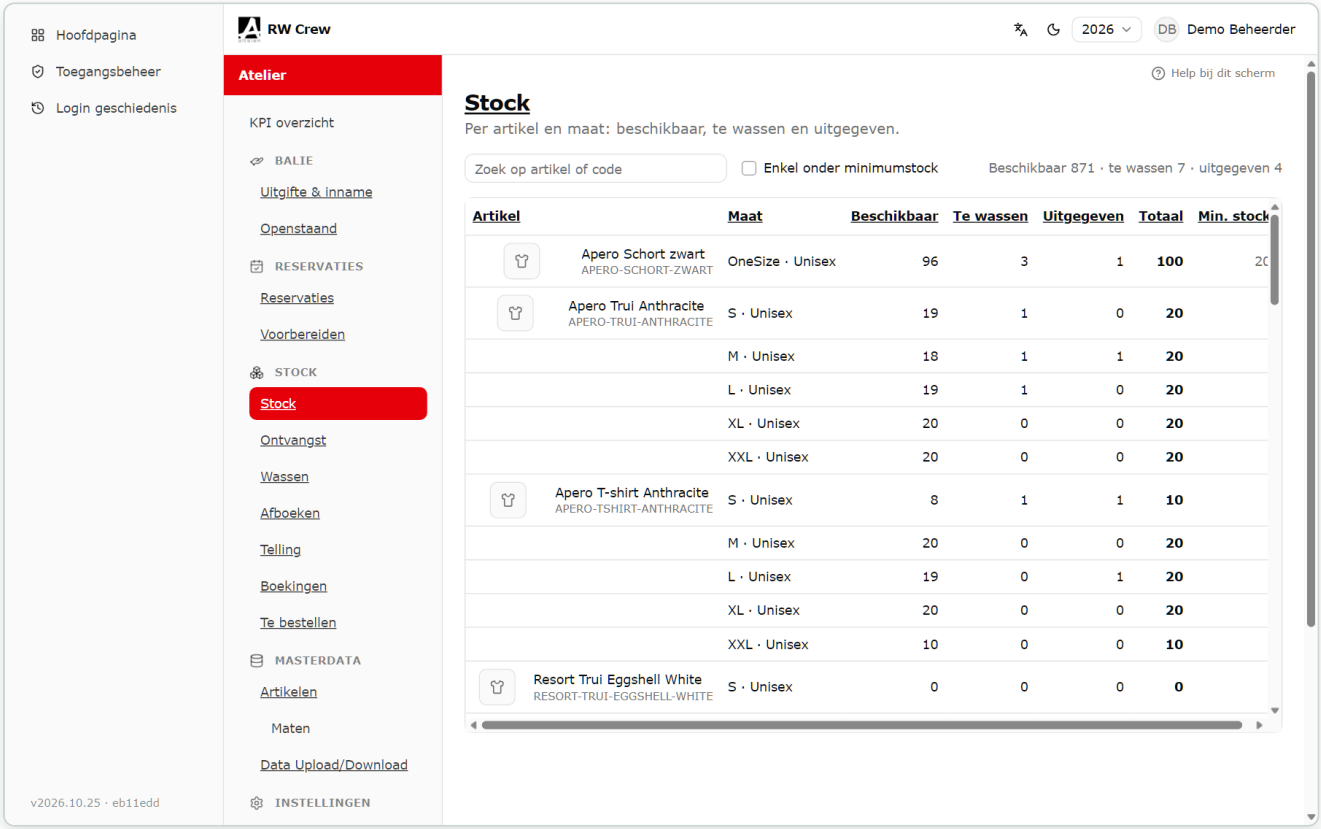
4 Balie	Elke transactie begint met de keuze **Ophalen** of **Terugbrengen**. Een persoon is gekend aan voornaam + familienaam; e-mail is optioneel en het polsbandnummer geldt per festival. "Openstaand" toont wat nog niet terug is.
5 Polsbandscans	Het polsbandbedrijf stuurt elke scan aan onze balie als JSON naar een **webhook**; de balie toont de persoon binnen enkele seconden en vult de transactie in. Een testscherm speelt berichten na tot de echte koppeling (vanaf mei 2027).
6 RFID	Elk kledingstuk opvolgen via Scans (nog niet gebouwd).



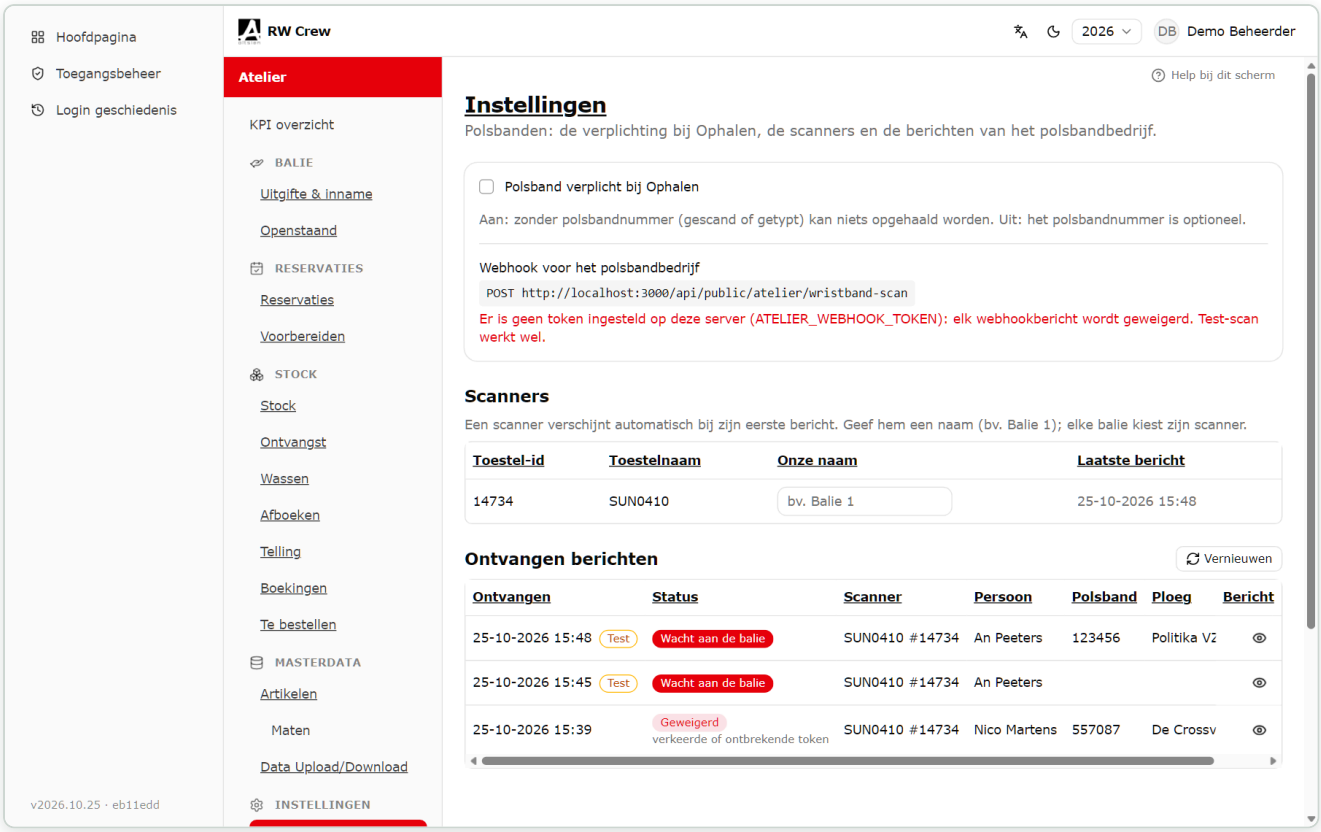
Figuur 3.15 – Atelier · Uitgifte & inname: festival, dag en de scanner van deze balie, en de twee grote knoppen waarmee elke transactie begint.



Figuur 3.16 – Atelier · Reservaties: per festival alle actieve ploegen, met het aantal personen, een opmerking en het pakket.



Figuur 3.17 – Atelier · Stock: per artikel en maat beschikbaar, te wassen en uitgegeven, met de minimumstock.



Figuur 3.18 – Atelier · Instellingen: polsband verplicht of niet, het webhookadres en of de token ingesteld is, de scanners en alle ontvangen berichten.

3.3.8 Walkies (module 6), Mobile App (module 10) en de plaatshouder (module 7)

Walkies heeft al een tegel, een KPI-pagina en het volledige rechtensysteem met rollen per scherm; het beheer van de walkietalkies volgt later. Module 10 toont het adres van de telefoon-app (`https://rwcrew.eu/m`), een QR-code om dat adres met de telefoon te openen, en de stappen om de app op het beginscherm te zetten. Module 7 is een voorbereide, lege plaats: ze heeft al een tegel, een kleur en toegangsrechten, en toont een "binnenkort"-pagina tot haar inhoud ontworpen is.

3.4 Gedeelde begrippen

Tabel 3.4 – Begrippen uit het domein die in meerdere modules terugkomen

Begrip	Betekenis
Seizoen (jaar)	Eén festivalseizoen, bv. "2026". Bijna alle keuzes en planningen gelden per seizoen. Een seizoen kan *open* of *gesloten* zijn (`periode_open`); de seizoenkiezer in de bovenbalk toont alleen open seizoenen.
Ploeg (team)	Een groep van een vereniging die op festivals meewerkt. Staat in MasterData (`MasterData_team`).
Festival	Een evenement waarop ploegen actief zijn. Per seizoen kiest een ploeg haar festivals.
Kar	Een genummerde rolcontainer die bij een ploeg geleverd wordt.
Altsien-kernlid	Een RWCrew-gebruiker met de vlag "Altsien Kernlid", gekoppeld aan ploegen; vult de Ploeg Wizard in.

Ploegverantwoordelijke	Een contactpersoon van een ploeg per seizoen. Is *geen* RWCrew-gebruiker; enkel naam en contactgegevens.
TeamKar	De RWCrew-medewerkers die de leverkar bemannen en aan interventie-aanvragen gekoppeld worden.
Levertoer	De ene route over het terrein waarin karren geleverd worden; de positie van een afleverlocatie erin heet `terrein_positie`. Daarnaast zijn er *speciale levertoeren* (bv. de wastoer) die langs ploegen komen.
Pakket (Atelier)	De artikelen die elke persoon van een ploeg op een festival krijgt, één stuk per artikel.
Polsband (Atelier)	Het festivalpolsbandje van een medewerker; elk festival heeft nieuwe polsbanden, dus het nummer is uniek binnen één festival.

3.5 Talen en weergave

De interface bestaat in het **Nederlands** (standaard) en het **Engels**. De taal staat in het adres: `/nl/...` of `/en/...`. Alle teksten staan in twee vertaalbestanden (`frontend/messages/nl.json` en `en.json`), zodat er in de code zelf geen schermteksten staan (zie §11.6). Mails en PDF's zijn in het Nederlands. De applicatie heeft een licht en een donker thema; donker is standaard. Elke module heeft een eigen accentkleur, zodat je altijd ziet in welke module je werkt.

4 Architectuuroverzicht

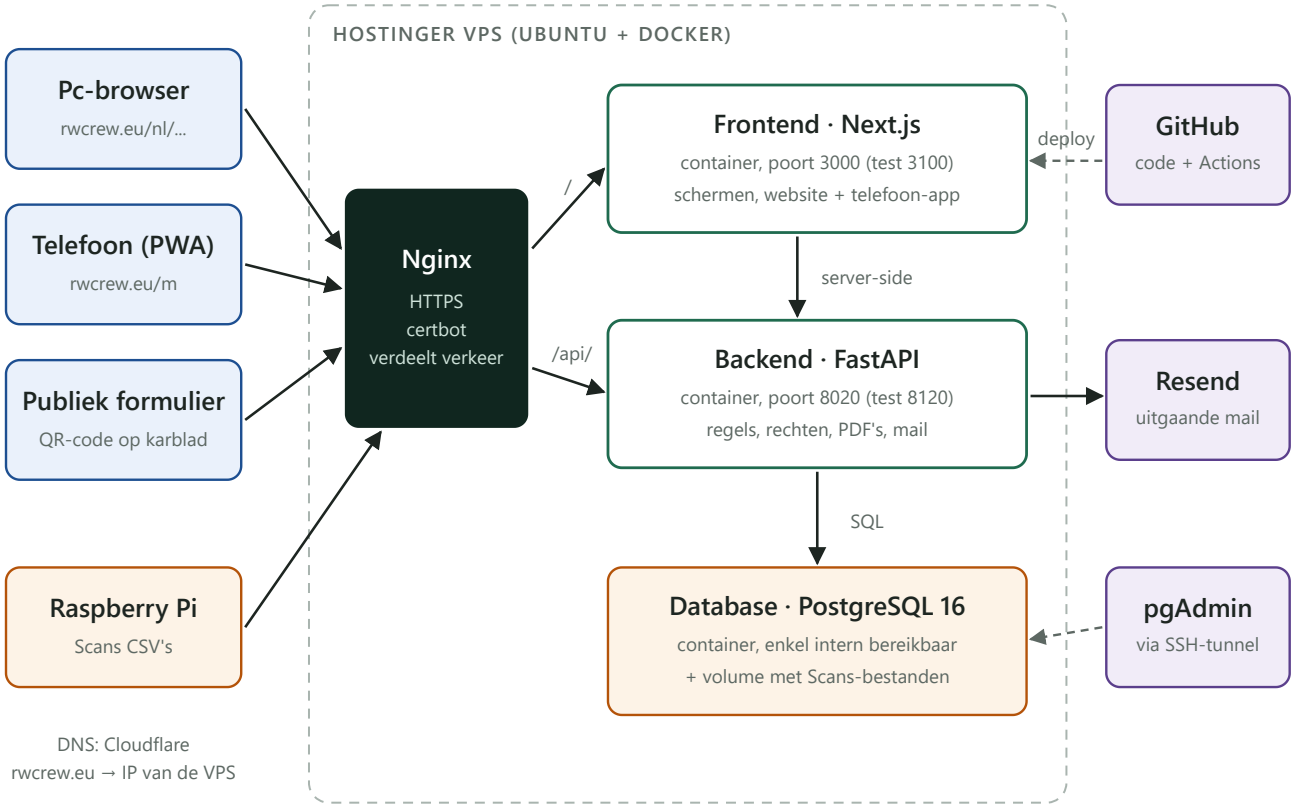
IEDEREEN

KORT GEZEGD

RWCrew bestaat uit drie programma's die elk in een eigen Docker-container draaien op één server (een VPS bij Hostinger): de **frontend** (Next.js, maakt de schermen), de **backend** (FastAPI, bevat alle regels en praat met de database) en de **database** (PostgreSQL). Vooraan staat **Nginx**, dat HTTPS afhandelt en het verkeer verdeelt. Daarnaast zijn er een paar externe koppelingen: **Resend** voor uitgaande mail, een **Raspberry Pi** die scanbestanden aanlevert, en **GitHub Actions** dat nieuwe versies automatisch installeert. Zowel de website als de telefoon-app gebruiken exact dezelfde frontend, backend en database.

4.1 Het grote plaatje

figuur 4.1 toont alle onderdelen en hoe ze met elkaar verbonden zijn. Lees de figuur van links naar rechts: een gebruiker typt `rwcrew.eu`, Cloudflare (DNS) vertaalt dat naar het IP-adres van de server, Nginx neemt het verzoek aan en stuurt het door naar de frontend of de backend.



Figuur 4.1 – Overzicht van alle onderdelen van RWCrew en hun verbindingen (productie; test is een tweede, identieke set containers op dezelfde server).

4.2 De frontend: Next.js

De frontend is gebouwd met **Next.js**, een veelgebruikt raamwerk op basis van **React**. Next.js maakt de pagina's grotendeels *op de server* klaar (*server-side rendering*): als je een pagina opent, haalt de frontend eerst zelf de nodige gegevens bij de backend en stuurt dan een volledig opgebouwde pagina naar de browser. Daarna neemt de browser het over voor interactie (knoppen, formulieren, dialoogvensters); die interacties praten rechtstreeks met de backend.

Dat betekent dat de backend vanuit twee plaatsen aangesproken wordt: door de **frontend-server** (bij het opbouwen van een pagina) en door de **browser** (bij een klik). Beide sturen dezelfde inlogcookies mee. Details: hoofdstuk 11.

4.3 De backend: FastAPI

De backend is een **Python**-programma gebouwd met **FastAPI**. Hij biedt alle endpoints onder `/api/...` aan en is de enige die met de database praat. Elke module heeft er een eigen pakket met zijn endpoints, regels, PDF-generatoren en importfuncties. De backend controleert bij *elk* verzoek wie de gebruiker is en of die het verzoek mag doen. Details: hoofdstuk 10.

Tabel 4.1 – Groepen van endpoints in de backend

Adres	Wat
`/api/auth/*`	Inloggen, sessie vernieuwen, uitloggen, wachtwoord wijzigen, "wie ben ik"
`/api/admin/*`	Superbeheerder: gebruikers, moduletoegang, inloghistoriek
`/api/modules`	Lijst van modules (voor de tegels)
`/api/modules/module-N/*`	De endpoints van module N, met haar eigen rechten
`/api/public/intervention-requests/*`	Publiek interventieformulier (geen login, wel snelheidslimiet)
`/api/public/tagscan-intake`	Ontvangst van CSV-bestanden van de Raspberry Pi (API-sleutel)
`/api/health`	"Leeft de backend?" Voor controles.

4.4 De database: PostgreSQL

Er is één PostgreSQL-database per omgeving. Alle modules gebruiken dezelfde database, maar elke tabel heeft een **voorvoegsel** dat zegt bij welk onderdeel ze hoort. Zo zie je in een databasetool meteen welke tabellen bij elkaar horen, en kunnen twee modules nooit per ongeluk dezelfde tabelnaam gebruiken.

Tabel 4.2 – Voorvoegsels van de databasetabellen

Voorvoegsel	Onderdeel	Voorbeelden
`Landing_`	Startscherm, gebruikers, toegang	`Landing_users`, `Landing_user_module_access`, `Landing_login_history`
`Tagscan_`	Scans (module 1)	`Tagscan_roles`, tag-header- en -linedata
`KarTracker_`	KarTracker (module 2)	`KarTracker_karren`, `KarTracker_kar_actions`

`InterventionRequests_`	Interventie Aanvragen (module 3)	`InterventionRequests_request`, `InterventionRequests_status`
`StockMaster_`	StockMaster (module 4)	`StockMaster_document`, `StockMaster_movement`, `StockMaster_balance`
`AltsienSelect_`	Altsien Select (module 8)	`AltsienSelect_step_progress`, `AltsienSelect_special_requests`
`MasterData_`	MasterData (module 9)	`MasterData_team`, `MasterData_season`, `MasterData_festival`

Eén uitzondering op de regel: de leverdata van KarTracker staan in de tabel `Festivals_leverdatum`. De volledige lijst van tabellen vind je in de map `backend/app/db/models/` (één bestand per tabel).

4.5 Externe koppelingen

Tabel 4.3 – Diensten buiten de eigen server

Dienst	Rol in RWCrew	Configuratie
GitHub	Bewaart de broncode; GitHub Actions installeert elke nieuwe versie op de server.	Repository-secrets `SSH_HOST`, `SSH_USER`, `SSH_PRIVATE_KEY`
Hostinger	Levert de VPS (virtuele server) waarop test en productie draaien.	Hostinger-account
Cloudflare	DNS van `rwcrew.eu`. Lokaal ook de "quick tunnel" om een telefoon via HTTPS met je pc te verbinden.	Cloudflare-account; `cloudflared` op de ontwikkel-pc
Let's Encrypt	Gratis HTTPS-certificaten, vernieuwd door certbot op de server.	Op de server
Resend	Verstuurt mails (nieuwe interventie-aanvraag) via een HTTP-API.	`RESEND_API_KEY`, `MAIL_FROM` (domein geverifieerd via DNS)
Raspberry Pi	Staat bij de RFID-scanners en stuurt CSV-bestanden door naar Scans.	`scripts/pi-watcher/`, API-sleutel per scanner
OpenStreetMap	Kaartachtergrond voor de Kar Map (via de bibliotheek Leaflet).	Geen sleutel nodig

4.6 Website en telefoon-app: één applicatie

De telefoon-app is **geen aparte applicatie**. Het is een apart deel van dezelfde website, onder het adres `/m`, met schermen die voor een telefoon ontworpen zijn. Ze gebruikt dezelfde login, dezelfde endpoints en dezelfde rechten. Een nieuwe versie van de website is automatisch ook een nieuwe versie van de app. (Tot oktober 2026 bestond er een aparte "native" app in React Native/Expo; die is volledig verwijderd.) Zie hoofdstuk 12.

4.7 Ontwerpprincipes

Een paar principes komen overal in de code terug. Wie ze kent, begrijpt sneller waarom iets is zoals het is:

1. **Alles per module.** Backend-code, frontend-schermen, telefoonschermen, vertalingen en databasetabellen zijn per module gegroepeerd. Wie aan module 3 werkt, hoeft niet in module 2 te zoeken.

2. **De backend beslist.** Rechten worden altijd in de backend gecontroleerd. De frontend verbergt alleen wat je toch niet mag gebruiken, voor het gebruiksgemak.
3. **Code is de bron van waarheid voor structuur.** Welke schermen een module heeft, welke stappen de wizard heeft en hoe de database eruitziet, staat in de code (en in migraties). Niets daarvan wordt met de hand in de database aangemaakt.
4. **Instellingen buiten de code.** Alles wat per omgeving verschilt, komt uit omgevingsvariabelen.
5. **Zo weinig mogelijk afhankelijkheden, en alleen zuivere Python-pakketten.** De ontwikkel-pc draait Python 3.14; pakketten die gecompileerd moeten worden, worden vermeden (zie §8.6).
6. **Eenvoud boven slimheid.** Een tweede module met hetzelfde patroon krijgt eerst een eigen kopie; pas als een patroon drie keer bestaat, wordt het eventueel veralgemeend.
7. **Leesbaar voor beginners.** Elk logisch blok code heeft commentaar in gewone taal dat uitlegt *wat* en vooral *waarom*.

5 Omgevingen: lokaal, test en productie

IEDEREEN

KORT GEZEGD

RWCrew draait op drie plaatsen. **Lokaal** op de pc van een ontwikkelaar (database in Docker, backend en frontend rechtstreeks gestart). **Test** op `test.rwcrew.eu` en **productie** op `rwcrew.eu`, allebei op dezelfde Hostinger-server, elk in een eigen map, met een eigen database, eigen instellingen en eigen poorten. Test volgt de Git-branch `develop`, productie volgt `main`.

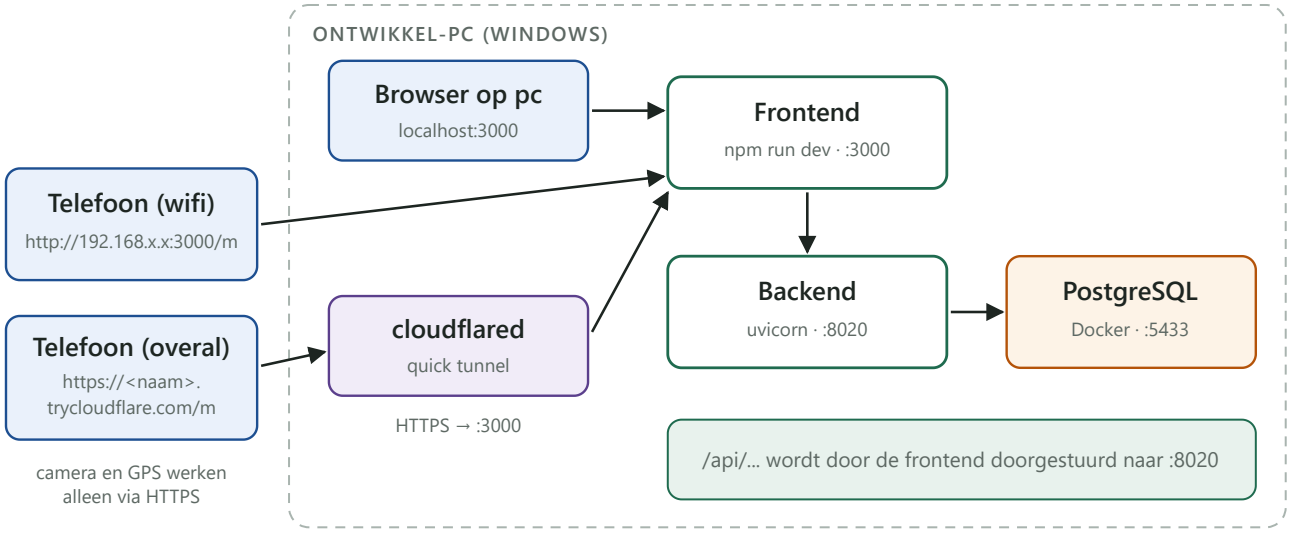
5.1 Overzicht

Tabel 5.1 – De drie omgevingen naast elkaar

	Lokaal	Test	Productie
Adres	`http://localhost:3000`	`https://test.rwcrew.eu`	`https://rwcrew.eu`
Telefoon-app	via wifi-IP of Cloudflare-tunnel + `/m`	`https://test.rwcrew.eu/m`	`https://rwcrew.eu/m`
Git-branch	eender welke	`develop`	`main`
Waar	ontwikkel-pc	VPS, `/opt/rwcrew/test`	VPS, `/opt/rwcrew/production`
Backend-poort	8020	8120 (alleen intern)	8020 (alleen intern)
Frontend-poort	3000	3100 (alleen intern)	3000 (alleen intern)
Database	Docker, poort 5433	container; voor pgAdmin 127.0.0.1:5442	container; voor pgAdmin 127.0.0.1:5443
Herkenbaar aan	adres localhost	banner "TEST", app-naam "RWCrew Test", mailonderwerp "[TEST]"	geen banner
Gegevens	proefgegevens	testgegevens, mogen weg	echte gegevens

5.2 De lokale ontwikkelomgeving

Lokaal draaien de onderdelen iets anders dan op de server: alleen de database draait in Docker, backend en frontend worden rechtstreeks gestart. Zo kun je de code aanpassen en meteen het resultaat zien. figuur 5.1 toont de opstelling, inclusief de manieren om een echte telefoon te laten meedoen.



Figuur 5.1 – De lokale opstelling: database in Docker, backend en frontend rechtstreeks, optioneel een tunnel voor een telefoon via HTTPS.

5.2.1 Eenmalige installatie

Installeer op een nieuwe ontwikkel-pc eerst deze programma's:

Tabel 5.2 – Benodigde software op een ontwikkel-pc

Programma	Versie	Waarvoor
Git	recent	Broncode ophalen en wijzigingen bewaren
Python	3.14 (zelfde als de server)	De backend
Node.js (met npm)	22 of hoger	De frontend
Docker Desktop	recent	De lokale PostgreSQL-database
Visual Studio Code	recent	Code-editor (aanbevolen, met de Python- en ESLint-extensies)
cloudflared	recent	Optioneel: telefoon via HTTPS verbinden
pgAdmin	recent	Optioneel: database bekijken

Daarna, eenmalig:

Eenmalige installatie (PowerShell of Git Bash, vanuit de map waar je de code wilt)

```
# 1. Code ophalen
git clone <adres-van-de-GitHub-repository> RWCrew
cd RWCrew

# 2. Database starten (Docker Desktop moet draaien)
docker compose up -d db

# 3. Backend: virtuele omgeving, pakketten, instellingen, database-structuur, eerste gebruiker
cd backend
python -m venv .venv
.venv/Scripts/pip install -r requirements.txt
cp .env.example .env
```

```
.venv/Scripts/python -m alembic upgrade head
.venv/Scripts/python -m app.cli.seed --email jij@voorbeeld.be --password "KiesEenWachtwoord!" --
name "Jouw Naam"

# 4. Frontend: pakketten en instellingen
cd ../frontend
npm install
cp .env.local.example .env.local
```

Het seed-script (`app.cli.seed`) maakt de tien modules aan, een begin-set seizoenen en de eerste **superbeheerder**. Het mag gerust meerdere keren uitgevoerd worden: wat al bestaat, wordt overgeslagen.

5.2.2 Dagelijks opstarten

Drie onderdelen starten, elk in een eigen terminal

```
# Database (repo-root), als ze nog niet draait
docker compose up -d db

# Backend (map backend) — bewust ZONDER --reload, zie de valkuil hieronder
.venv/Scripts/python -m uvicorn app.main:app --port 8020

# Frontend (map frontend)
npm run dev
```

Open daarna `http://localhost:3000` . Je wordt doorgestuurd naar `/nl/login` . De interactieve API-documentatie staat op `http://localhost:8020/docs` .

> LET OP
>
> Start de backend **niet** met `--reload` . Op Windows laat die optie bij het stoppen een "weesproces" achter dat de poort blijft bezetten met de *oude* code. Het gevolg: verzoeken gaan willekeurig naar de oude of de nieuwe versie en fouten lijken "niet opgelost" terwijl ze dat wel zijn. Dit heeft in het verleden veel tijd gekost. Herstart de backend gewoon met de hand na een wijziging. Zie §17.1 voor de volledige uitleg en hoe je zo'n proces opspoort.

5.2.3 Testen op een echte telefoon

Er zijn twee manieren om de lokale versie op een telefoon te openen:

- **Via wifi** (eenvoudig, maar zonder camera en GPS): zorg dat telefoon en pc op hetzelfde wifi-netwerk zitten, zoek het IPv4-adres van de pc op (`Get-NetIPAddress` , interface "Wi-Fi") en open `http://<dat-adres>:3000/m` .
- **Via een Cloudflare-tunnel** (met camera en GPS, want HTTPS): start `cloudflared tunnel --url http://localhost:3000` en open het getoonde adres `https://<naam>.trycloudflare.com/m` op de telefoon. Het adres verandert bij elke start.

Beide werken alleen als in `frontend/.env.local` deze twee regels staan (in plaats van een vast backend-adres), omdat de browser dan alle API-verzoeken naar het adres stuurt waarop de pagina geopend werd, en de frontend ze doorgeeft aan de backend:

frontend/.env.local (voor telefoontests)

```
NEXT_PUBLIC_API_URL=
```

```
DEV_API_PROXY_TARGET=http://localhost:8020
```

> **LET OP**
>
> Zet het tunneladres **nooit** in `NEXT_PUBLIC_API_URL`. Inloggen op localhost mislukt dan stilzwijgend: de inlogcookie wordt voor het tunneladres gezet en het loginscherm komt zonder foutmelding terug. Om dezelfde reden moeten het adres van de frontend en `NEXT_PUBLIC_API_URL` altijd dezelfde hostnaam gebruiken (`localhost` en `127.0.0.1` gelden als verschillende sites).

5.3 De server (Hostinger VPS)

Test en productie draaien op één **VPS** (*Virtual Private Server*) bij Hostinger, met Ubuntu Linux en Docker. Op de server staan twee volledig gescheiden kopieën van de repository:

- `/opt/rwcrew/test`: volgt `develop`, bereikbaar via `test.rwcrew.eu`;
- `/opt/rwcrew/production`: volgt `main`, bereikbaar via `rwcrew.eu`.

Elke map heeft een eigen `.env`-bestand met eigen wachtwoorden en sleutels, en wordt gestart als een eigen **docker compose-project**. Daardoor hebben test en productie elk hun eigen drie containers (`db`, `backend`, `frontend`) en hun eigen opslag (*volumes*), en kunnen ze elkaar nooit hinderen. Beide gebruiken hetzelfde bestand `docker-compose.prod.yml`.

De containers luisteren alleen op het interne adres `127.0.0.1` van de server; ze zijn niet rechtstreeks vanaf het internet bereikbaar. Alleen **Nginx** luistert naar buiten (poorten 80 en 443). De configuratie staat in `deploy/nginx/rwcrew.conf`:

Tabel 5.3 – Hoe Nginx verzoeken verdeelt

Adres	Productie gaat naar	Test gaat naar
`/api/public/tagscan-intake`	backend :8020, max. 25 MB, snelheidslimiet	backend :8120, idem
`/api/...`	backend :8020	backend :8120
al de rest	frontend :3000	frontend :3100

HTTPS-certificaten worden aangevraagd en automatisch vernieuwd door **certbot**. Certbot herschrijft het Nginx-bestand op de server en voegt aparte blokken voor poort 443 toe.

> **LET OP**
>
> Nginx weigert standaard verzoeken groter dan 1 MB. Daarom zet de configuratie `client_max_body_size 25m` op het Scans-adres. Na een certbot-wijziging moeten die regel en de snelheidslimiet (`limit_req`) ook in de 443-blokken staan, anders worden grote CSV-bestanden van de Raspberry Pi geweigerd voordat de backend ze ziet.

De volledige installatie van de server (gebruikers, firewall, Docker, Nginx, certbot, GitHub-sleutels) staat stap voor stap in `docs/deploying-to-hostinger.pdf`.

5.4 Instellingen per omgeving

Instellingen komen uit omgevingsvariabelen. Lokaal staan ze in `backend/.env` en `frontend/.env.local`; op de server in `/opt/rwcrew/<omgeving>/.env`. Voorbeelden met uitleg staan in `backend/.env.example`, `frontend/.env.local.example`, `.env.test.example` en `.env.production.example`. Geen enkel echt `.env`-bestand staat in Git.

Tabel 5.4 – Belangrijkste instellingen

Variabele	Voor	Betekenis
`DATABASE_URL`	backend	Verbinding met PostgreSQL, bv. `postgresql+pg8000://rwcrew:…@db:5432/rwcrew`
`JWT_SECRET`	backend	Geheime sleutel om inlogtokens te ondertekenen. Lang en willekeurig, verschillend per omgeving.
`CORS_ALLOWED_ORIGINS`	backend	Welke websites de API vanuit een browser mogen aanroepen
`RESEND_API_KEY`, `MAIL_FROM`	backend	Uitgaande mail. Leeg = mails worden overgeslagen (alleen een logregel).
`MAIL_SUBJECT_PREFIX`	backend	Voor elk mailonderwerp, bv. `[TEST]` op test
`APP_PUBLIC_URL`	backend	Publiek adres van de app, voor links in mails en de QR-code van module 10
`TAGSCAN_SOURCE_DIR`	backend	Map voor de Scans-bestanden (op de server een Docker-volume)
`TAGSCAN_ENVIRONMENT`	backend	Naam van deze omgeving voor de kolom `Mode` van Scans-CSV's (`test` of `production`); leeg = nooit doorsturen
`TAGSCAN_FORWARD_URL`, `TAGSCAN_FORWARD_API_KEY`	backend	Ontvangstadres van de *andere* omgeving en een scannersleutel die op die andere omgeving is aangemaakt
`TAGSCAN_AUTO_SCAN_WORKER`	backend	Noodschakelaar: `false` start het automatisch scannen niet (standaard `true`)
`APP_COMMIT`, `APP_COMMIT_DATE`	beide	Versielabel; wordt door `deploy.sh` zelf uit Git gezet, niet in `.env` (zie §6.6)
`NEXT_PUBLIC_API_URL`	frontend	Adres van de backend zoals de browser het aanroept. Wordt bij het *bouwen* ingebakken.
`DEV_API_PROXY_TARGET`	frontend	Alleen lokaal: stuurt `/api/...` door naar de backend
`NEXT_PUBLIC_SITE_URL`	frontend	Publiek adres voor QR-codes op afgedrukte karbladen
`NEXT_PUBLIC_ENVIRONMENT_LABEL`	frontend	Alleen op test: toont de banner "TEST" en noemt de telefoon-app "RWCrew Test"
`BACKEND_PORT`, `FRONTEND_PORT`, `DB_HOST_PORT`	server	Interne poorten per omgeving (zie tabel 5.1)
`POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB`	server	Aanmaakgegevens van de database-container

Instellingen in code staan in `backend/app/core/config.py` (backend) en `frontend/src/lib/config.ts` (frontend). Daar staat bij elke instelling ook de standaardwaarde en uitleg. Twee vaste waarden staan alleen in de code: het toegangstoken leeft **15 minuten** en een inlogsessie maximaal **6 uur**.

5.5 De database bekijken (pgAdmin)

Om een probleem op test of productie te onderzoeken, kun je de database bekijken met **pgAdmin** op je eigen pc. De database is nooit rechtstreeks via het internet bereikbaar; pgAdmin maakt daarom eerst een beveiligde **SSH-tunnel** naar de server en logt dan in met de gebruiker `rwcrew_readonly`, die alles kan lezen maar niets kan wijzigen.

Tabel 5.5 – Instellingen in pgAdmin

Instelling	Waarde
Host / poort	`127.0.0.1` / `5442` (test) of `5443` (productie)
Database / gebruiker	`rwcrew` / `rwcrew_readonly`
Parameters	`GSS encryption mode = disable` (anders crasht pgAdmin op Windows)
SSH-tunnel	host = IP van de VPS, poort 22, gebruiker `ubuntu`, identiteitsbestand = jouw SSH-sleutel

De read-only gebruiker wordt per omgeving één keer aangemaakt met `bash deploy/scripts/create-readonly-db-user.sh` (vanuit de map van die omgeving). Opnieuw uitvoeren zet een nieuw wachtwoord.

> **TIP**
> Geef de productieserver in pgAdmin een **rode achtergrondkleur**. Zo weet je altijd zeker dat je naar de echte gegevens kijkt.

5.6 Back-ups: aandachtspunt

In de repository staat **geen** script dat automatisch back-ups van de databases maakt. De gegevens staan in Docker-volumes op de VPS. Spreek als team af hoe back-ups geregeld worden en leg dat vast, bijvoorbeeld:

- controleren of de automatische snapshots van Hostinger aan staan en hoe lang ze bewaard worden;
- een dagelijkse `pg_dump` van de productiedatabase (via cron) naar een opslag buiten de server;
- minstens één keer per seizoen een terugzetting proberen op test, zodat je weet dat de back-up werkt.

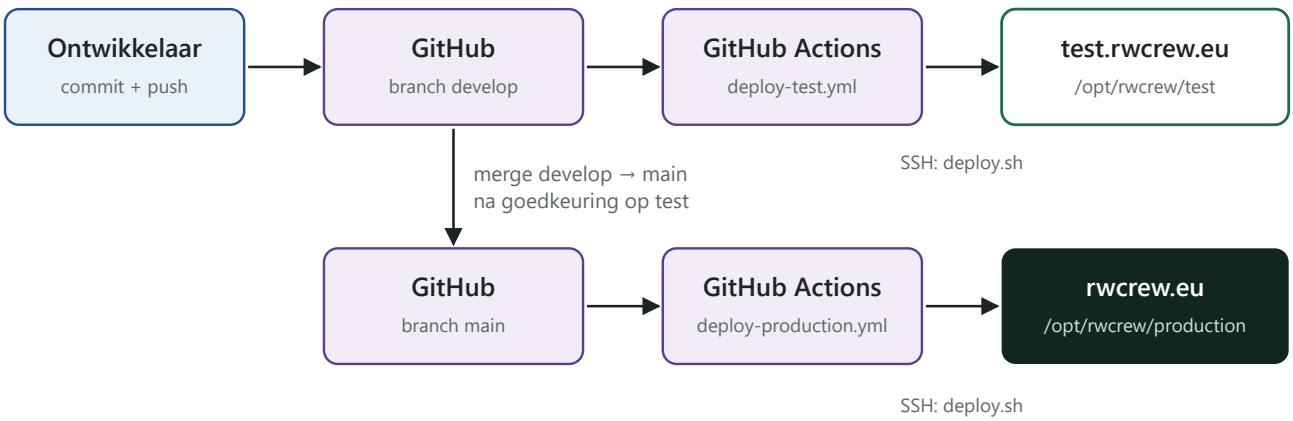
Hetzelfde geldt voor het volume met Scans-bestanden (`tagscan_data`).

6 Deployment en CI/CD

IEDEREEN

> **KORT GEZEGD**
> Een nieuwe versie online zetten gebeurt automatisch. Elke **push naar** `develop` zet die versie binnen enkele minuten op `test.rwcrew.eu`; elke **push naar** `main` zet ze op `rwcrew.eu`. GitHub Actions logt daarvoor in op de server en voert het script `deploy/scripts/deploy.sh` uit: code ophalen, containers opnieuw bouwen en starten, databasemigraties uitvoeren. Er is dus geen aparte "release"-knop. Wat in `main` staat, staat in productie.

6.1 Van commit tot live



Figuur 6.1 – De weg van een wijziging: eerst naar test via `develop`, na controle naar productie via `main`.

6.2 GitHub Actions

De map `.github/workflows/` bevat twee workflows: `deploy-test.yml` (start bij een push naar `develop`) en `deploy-production.yml` (start bij een push naar `main`). Ze doen allebei één ding: met SSH inloggen op de VPS en `deploy.sh` uitvoeren in de map van die omgeving. De inloggegevens staan als **secrets** in de GitHub-repository (Settings > Secrets and variables > Actions):

Tabel 6.1 – GitHub-secrets voor de deployment

Secret	Inhoud
`SSH_HOST`	IP-adres of hostnaam van de VPS
`SSH_USER`	De gebruiker waarmee gedeployed wordt
`SSH_PRIVATE_KEY`	De private SSH-sleutel van die gebruiker (de publieke sleutel staat op de server)

Je volgt een deployment op GitHub onder het tabblad **Actions**. Een groen vinkje betekent dat het script zonder fout liep; bij een rood kruis zie je daar de foutmelding.

6.3 Wat `deploy.sh` doet

deploy/scripts/deploy.sh (ingekort)

```
set -euo pipefail                       # stop bij de eerste fout
cd "$DEPLOY_DIR"                        # /opt/rwcrew/test of /opt/rwcrew/production
git pull --ff-only                      # 1. nieuwste code van de gevolgde branch
docker compose -f docker-compose.prod.yml up -d --build              # 2. bouwen + herstarten
docker compose -f docker-compose.prod.yml exec -T backend \
    python -m alembic upgrade head      # 3. databasemigraties
docker image prune -f                   # 4. oude images opruimen
```

1. **Code ophalen.** `--ff-only` weigert als er op de server lokale wijzigingen staan. Wijzig op de server dus nooit code met de hand.
2. **Bouwen en starten.** Docker bouwt alleen de containers opnieuw waarvan iets veranderd is en vervangt ze. Tijdens het herstarten is de site enkele seconden onbereikbaar.
3. **Migraties.** Nieuwe databasemigraties worden uitgevoerd in de backend-container. Een migratie die faalt, laat de deployment falen (rood kruis in GitHub Actions).
4. **Opruimen.** Oude, ongebruikte Docker-images worden verwijderd zodat de schijf niet volloopt.

6.4 De Docker-images

- **Backend** (`backend/Dockerfile`): vertrekt van `python:3.14-slim` , installeert `requirements.txt` , kopieert de code en start `uvicorn app.main:app` op poort 8020.
- **Frontend** (`frontend/Dockerfile`): bouwt in drie stappen (pakketten installeren, de app bouwen met `npm run build` , een kleine eindcontainer met alleen het resultaat) op basis van `node:22-slim` . Next.js staat op `output: "standalone"` , zodat de eindcontainer klein blijft.
- **Database**: het officiële image `postgres:16` , zonder aanpassingen.

> VOOR ONTWIKKELAARS
>
> Variabelen die met `NEXT_PUBLIC_` beginnen, worden tijdens `npm run build` in de JavaScript voor de browser **ingebakken**. Ze worden daarom als *build-argument* aan de frontend-container gegeven (`args:` in `docker-compose.prod.yml`) en niet als gewone omgevingsvariabele. Wijzig je zo'n waarde in de `.env` van de server, dan moet de frontend opnieuw *gebouwd* worden; herstarten is niet genoeg.

6.5 Een nieuwe instelling toevoegen

> LET OP
>
> `docker-compose.prod.yml` geeft de backend een **expliciete lijst** van omgevingsvariabelen (`environment:`). Een nieuwe instelling die alleen in de `.env` van de server staat, komt **niet** in de container terecht. Dat is de reden dat de eerste test-deployment geen mails verstuurde.

Een nieuwe backend-instelling toevoegen gaat dus altijd in vier stappen:

1. het veld toevoegen aan `Settings` in `backend/app/core/config.py` (met standaardwaarde en commentaar);

2. de variabele toevoegen aan de `environment:`-lijst van de backend in `docker-compose.prod.yml`;
3. de variabele met uitleg toevoegen aan `.env.test.example`, `.env.production.example` en `backend/.env.example`;
4. de echte waarde invullen in de `.env` op de server, in beide mappen, en de backend herstarten (`docker compose -f docker-compose.prod.yml up -d backend`).

6.6 Welke versie draait waar?

`deploy.sh` haalt bij elke deployment de korte commitcode (`APP_COMMIT`) en de commitdatum in Belgische tijd (`APP_COMMIT_DATE`, bv. `2026.10.04`) uit Git en geeft die aan beide containers. Zo zie je altijd welke code er draait:

- onderaan het beheermenu links staat `v<datum> · <commit>` (alleen zichtbaar voor de superbeheerder);
- `https://rwcrew.eu/api/health` geeft naast `"status":"ok"` ook de `"version"` van de backend.

Lokaal leest `next.config.ts` dezelfde waarden uit Git; zonder Git is het label `dev`. Vergelijk het label met de commit op GitHub om na een deployment te controleren dat de nieuwe versie echt live staat.

6.7 Handmatig deployen en terugdraaien

Een deployment opnieuw uitvoeren, bijvoorbeeld na het aanpassen van een `.env`, kan met de hand op de server, vanuit de map van de omgeving:

Op de VPS

```
cd /opt/rwcrew/production
bash deploy/scripts/deploy.sh /opt/rwcrew/production

# Containers bekijken, logs lezen
docker compose -f docker-compose.prod.yml ps
docker compose -f docker-compose.prod.yml logs --tail=200 backend
```

Een slechte versie **terugdraaien** gebeurt via Git, niet op de server: maak een `git revert` van de foute commit(s) en push die naar `main` (en `develop`). De normale deployment zet dan de vorige toestand terug. Let op: een migratie die al uitgevoerd is, wordt daarbij niet automatisch teruggedraaid. Bevatte de foute versie een migratie, overleg dan eerst (zie §10.14).

6.8 Checklist voor een release naar productie

- ☐ Alle backend-tests slagen lokaal (`pytest`) en de frontend bouwt zonder fouten (`npm run build`, `npm run lint`).
- ☐ De wijziging staat op `develop` en is op `test.rwcrew.eu` door een tweede persoon gecontroleerd.
- ☐ Nieuwe instellingen zijn op de productieserver ingevuld (zie §6.5).
- ☐ Bevat de release een migratie die gegevens wijzigt of verwijdert? Dan is er vlak voordien een back-up gemaakt.
- ☐ Het moment is gekozen: niet tijdens een festival of een drukke periode, tenzij het een dringende oplossing is.

- ☐ Na de deployment: GitHub Actions is groen, `https://rwcrew.eu/api/health` antwoordt, en de gewijzigde schermen werken.
- ☐ De handleiding en `CLAUDE.md` zijn bijgewerkt als de werking veranderde.

7 Beveiliging en toegangsrechten

IEDEREEN

> **KORT GEZEGD**
> Wie mag wat in RWCrew, wordt op twee onafhankelijke niveaus bepaald. (1) **Moduletoegang**: de superbeheerder beslist per gebruiker welke modules die mag openen. (2) **Rechten binnen een module**: de beheerder van die module bepaalt met zelfgemaakte rollen wat iemand per scherm mag zien, aanmaken, wijzigen of verwijderen. Je hebt beide nodig. Inloggen gebeurt met e-mail en wachtwoord; de sessie blijft maximaal 6 uur geldig. Alle controles gebeuren in de backend.

7.1 Inloggen en wachtwoorden

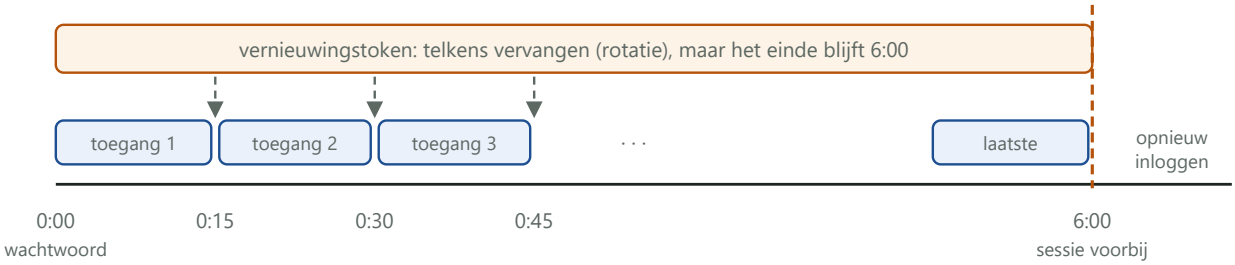
- Wachtwoorden worden nooit bewaard, alleen een **hash** met PBKDF2-HMAC-SHA256, een willekeurige *salt* per wachtwoord en 260.000 herhalingen. Dat maakt het raden van wachtwoorden uit een gestolen database zeer traag. Code: `backend/app/core/security.py`.
- Bij een fout e-mailadres en bij een fout wachtwoord geeft de backend **exact dezelfde** foutmelding, zodat niemand kan uitzoeken welke e-mailadressen bestaan.
- Elke inlogpoging, geslaagd of niet, komt in de **inloghistoriek** (`Landing_login_history`): tijdstip, e-mailadres, IP-adres, of het via de website (`web`) of de telefoon-app (`pwa`) was, en het soort toestel (pc, telefoon, tablet).
- Een gebruiker kan zijn eigen wachtwoord wijzigen (huidig wachtwoord vereist). Daarbij worden al zijn andere sessies (andere browsers, telefoon) beëindigd.
- De superbeheerder kan in *Manage Access* het wachtwoord van een gebruiker **resetten** (alle sessies van die gebruiker worden dan beëindigd) of een account **verwijderen**. Een account met `is_active` uit kan niet inloggen en wordt bij elk verzoek geweigerd; daar is nog geen knop voor in de interface.

7.2 Sessies: toegangstoken en vernieuwingstoken

Na het inloggen krijgt de browser twee cookies, allebei **HttpOnly** (JavaScript in de pagina kan ze niet lezen, wat diefstal via kwaadaardige code bemoeilijkt) en **SameSite=Lax** (ze worden niet meegestuurd met verzoeken die vanaf andere websites vertrekken):

Tabel 7.1 – De twee inlogcookies

Cookie	Geldig	Doel
`rwcrew_access_token`	15 minuten	Bewijst bij elk verzoek wie je bent.
`rwcrew_refresh_token`	tot 6 uur na inloggen	Alleen om een nieuw toegangstoken te halen (`POST /api/auth/refresh`).



Elke 15 minuten wordt het toegangstoken stil vernieuwd; de gebruiker merkt niets.

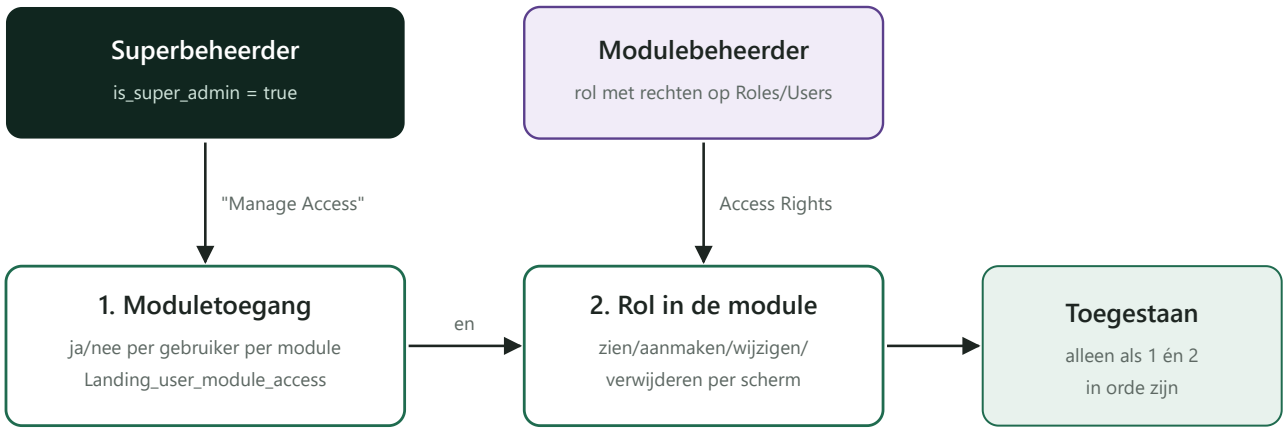
Figuur 7.1 – Levensloop van een inlogsessie: korte toegangstokens, een vernieuwingstoken dat bij elk gebruik vervangen wordt, en een harde grens van 6 uur.

Waarom zo? Een kortlevend toegangstoken beperkt de schade als het ooit onderschept wordt. Het vernieuwingstoken zorgt ervoor dat de gebruiker toch niet elk kwartier moet inloggen. De harde grens van 6 uur na het ingeven van het wachtwoord zorgt ervoor dat er geen sessies dagenlang open blijven staan, en dat de inloghistoriek echte, recente logins toont.

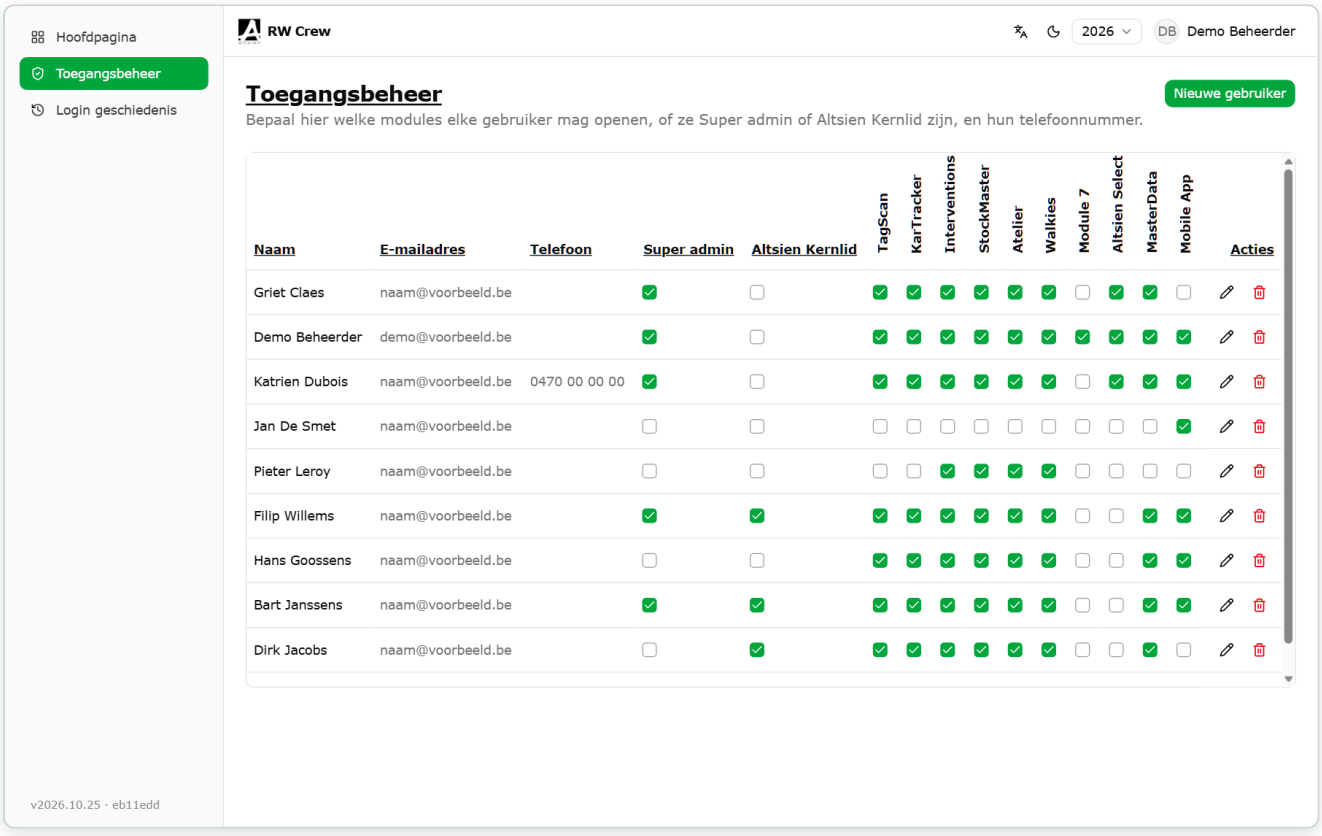
Elke keer dat het vernieuwingstoken gebruikt wordt, wordt het **vervangen** door een nieuw (*rotatie*) en wordt het oude ongeldig gemaakt. In de database staat alleen een hash van elk vernieuwingstoken (`Landing_refresh_tokens`). Een oud, al gebruikt token dat nog eens opduikt, wordt geweigerd.

> VOOR ONTWIKKELAARS
>
> Het stil vernieuwen gebeurt op **twee plaatsen** in de frontend, en beide zijn nodig:
>
> 1. `frontend/src/proxy.ts` : bij het openen van een pagina met wel een vernieuwings- maar geen toegangscookie vraagt de proxy eerst nieuwe tokens aan (via `lib/session-refresh.ts`), nog vóór de pagina opgebouwd wordt. Server Components kunnen zelf geen cookies zetten, dus dit kan niet later gebeuren.
> 2. `frontend/src/lib/api.ts` , functie `fetchWithRefresh` : bij een 401-antwoord op een verzoek vanuit de browser wordt één keer vernieuwd en het verzoek herhaald.
>
> Omdat de backend het vernieuwingstoken bij elk gebruik roteert, mag er per token maar **één** vernieuwing tegelijk lopen; een tweede gelijktijdige poging met hetzelfde token krijgt een 401. Beide plaatsen delen daarom één lopende vernieuwing (*single-flight*) en de proxy onthoudt het resultaat 30 seconden. Behoud dat als je hier iets aanpast. Elk token bevat ook een willekeurige `jti` ; zonder die waarde zouden twee tokens die in dezelfde seconde voor dezelfde gebruiker gemaakt worden identiek zijn, en zou de rotatie stilzwijgend falen. Verwijder die niet.

7.3 Twee onafhankelijke toegangsniveaus



Figuur 7.2 – De twee toegangsniveaus. Een gebruiker heeft eerst moduletoegang nodig; pas dan telt zijn rol binnen de module.



Figuur 7.3 – Manage Access (Toegangsbeheer): per gebruiker de vlaggen Super admin en Altsien Kernlid, en één vinkje per module.

7.3.1 Niveau 1: moduletoegang

De **superbeheerder** (een gebruiker met `is_super_admin`) beslist op het scherm *Manage Access* per gebruiker welke modules die mag openen. Dat is een eenvoudig vinkje per module, bewaard in `Landing_user_module_access`. Een module zonder vinkje verschijnt niet op het startscherm, en elk endpoint van die module antwoordt met 403.

> **LET OP**
> Superbeheerder zijn geeft **niet automatisch** toegang tot de modules, ook niet tot de eigen modules. Dat is een bewuste keuze: de macht van de superbeheerder is *beslissen wie wat mag openen*, niet alles zien. Wil de superbeheerder zelf een module openen, dan zet die bij zichzelf het vinkje, zoals voor iedereen. Code: `backend/app/shared/access.py` .

7.3.2 Niveau 2: rollen met rechten per scherm

Binnen modules 1, 2, 3, 4, 8 en 9 bepaalt een **rollensysteem** wat iemand mag. Elke module heeft daarvoor een eigen menu *Access Rights* met twee schermen:

- **Roles**: rollen aanmaken (vrij te kiezen namen, bv. "Kernlid", "Logistiek", "Lezer") en per rol een **rechtenmatrix** invullen: per scherm van de module vier vinkjes, *view* (zien), *create* (aanmaken), *edit* (wijzigen) en *delete* (verwijderen).
- **Users**: per gebruiker met moduletoegang één rol kiezen, en nieuwe gebruikers rechtstreeks toegang geven tot deze module.

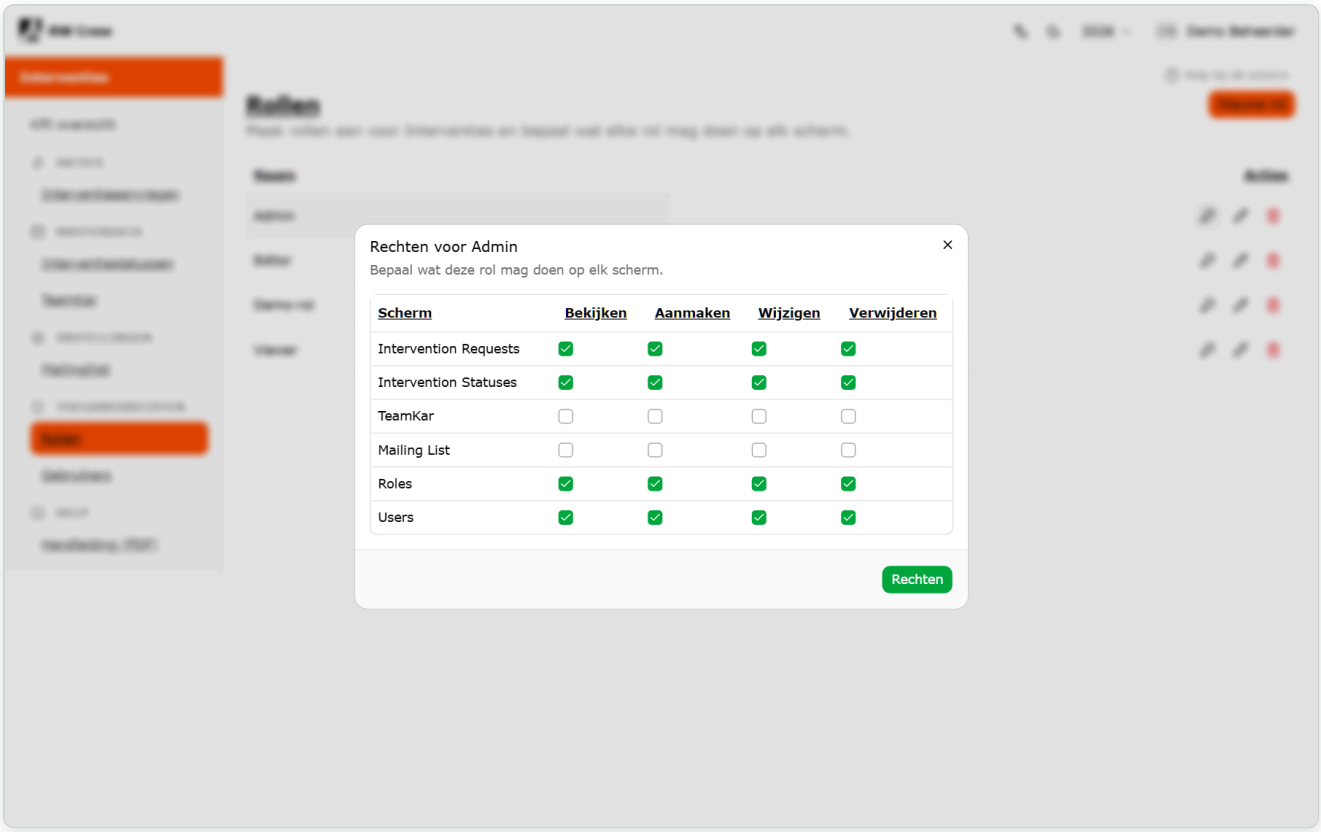
Tabel 7.2 – Voorbeeld van een rechtenmatrix voor de rol "Logistiek" in Interventie Aanvragen

Scherm	Zien	Aanmaken	Wijzigen	Verwijderen
Intervention Requests	✓	✓	✓	–
Intervention Statuses	✓	–	–	–
TeamKar	✓	–	–	–
Mailing List	–	–	–	–
Roles / Users	–	–	–	–

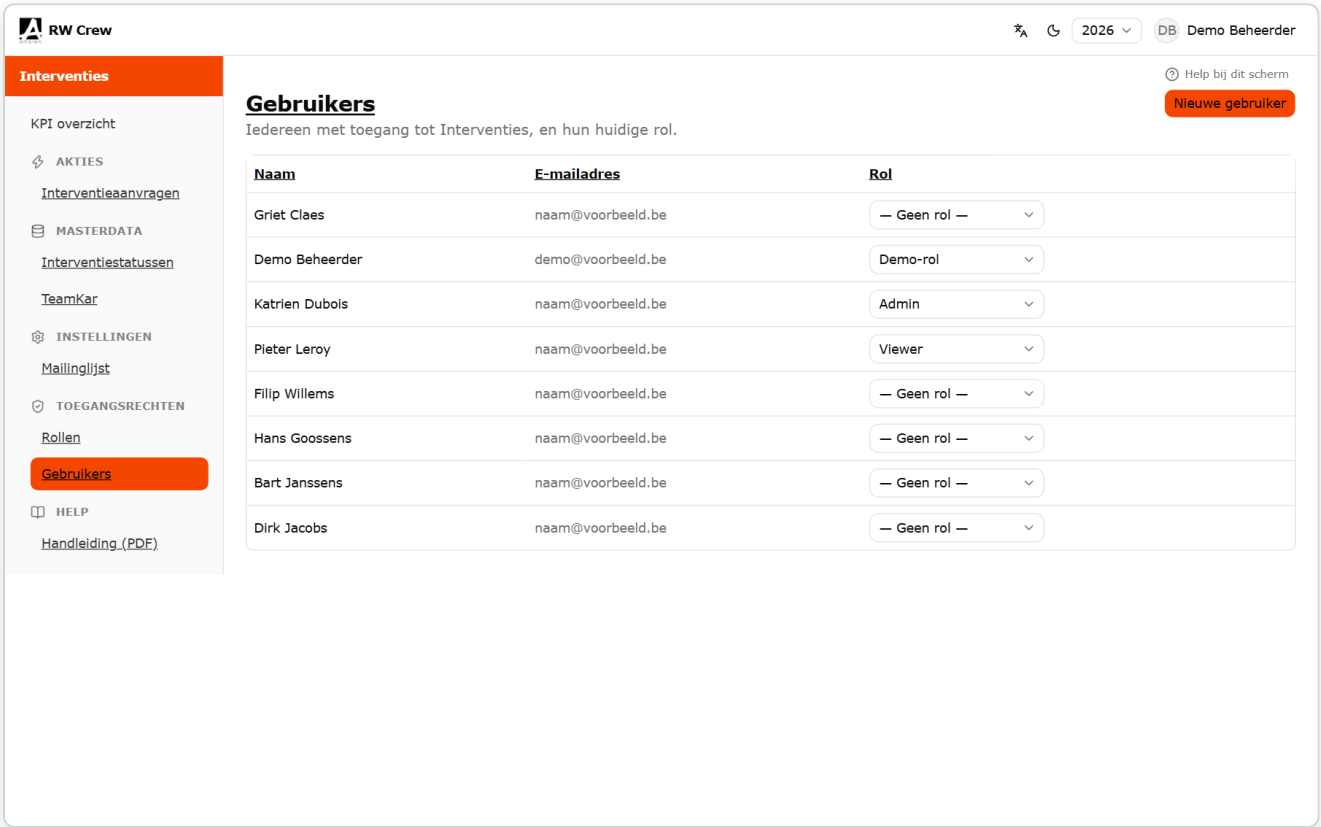
Belangrijke regels van dit systeem:

- **Geen vinkje is geen recht.** Een nieuw scherm verschijnt automatisch als lege rij in elke rechtenmatrix; niemand kan er iets tot een beheerder een rol aanvinkt. Veilig standaard.
- **Eén rol per gebruiker per module.** Wil je iemand meer rechten geven, pas dan zijn rol aan of maak een nieuwe rol.
- **Schermen zijn onafhankelijk.** Recht op het ene scherm zegt niets over een ander scherm. Ook de schermen Roles en Users zijn apart: iemand kan gebruikers toevoegen zonder rollen te mogen aanpassen.
- **De superbeheerder mag binnen een module alles**, op voorwaarde dat hij moduletoegang heeft. Zo kan hij de allereerste rol van een module aanmaken, of ingrijpen als de enige modulebeheerder vertrekt.
- Een rol toekennen kan alleen aan iemand die al moduletoegang heeft.

De volledige lijst van schermen per module staat in bijlage B. Hoe dit in de code gebouwd is, staat in §10.7.



Figuur 7.4 – Access Rights > Rollen: de rechtenmatrix van een rol, met per scherm de vinkjes Bekijken, Aanmaken, Wijzigen en Verwijderen.



Figuur 7.5 – Access Rights > Gebruikers: per gebruiker met moduletoegang één rol in deze module.

7.3.3 Modules met vaste rollen

Module 7 en module 10 gebruiken nog een eenvoudiger, ouder systeem: drie vaste rollen (*admin*, *editor*, *reader*) in één gedeelde tabel `Landing_module_roles`. Zodra zo'n module echte inhoud krijgt, krijgt ze normaal ook het systeem met rollen per scherm (zie §14.3).

7.4 Extra regels in Altsien Select

Altsien Select heeft bovenop de rechten per scherm twee extra regels:

- **Ploegafbakening.** Een gebruiker ziet alleen de ploegen waaraan hij gekoppeld is als Altsien-kernlid (`MasterData_team_kernlid`). Een rol met "zien" op het speciale scherm `altsienselect.allteams` ("All teams (organisation)") ziet alle ploegen. Een andere ploeg opvragen geeft 404, zodat je niet eens kunt achterhalen dat ze bestaat.
- **Seizoensslot.** Zodra bij een seizoen `periode_open` uit staat, weigert de backend elke wijziging (403), behalve voor wie "wijzigen" heeft op `altsienselect.allteams`.

7.5 Endpoints zonder login

Drie groepen endpoints zijn bewust bereikbaar zonder inloggen, elk met eigen beveiliging:

Tabel 7.3 – Publieke endpoints en hun beveiliging

Endpoint	Waarvoor	Beveiliging
`/api/public/intervention-requests`	Het publieke interventieformulier (QR-code op het karblad)	Snelheidslimiet: max. 10 aanvragen per minuut per IP-adres; alle invoer wordt gecontroleerd en in mails ge-escaped
`/api/public/tagscan-intake`	CSV-bestanden van de Raspberry Pi	API-sleutel per scanner (header `X-API-Key`, gehasht bewaard), max. 20 MB per bestand, Nginx-snelheidslimiet (30 per minuut)
`/api/public/wristband-scan`	De polsbandscans van het polsbandbedrijf aan de Atelier-balie (JSON per scan)	Gedeelde token in het bericht, vergeleken in constante tijd met `WRISTBAND_WEBHOOK_TOKEN` (geen token ingesteld = alles geweigerd, 401). Elk bericht, ook een geweigerd, wordt gelogd en is te zien op Atelier > Instellingen. Een bericht maakt zelf nooit een persoon aan.

Een API-sleutel van een scanner werkt alleen op de omgeving waar ze aangemaakt is (test of productie).

7.6 Overige maatregelen

- **HTTPS** op alle publieke adressen (Let's Encrypt).
- **CORS**: de backend accepteert browserverzoeken alleen vanaf de eigen website(s) (`CORS_ALLOWED_ORIGINS`).
- **Database en containers alleen intern**: alleen Nginx is van buitenaf bereikbaar. Voor debuggen is er een read-only databasegebruiker via een SSH-tunnel.

- **Geheimen buiten Git**: `.env`-bestanden staan in `.gitignore`. De `JWT_SECRET` is per omgeving verschillend, zodat een token van test nooit op productie werkt.
- **Mails falen veilig**: een probleem bij het versturen wordt gelogd maar maakt de bewerking (bv. het opslaan van een aanvraag) nooit ongedaan.

> **LET OP**
> Wie toegang heeft tot de server, de GitHub-repository-instellingen, Hostinger, Cloudflare of Resend, heeft feitelijk toegang tot alles. Houd bij wie welke toegang heeft, geef ze alleen aan wie ze nodig heeft, en wijzig sleutels en wachtwoorden als iemand het team verlaat (zie §16.8).

DEEL B

Gebruikte tools

Elke technologie, bibliotheek en dienst waarmee RWCrew gebouwd is en draait: wat ze doet, waar je ze tegenkomt en waarom ze gekozen is.

Doelgroep: iedereen. De tabellen zijn ook een naslagwerk voor ontwikkelaars.

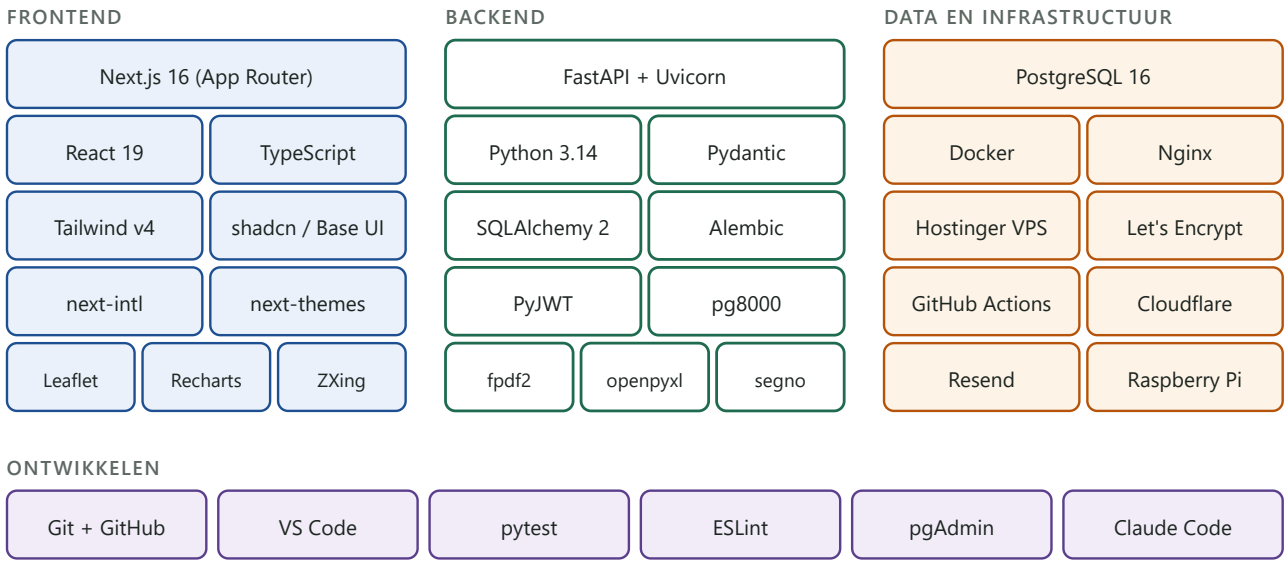
8 Tools en technologieën

IEDEREEN

KORT GEZEGD

De frontend is geschreven in **TypeScript** met **Next.js** en **React**, opgemaakt met **Tailwind CSS** en kant-en-klare onderdelen van **shadcn/ui**. De backend is geschreven in **Python** met **FastAPI** en **SQLAlchemy**; de database is **PostgreSQL**. Alles draait in **Docker** op een Linux-server achter **Nginx**, en wordt automatisch gedeployed met **GitHub Actions**. Gekozen is telkens voor breed gebruikte, gratis, goed gedocumenteerde tools, zodat nieuwe teamleden er makkelijk informatie over vinden.

8.1 De technologie in één beeld



Figuur 8.1 – Alle tools van RWCrew, gegroepeerd per laag.

8.2 Frontend

Versies volgens `frontend/package.json`. Een `^` voor een versienummer betekent dat kleinere updates (zelfde hoofdversie) automatisch toegelaten zijn.

Tabel 8.1 – Tools van de frontend

Tool	Versie	Wat het doet	Waarom gekozen
TypeScript	5	JavaScript met *types*: de editor weet welke velden een object heeft en waarschuwt bij fouten nog vóór je de app start.	Vangt een grote klasse fouten vroeg op; standaard bij Next.js.
React	19.2	Bibliotheek om schermen op te bouwen uit herbruikbare *componenten* (een knop, een tabel, een formulier).	De meest gebruikte UI-bibliotheek; enorm veel kennis en voorbeelden online.

Next.js	16.3	Raamwerk rond React: routering op basis van mappen, pagina's op de server opbouwen, productiebuild.	Pagina's tonen meteen met gegevens (geen laadflits), ingelogd-zijn wordt op de server gecontroleerd, één project voor website en telefoon-app.
Tailwind CSS	4	Opmaak via korte klassen in de HTML (`p-6` , `text-sm` , `bg-muted`) in plaats van aparte CSS-bestanden.	Snel, consistent, geen botsende stylesheets; werkt naadloos met shadcn/ui.
shadcn/ui op **Base UI**	4.19 / 1.7	Kant-en-klare, toegankelijke onderdelen (knoppen, dialogen, tabellen, keuzelijsten) die als broncode in `src/components/ui/` staan.	Je bezit de code zelf en kunt ze aanpassen; Base UI zorgt voor toetsenbordbediening en toegankelijkheid.
next-intl	4.14	Meertaligheid: taal in het adres (`/nl` , `/en`) en alle teksten uit `messages/*.json` .	De standaardkeuze voor vertalingen in Next.js met de App Router.
next-themes	0.4	Wisselen tussen licht en donker thema zonder flikkering.	Klein en betrouwbaar.
lucide-react	1.35	Pictogrammen.	Groot, consistent pakket; standaard bij shadcn.
sonner	2.0	Korte meldingen ("Opgeslagen", "Fout bij opslaan") rechtsonder.	Standaard meldingscomponent van shadcn.
Leaflet + react-leaflet	1.9 / 5	Interactieve kaarten (Kar Map, afleverlocaties) op basis van OpenStreetMap.	Gratis, geen API-sleutel nodig, licht.
Recharts	3.8	Grafieken op de KPI- en dashboardschermen.	Eenvoudig te gebruiken met React; integreert met shadcn's grafiekcomponent.
@zxing/browser	0.2	QR-codes lezen met de camera (KarScan).	Werkt volledig in de browser, ook op telefoons.
ESLint	9	Controleert de code op fouten en slordigheden (`npm run lint`).	Standaard bij Next.js.

8.3 Backend

Versies volgens `backend/requirements.txt` ; daar staan ze vastgepind (`==`), zodat elke installatie exact dezelfde versies krijgt.

Tabel 8.2 – Tools van de backend

Tool	Versie	Wat het doet	Waarom gekozen
Python	3.14	De programmeertaal van de backend.	Leesbaar, veel gebruikt, sterk voor data en rapporten.
FastAPI	0.141	Raamwerk om API-endpoints te schrijven, met automatische controle van invoer en automatische documentatie (`/docs`).	Modern, snel, weinig code per endpoint, uitstekende documentatie.
Uvicorn	0.52	De webserver die de FastAPI-applicatie laat draaien.	De standaardserver voor FastAPI.
Pydantic + pydantic-settings	2.13 / 2.15	Beschrijft de vorm van gegevens (*schema's*) en controleert ze; leest instellingen uit `.env` .	Ingebouwd in FastAPI; foutieve invoer wordt automatisch geweigerd met een duidelijke melding.
SQLAlchemy	2.0	Vertaalt Python-klassen naar databasetabellen en Python-code naar SQL (een *ORM*).	De standaard in Python; werkt met PostgreSQL én met SQLite voor de tests.
Alembic	1.19	Databasemigraties: genummerde scripts die de structuur van de database aanpassen.	Hoort bij SQLAlchemy; kan migraties automatisch voorstellen.
pg8000	1.31	De verbinding tussen Python en PostgreSQL (*driver*).	Puur Python, dus altijd installeerbaar op Python 3.14 (zie §8.6).

PyJWT	2.13	Maakt en controleert de inlogtokens (JWT).	Klein, betrouwbaar, puur Python.
hashlib (standaard)	–	Wachtwoord-hashing met PBKDF2.	Standaardbibliotheek; geen gecompileerd pakket zoals bcrypt nodig.
fpdf2	2.8	Maakt PDF's: leveringsbon, karbladen, tag-headers, Ploegfiche.	Puur Python, volledige controle over de opmaak.
segno	1.6	Maakt QR-codes (karbladen, installatiepagina).	Puur Python, scherpe vectoruitvoer.
openpyxl	3.1	Leest en schrijft Excel-bestanden (sjablonen, import, export).	De standaard voor `.xlsx` in Python.
httpx	0.28	HTTP-verzoeken naar andere diensten (Resend).	Was al aanwezig via FastAPI; geen extra mailpakket nodig.
python-multipart	0.0.32	Bestanden ontvangen via formulieren (CSV- en Excel-uploads).	Vereist door FastAPI voor uploads.
pytest	9.1	Automatische tests (meer dan 600 testfuncties).	De standaard in Python.

8.4 Database, infrastructuur en diensten

Tabel 8.3 – Database, hosting en externe diensten

Tool	Wat het doet	Waarom gekozen
PostgreSQL 16	De database.	Gratis, zeer betrouwbaar, de meest gebruikte open-source database voor zakelijke apps.
Docker + Docker Compose	Verpakt backend, frontend en database in containers; lokaal alleen de database.	Elke omgeving draait identiek; test en productie naast elkaar op één server zonder te botsen.
Hostinger VPS (Ubuntu)	De server.	Betaalbaar, volledige controle, genoeg voor beide omgevingen.
Nginx	Reverse proxy: HTTPS, verkeer verdelen, uploadlimieten, snelheidslimieten.	De standaard; licht en snel.
Let's Encrypt + certbot	Gratis HTTPS-certificaten met automatische vernieuwing.	Gratis en automatisch.
Cloudflare	DNS van `rwcrew.eu`; lokaal de "quick tunnel" (`cloudflared`).	Gratis, snel; de tunnel geeft lokaal HTTPS voor camera en GPS op een telefoon.
GitHub + GitHub Actions	Broncode, geschiedenis, automatische deployment.	Gratis voor dit gebruik; alles op één plaats.
Resend	Uitgaande mail via een eenvoudige HTTP-API.	Eenvoudiger en betrouwbaarder dan een eigen mailserver; goede aflevering dankzij domeinverificatie.
Raspberry Pi	Stuurt Scans-bestanden naar de server (`scripts/pi-watcher/`).	Goedkoop, zuinig, kan naast de scanners blijven staan.

8.5 Ontwikkeltools

Tabel 8.4 – Tools voor het dagelijkse ontwikkelwerk

Tool	Waarvoor
Git	Versiebeheer: commits, branches, geschiedenis.

Visual Studio Code	Aanbevolen code-editor. Handige extensies: Python, Pylance, ESLint, Tailwind CSS IntelliSense, GitLens, Claude Code.
Docker Desktop	Docker op Windows, voor de lokale database.
pgAdmin	De database bekijken, lokaal of via een SSH-tunnel op de server.
cloudflared	Lokale app via HTTPS bereikbaar maken voor een telefoon.
Browser-ontwikkelaarstools (`F12`)	Netwerkverzoeken, statuscodes en fouten bekijken; het tabblad *Network* is je beste vriend bij een probleem.
FastAPI `/docs`	Interactieve documentatie van alle endpoints, lokaal op `http://localhost:8020/docs`.
Claude Code	AI-programmeerassistent waarmee RWCrew grotendeels ontwikkeld is (zie §8.7).

8.6 De Python 3.14-beperking

De ontwikkel-pc en de backend-container draaien **Python 3.14**, een zeer recente versie. Veel populaire Python-pakketten bevatten stukken in C of Rust die voor elke Python-versie apart gecompileerd moeten worden. Voor een gloednieuwe Python-versie zijn die voorgecompileerde versies (*wheels*) er soms nog niet, en dan moet je zelf een C/Rust-compiler installeren.

Daarom werd bij elke keuze bewust een pakket genomen dat **puur Python** is, of de standaardbibliotheek:

Tabel 8.5 – Bewuste keuzes door de Python 3.14-beperking

Gekozen	In plaats van
`pg8000` (databasedriver)	`psycopg2` of `asyncpg` (gecompileerd)
`hashlib.pbkdf2_hmac` (standaard)	`passlib` / `bcrypt`
`httpx` rechtstreeks naar Resend	een apart mailpakket
gewone reguliere expressies voor het toesteltype	een user-agent-bibliotheek
`fpdf2` en `segno`	PDF- of QR-pakketten met C-onderdelen

> **LET OP**
> Controleer vóór je een nieuw Python-pakket toevoegt of het voor Python 3.14 installeerbaar is zonder compiler (`pip install` in de `.venv` moet slagen zonder foutmeldingen over "building wheel"). Twijfel je, kies dan een alternatief in puur Python of los het op met de standaardbibliotheek.

8.7 Claude Code en CLAUDE.md

RWCrew is ontwikkeld met de hulp van **Claude Code**, een AI-programmeerassistent van Anthropic die in de terminal of in VS Code werkt. Je beschrijft in gewone taal wat je wilt ("voeg een mailinglijst toe aan module 3"), en Claude Code leest de bestaande code, stelt wijzigingen voor, voert ze uit en draait de tests. De dichte, uitleggende commentaarstijl en de consequente opbouw per module zijn mee daardoor ontstaan.

Het bestand `CLAUDE.md` in de hoofdmap van de repository is het **projectgeheugen**: Claude Code leest het bij elke start automatisch en volgt de afspraken die erin staan (architectuur, valkuilen, codeerafspraken, hoe je lokaal alles opstart). Daardoor maakt de assistent niet telkens dezelfde fouten opnieuw. Voor mensen is het de beknoptste samenvatting van het project.

> **TIP**
> Werk je met Claude Code (of een andere AI-assistent), houd dan deze regels aan:
> - Laat de assistent nooit zonder controle naar `main` pushen: lees elke wijziging na zoals je die van een collega zou nalezen.
> - Ontdek je een nieuwe valkuil of maak je een nieuwe afspraak, zet die dan in `CLAUDE.md`. Zo profiteren mensen én de assistent ervan.
> - Geef nooit geheimen (wachtwoorden, API-sleutels, `.env`-inhoud) aan de assistent als dat niet nodig is.
> - Laat de assistent de tests draaien en de documentatie bijwerken als deel van de taak.

8.8 Afhankelijkheden bijwerken

Bibliotheken krijgen regelmatig updates met oplossingen voor beveiligingsproblemen. Spreek af dat iemand ze bijvoorbeeld één keer per kwartaal (en niet vlak voor een festival) nakijkt:

- **Frontend**: `npm outdated` toont wat verouderd is; `npm audit` toont gekende beveiligingsproblemen. Werk bij met `npm install <pakket>@<versie>`, test lokaal (`npm run build`) en commit `package.json` én `package-lock.json`.
- **Backend**: `.venv/Scripts/pip list --outdated`. Pas de versie aan in `requirements.txt`, installeer, draai alle tests (`pytest`) en commit.
- Grote versiesprongen (bv. Next.js 16 naar 17, of een nieuwe hoofdversie van FastAPI) apart en bewust doen: lees de migratiegids van die tool, werk op een aparte branch en test grondig op test.

DEEL C

Hoe de applicatie ontwikkeld is

De code zelf: waar alles staat, hoe backend, frontend en telefoon-app opgebouwd zijn, een volledige doorloop van één verzoek, recepten voor veelvoorkomende uitbreidingen, de codeerafspraken, samenwerken als team en de valkuilen die al eens tijd gekost hebben.

Doelgroep: ontwikkelaars. Beheerders vinden hun taken in Deel D.

9 Structuur van de repository

ONTWIKKELAARS

KORT GEZEGD

De repository heeft twee hoofdmappen: `backend/` (Python/FastAPI) en `frontend/` (Next.js). Daarnaast staan er de deploymentbestanden (`deploy/` , `docker-compose*.yml` , `.github/`), documentatie (`docs/`) en het Raspberry Pi-script (`scripts/`). Binnen backend en frontend is alles **per module** georganiseerd: wie aan module 3 werkt, vindt alles in mappen met "module_3" of "module-3" in de naam.

9.1 De hoofdmap

Hoofdmap van de repository

```
RWCrew/
├── backend/                  FastAPI-backend (Python)
├── frontend/                 Next.js-frontend (TypeScript), website + telefoon-app
├── deploy/
│   ├── nginx/rwcrew.conf     Nginx-configuratie voor beide domeinen
│   └── scripts/              deploy.sh, create-readonly-db-user.sh
├── .github/workflows/        deploy-test.yml, deploy-production.yml
├── docs/                     documentatie (PDF's, rollenpatroon, deze handleiding)
├── scripts/pi-watcher/       script voor de Raspberry Pi (Scans)
├── docker-compose.yml        lokaal: alleen PostgreSQL (poort 5433)
├── docker-compose.prod.yml   server: db + backend + frontend (test én productie)
├── .env.test.example         voorbeeld-instellingen voor test
├── .env.production.example   voorbeeld-instellingen voor productie
├── CLAUDE.md                 projectgeheugen: architectuur, valkuilen, afspraken
└── README.md
```

9.2 De backend-map

backend/

```
backend/
├── app/
│   ├── main.py               startpunt: maakt de FastAPI-app, registreert alle routers
│   ├── core/                 config.py (instellingen), database.py, security.py, mail.py
│   ├── db/
│   │   ├── base.py           importeert alle modellen (voor Alembic en tests)
│   │   └── models/           één bestand per databasetabel
│   ├── schemas/              Pydantic-schema's: vorm van invoer en uitvoer, per module
│   ├── landing/              auth.py (/api/auth), admin.py (/api/admin), deps.py, modules.py
│   ├── modules/
│   │   ├── common.py         gedeelde fabriek voor eenvoudige modules
│   │   ├── registry.py       de lijst van modules (sleutel, naam, volgorde)
│   │   └── module_1 … module_10/   één pakket per module
│   ├── shared/               access.py (moduletoegang), device.py (toesteltype)
│   ├── cli/seed.py           eerste gegevens + eerste superbeheerder
│   └── assets/               lettertypes/afbeeldingen voor PDF's
├── migrations/               Alembic: env.py en versions/ (de migraties)
```

```
├── tests/                    pytest, zelfde opbouw als app/ (landing/, modules/module_N/)
├── requirements.txt          vastgepinde Python-pakketten
├── Dockerfile
└── .env.example
```

9.3 De frontend-map

frontend/

```
frontend/
├── src/
│   ├── proxy.ts              draait vóór elke pagina: taal + stil sessie vernieuwen
│   ├── app/
│   │   ├── layout.tsx        echte root: <html>, <body>, thema
│   │   ├── manifest.ts       PWA-manifest (installeerbare telefoon-app)
│   │   └── [locale]/         alles onder /nl of /en
│   │       ├── layout.tsx    vertalingen beschikbaar maken
│   │       ├── (auth)/login/            inlogpagina website
│   │       ├── (public)/intervention-request/  publiek formulier
│   │       ├── (protected)/             alles waarvoor je ingelogd moet zijn
│   │       │   ├── page.tsx             startscherm met tegels
│   │       │   ├── admin/access/, admin/login-history/, admin/changelog/
│   │       │   └── modules/module-1 … module-10/
│   │       └── m/                       telefoon-app
│   │           ├── login/
│   │           └── (signed-in)/module-2/, module-3/
│   ├── components/
│   │   ├── ui/               shadcn-onderdelen (gegenereerd)
│   │   ├── shared/           bovenbalk, menu's, taal, thema, seizoenkiezer
│   │   ├── admin/, landing/  beheerschermen, tegels, loginformulier
│   │   ├── module-1 … module-10/       schermen per module
│   │   └── phone/shared/, phone/module-2/, phone/module-3/
│   ├── lib/                  api.ts, server-api.ts, server-auth.ts, types.ts, config.ts, …
│   └── i18n/                 taalinstellingen (routing.ts, request.ts, navigation.ts)
├── messages/nl.json, en.json alle schermteksten
├── public/                   iconen, afbeeldingen
├── next.config.ts, package.json, Dockerfile, .env.local.example
```

9.4 Waar vind ik…?

Tabel 9.1 – Snel de juiste plaats vinden

Ik zoek…	Kijk in
de endpoints van module N	`backend/app/modules/module_N/router.py`
wie wat mag in module N	`…/module_N/deps.py` en `…/module_N/screens.py`
een databasetabel	`backend/app/db/models/<naam>.py`
de vorm van gegevens die een endpoint ontvangt/terugstuurt	`backend/app/schemas/<module>.py`
een PDF (leveringsbon, karblad, Ploegfiche)	`…/module_N/*_pdf.py`
een Excel-import	`…/module_N/*_import.py`
de stappen van de Ploeg Wizard	`backend/app/modules/module_8/steps.py`

een webscherm	`frontend/src/app/[locale]/(protected)/modules/module-N/…/page.tsx` (de pagina) en `frontend/src/components/module-N/` (het scherm zelf)
het menu links in een module	`frontend/src/components/module-N/*-sidebar.tsx`
een telefoonscherm	`frontend/src/app/[locale]/m/(signed-in)/module-N/` en `frontend/src/components/phone/module-N/`
de functie die een endpoint aanroept vanuit de browser	`frontend/src/lib/api.ts`
de TypeScript-types van de gegevens	`frontend/src/lib/types.ts`
een schermtekst	`frontend/messages/nl.json` en `en.json`
de kleur van een module	`frontend/src/lib/module-theme.ts`
de tests van module N	`backend/tests/modules/module_N/`

10 De backend in detail

ONTWIKKELAARS

> KORT GEZEGD
> De backend is een FastAPI-applicatie. `main.py` registreert per module een **router** (een groep endpoints). Elk endpoint krijgt via **dependencies** een databasesessie en de ingelogde gebruiker, en controleert met `require_screen_permission` of die gebruiker het mag. Tabellen zijn **SQLAlchemy-modellen**, invoer en uitvoer zijn **Pydantic-schema's**, wijzigingen aan de database gaan via **Alembic-migraties**, en alles is gedekt door meer dan 900 **pytest**-tests die op een tijdelijke SQLite-database draaien.

10.1 Opstart: `main.py`

`backend/app/main.py` is het startpunt. Het doet vier dingen:

1. Het maakt het FastAPI-object (`app = FastAPI(title="RW Crew API", lifespan=lifespan)`).
2. Het voegt **CORS** toe, zodat de frontend vanuit de browser de API mag aanroepen en cookies mag meesturen.
3. Het registreert alle routers: `auth` , `admin` , `modules` en die van elke module (inclusief de publieke routers van module 1 en 3).
4. In de `lifespan` -functie, die één keer bij het opstarten loopt, synchroniseert het de **schermregisters** van modules 1, 2, 3, 4, 8 en 9 met de code (zie §10.7), en start het **automatisch scannen** van Scans als achtergrondtaak (`module_1/auto_scan.py`). Dat rekent op precies één backendproces per omgeving; komen er ooit meerdere workers, verhuis die taak dan naar één worker of een cron-job.

10.2 Instellingen en databaseverbinding

`app/core/config.py` definieert één klasse `Settings` (pydantic-settings) met alle instellingen en hun standaardwaarden; die worden overschreven door omgevingsvariabelen of `backend/.env` . Gebruik in de code altijd `from app.core.config import settings` , nooit `os.environ` .

`app/core/database.py` maakt de *engine* (de verbinding met PostgreSQL) en de functie `get_db()` , die per verzoek een databasesessie geeft en die na afloop altijd sluit. Elk endpoint dat de database nodig heeft, vraagt die sessie aan met `db: Session = Depends(get_db)` .

10.3 Modellen: tabellen als Python-klassen

Elke tabel is een klasse in een eigen bestand onder `app/db/models/` . Een ingekort voorbeeld:

backend/app/db/models/intervention_request.py (ingekort)

```python
class InterventionRequest(Base):
    """One row per intervention request logged by an association."""

    __tablename__ = "InterventionRequests_request"

    id: Mapped[int] = mapped_column(primary_key=True)
```

```
    # "Aanvraagnummer": generated server-side as "IA<yy>_<sequence>", never from the client.
    request_number: Mapped[str] = mapped_column(String(20), unique=True, index=True,
nullable=False)

    # "Ploeg": a foreign key into MasterData_team (nullable: the public form may type a new
name).
    team_id: Mapped[int | None] = mapped_column(ForeignKey("MasterData_team.id"), nullable=True)

    question: Mapped[str] = mapped_column(Text, nullable=False)
    status_id: Mapped[int] = mapped_column(ForeignKey("InterventionRequests_status.id"),
nullable=False)
```

- `Mapped[int | None]` betekent: kolom van type int die leeg mag zijn. SQLAlchemy leidt het databasetype af.
- De tabelnaam volgt het voorvoegsel van de module (zie tabel 4.2). Omdat de namen hoofdletters bevatten, zet SQLAlchemy ze automatisch tussen aanhalingstekens; in SQL in pgAdmin moet je dat zelf doen: `SELECT * FROM "InterventionRequests_request";`
- Een nieuw model moet ook geïmporteerd worden in `app/db/base.py`, anders zien Alembic en de tests de tabel niet.

10.4 Schema's: de vorm van invoer en uitvoer

Wat een endpoint ontvangt en terugstuurt, is beschreven met **Pydantic**-klassen in `app/schemas/<module>.py`. FastAPI controleert binnenkomende gegevens daar automatisch tegen: een verplicht veld dat ontbreekt of een tekst die te lang is, levert een 422-antwoord op zonder dat je daar zelf code voor schrijft.

backend/app/schemas/intervention_requests.py (ingekort)

```
class InterventionRequestCreateRequest(BaseModel):
    team_id: int | None = None
    team_name: str | None = Field(default=None, max_length=255)
    question: str = Field(min_length=1)
    status_id: int
    # ... meer velden ...

    @model_validator(mode="after")
    def _check_team_reference(self):
        _validate_team_reference(self.team_id, self.team_name)   # precies één van beide
        return self
```

Velden die de server zelf bepaalt (het aanvraagnummer, de aanmaakdatum), staan bewust *niet* in het create-schema, zodat een client ze nooit kan meesturen.

10.5 Het pakket van een module

Elke module met echte inhoud heeft dezelfde soorten bestanden. Niet elke module heeft ze allemaal:

Tabel 10.1 – Bestanden in `backend/app/modules/module_N/`

Bestand	Rol
`router.py`	Alle endpoints van de module onder `/api/modules/module-N` : rollen en gebruikers, de eigen schermen, `me/permissions` .

`deps.py`	De rechtencontroles: `require_module_access`, `user_can`, `require_screen_permission`.
`screens.py`	De lijst van schermen (`SCREEN_DEFINITIONS`) en `sync_screens()`.
`service.py`, `*_service.py`	Bedrijfslogica die door meerdere endpoints of modules gedeeld wordt.
`*_pdf.py`	PDF-generatoren (fpdf2).
`*_import.py`	Excel-sjablonen, import en export (openpyxl).
`*_dashboard.py`, `dashboard.py`	Berekeningen voor KPI- en dashboardschermen.
`*_router.py`	Een grote module verdeelt haar endpoints over meerdere routers op hetzelfde voorvoegsel, elk opgenomen in `main.py` (bv. `module_4/product_masterdata_router.py`, `module_2/karcode_router.py`, `module_5/desk_router.py`).
`public_router.py`, `device_router.py`, `wristband/webhook_router.py`	Endpoints zonder login (module 3: publiek formulier; module 1: Raspberry Pi; module 5: polsbandscans).
`notifications.py`	Mails (module 3).
`steps.py`, `previous_season.py`	Wizardstappen en "kopiëren van vorig seizoen" (module 8).

10.6 Dependencies: wie roept aan en mag dat?

FastAPI-*dependencies* zijn kleine functies die automatisch vóór een endpoint lopen. RWCrew gebruikt ze voor alle controles, zodat elk endpoint in één regel aangeeft wat het nodig heeft:

Tabel 10.2 – De belangrijkste dependencies

Dependency	Waar	Doet
`get_db`	`app/core/database.py`	Geeft een databasesessie voor dit verzoek.
`get_current_user`	`app/landing/deps.py`	Leest de toegangscookie, controleert het token, laadt de gebruiker; anders 401.
`require_super_admin`	`app/landing/deps.py`	Zoals hierboven, plus de superbeheerder-vlag; anders 403.
`require_module_access`	`…/module_N/deps.py`	De gebruiker heeft moduletoegang; anders 403.
`require_screen_permission(key, actie)`	`…/module_N/deps.py`	Moduletoegang én de rol heeft *actie* op scherm *key*; anders 403.

backend/app/modules/module_3/router.py (ingekort)

```
@router.post("/intervention-requests", response_model=InterventionRequestResponse,
status_code=201)
def create_intervention_request(
    payload: InterventionRequestCreateRequest,        # gecontroleerde invoer
    background_tasks: BackgroundTasks,                # om na het antwoord de mail te sturen
    db: Session = Depends(get_db),                    # databasesessie
    _user: User = Depends(require_screen_permission("interventionrequests.requests", "create")),
) -> InterventionRequest:
    ...
```

10.7 Het rechtensysteem in code

Het systeem met rollen per scherm (uitgelegd voor gebruikers in §7.3.2) bestaat per module uit vier tabellen en drie bestanden:

Tabel 10.3 – Onderdelen van het rechtensysteem per module (voorbeeld: module 3)

Onderdeel	Inhoud
`InterventionRequests_screens`	Register van schermen: `key`, `label`, `sort_order`
`InterventionRequests_roles`	De rollen van deze module (unieke naam)
`InterventionRequests_role_permissions`	Per rol en scherm: `can_view`, `can_create`, `can_edit`, `can_delete`
`InterventionRequests_user_roles`	Welke rol een gebruiker heeft (één per gebruiker)
`screens.py`	`SCREEN_DEFINITIONS` + `sync_screens()`
`deps.py`	`user_can()` en `require_screen_permission()`
`router.py`	Endpoints voor schermen, rollen, rechtenmatrix, gebruikers en `me/permissions`

Schermen staan in de code. `SCREEN_DEFINITIONS` in `screens.py` is de enige bron van waarheid; `sync_screens()` zet die lijst bij elke opstart van de backend in de tabel (nieuwe schermen erbij, labels en volgorde bijgewerkt). Een scherm toevoegen is dus: één regel in de lijst en de backend herstarten.

`backend/app/modules/module_3/screens.py (ingekort)`

```
SCREEN_DEFINITIONS: list[ScreenDefinition] = [
    ScreenDefinition(key="interventionrequests.requests", label="Intervention Requests",
sort_order=1),
    ScreenDefinition(key="interventionrequests.statuses", label="Intervention Statuses",
sort_order=2),
    ScreenDefinition(key="interventionrequests.teamkar", label="TeamKar", sort_order=3),
    ScreenDefinition(key="interventionrequests.mailinglist", label="Mailing List", sort_order=4),
    ScreenDefinition(key="interventionrequests.roles", label="Roles", sort_order=5),
    ScreenDefinition(key="interventionrequests.users", label="Users", sort_order=6),
]
```

De controle zelf staat in `user_can(db, user, screen_key, action)`: superbeheerder → ja; anders de rol van de gebruiker opzoeken, dan de rechtenrij voor dat scherm, en de juiste vlag teruggeven. Geen rol of geen rij = nee. `require_screen_permission` roept eerst `require_module_access` aan, zodat ook de superbeheerder moduletoegang nodig heeft.

De frontend vraagt de rechten op via `GET /api/modules/module-N/me/permissions`. Het antwoord bevat vier lijsten schermsleutels (`viewable_screen_keys`, `creatable_…`, `editable_…`, `deletable_…`). Daarmee toont het menu alleen de schermen die je mag zien, en verbergt een scherm de knoppen die je niet mag gebruiken.

> VOOR ONTWIKKELAARS
>
> Dit systeem is **per module gekopieerd** (modules 1, 2, 3, 4, 5, 6, 8 en 9 hebben elk hun eigen modellen, schema's, `deps.py` en endpoints), in plaats van één gedeelde fabriek. Dat was een bewuste keuze: eerst echte varianten bouwen en pas veralgemenen als het patroon zich bewezen heeft. Nu het acht keer bestaat, is een gedeelde versie (geparametriseerd met modulesleutel en tabelvoorvoegsel, zoals `create_module_router`) een kandidaat voor een opkuis, maar geen verplichting. De volledige kopieerhandleiding staat in `docs/module-custom-roles-pattern.md`.

10.8 De gedeelde fabriek voor eenvoudige modules

Module 7 (zonder eigen inhoud) en module 10 gebruiken `create_module_router(module_key, module_label)` uit `app/modules/common.py`. Die maakt in één regel een router met drie endpoints: `GET /status` (voor de "binnenkort"-pagina), `GET /roles` en `PUT /roles/{user_id}` (de vaste rollen admin/editor/reader). Een module die echte inhoud krijgt, zet eigen endpoints naast die oproep in haar `router.py`, of stapt over op het volledige rechtensysteem.

10.9 Gedeelde logica tussen modules

Sommige gegevens worden vanuit meerdere modules gewijzigd. In plaats van die logica te dupliceren, staat ze in één *service* bij de module die eigenaar is van de gegevens, en roepen de andere modules die service aan:

Tabel 10.4 – Gedeelde services

Service	Gebruikt door
`module_2/plan_kar_service.py` (afleverlocaties per ploeg/festival/seizoen)	KarTracker "Plan a kar" én Altsien Select stap 2
`module_9/team_responsible_service.py` (ploegverantwoordelijken)	MasterData "Ploegverantwoordelijken" én Altsien Select stap 3
`module_2/kar_action_service.py` (karbewegingen)	KarTracker-scherm én KarScan op de telefoon
`module_3/service.py` (aanvraagnummer, standaardstatus, ploegen)	Ingelogde router én publieke router van module 3
`module_4/stock_service.py` (de *enige* code die voorraad wijzigt)	Alle boekingsschermen van StockMaster; later ook automatische boekingen vanuit Scans
`module_4/requirement_service.py` (benodigdheden per kar)	StockMaster "Benodigdheden" én Altsien Select stap 4 (Producten)
`module_4/delivery_entitlement.py` (wat "nu beschikbaar is voor levering")	Kar laden; het stroomdiagram van de organisatie staat enkel hier
`module_4/tagscan_book_in.py` (Scans-regels "Book In")	StockMaster Inboeken; zet de Scans-regels in dezelfde transactie op *loaded*
`module_2/delivery_order_service.py` (levertoer)	Leveringsvolgorde; de planningsanalyses lezen `terrein_positie`
`module_2/karcode_formula.py` (karcodeformule, via `ast` en een witte lijst, nooit `eval`)	Karcode Simulator (web en telefoon) en Instellingen
`module_5/stock_service.py`, `desk_service.py`, `scan_service.py`	Atelier: de enige code die respectievelijk de stock, de personen/uitgiftes en de polsbandscans schrijft (zie §10.11)

`module_1/line_processing.py` (acties van Scans-regels)	Het verwerkingsvenster van Scans; nieuwe acties komen hier als functie bij

10.10 StockMaster: grootboek en saldo

StockMaster werkt als een boekhouding. Elke handeling is een `StockMaster_document` met één `StockMaster_movement` per product ("van" en "naar": `free`, `kar` of `external`). Die rijen worden nooit gewijzigd of verwijderd; een correctie is een tegenboeking (`reverse_document`). Daarnaast houdt `StockMaster_balance` het actuele saldo bij: één rij per product per plaats (`kar_id` leeg = vrije voorraad).

- **Alleen `stock_service.py` wijzigt voorraad.** Het vergrendelt de saldorijen (`SELECT … FOR UPDATE`), schrijft document, bewegingen en saldo in dezelfde transactie en weigert alles wat negatief zou worden.
- **De service doet zelf geen commit**; de router (of later een automatische Scans-verwerking) doet dat. Zo kan een aanroeper meerdere stappen samen laten slagen of mislukken.
- **Foutmeldingen zijn codes.** Een `StockError("code", *parameters)` wordt HTTP-detail `"code|param|…"`; de frontend vertaalt die onder `stockMaster.errors.<code>`. Een nieuwe foutcode heeft dus een vertaling in beide taalbestanden nodig.
- **De gebruiker komt uit de sessie** (`created_by` + een kopie van de naam in `created_by_name`), nooit uit het verzoek.
- MasterData en KarTracker weigeren het verwijderen van een product met voorraadgeschiedenis of een kar met voorraad.

10.11 Atelier: stock, balie en polsbandscans

Atelier volgt hetzelfde boekhoudpatroon als StockMaster, maar met een eigen set tabellen:

- **Stock**: `Atelier_stock_document` (`AT-<jaar>-<id>`; ontvangst, wassen, afboeken, telling, uitgifte, terugname, tegenboeking) met `Atelier_stock_movement` (van/naar-status: `available`, `to_wash`, `handed_out`, leeg = buiten) en `Atelier_stock_balance` (één rij per artikelmaat en status, vergrendeld met `FOR UPDATE`, nooit negatief). Alleen `module_5/stock_service.py` schrijft ze; foutcodes worden vertaald onder `atelier.stock.errors.<code>`.
- **Reservaties**: `Atelier_reservation` (festival + ploeg: personen, opmerking) en `Atelier_reservation_line` (het pakket), enkel via `reservation_service.py`. `preparation_service.py` berekent de dagelijkse behoefte zonder iets op te slaan.
- **Balie**: `desk_service.py` is de enige die `Atelier_person` (sleutel `name_key` = voornaam|familienaam), `Atelier_person_bracelet` (polsbandnummer per festival), `Atelier_handout` en `Atelier_handout_line` schrijft. Ophalen boekt *beschikbaar → uitgegeven*, terugbrengen *uitgegeven → te wassen*.
- **Polsbandscans**: `scan_service.process_message` verwerkt elk bericht, van de webhook én van het testscherm (dat de tokencontrole overslaat en het bericht als `test` markeert). Het logt alles in `Wristband_scan`, registreert de scanner (`Wristband_scanner`, op `data.device.id`) en zoekt de ploeg op naam. De balie vraagt elke 2 seconden de wachtende scans van haar scanner op (`GET /desk/scans?device_id=`); een transactie die een scan gebruikt, stuurt `scan_id` mee.
- **Instelling**: `Atelier_settings` (één rij) met `wristband_required`, de schakelaar "Polsband verplicht bij Ophalen".

10.12 PDF's, Excel en QR-codes

- **PDF's** worden in Python opgebouwd met **fpdf2**: `module_3/intervention_request_pdf.py` (leveringsbon), `module_2/karblad_pdf.py` (karbladen), `module_1/tag_header_pdf.py`, `module_8/ploegfiche_pdf.py`, de StockMaster-PDF's in `module_4/stock_pdfs.py` (huisstijl in `pdf_layout.py`, ook gebruikt door Atelier: `module_5/picklist_pdf.py` en `stock_pdfs.py`) en de planningen van KarTracker (`module_2/delivery_planning_pdf.py`, `kar_moves_pdf.py`, `special_tour_pdf.py`, met een kaart die op het grondplan getekend wordt zonder kaarttegels). Elke module heeft daarnaast een handleiding als PDF (`backend/app/help/`). Het endpoint stuurt de bytes terug met `media_type="application/pdf"`.
- **QR-codes** komen van **segno**. Op karbladen worden ze als vectorvlakjes getekend, zodat ze bij elke afdrukresolutie scherp blijven.
- **Excel**-import en -export gebruiken **openpyxl**. Een import is *best-effort*: elke rij wordt apart gecontroleerd en opgeslagen binnen een eigen SAVEPOINT (`db.begin_nested()`), zodat een foute rij alleen zichzelf terugdraait. De gebruiker krijgt per rij het resultaat (aangemaakt, bijgewerkt, fout + reden).

10.13 Uitgaande mail

`app/core/mail.py` heeft één functie, `send_email(mail)`, die via `httpx` de HTTP-API van Resend aanroept. Ze **gooit nooit een fout**: problemen worden gelogd. Zonder `RESEND_API_KEY` of `MAIL_FROM` (lokaal, in tests) wordt de mail met een logregel overgeslagen. Ontvangerslijsten langer dan 50 worden opgesplitst.

Het patroon voor een mail na een bewerking staat in `module_3/notifications.py`: eerst binnen de databasesessie de mail *opbouwen* (ontvangers, gegevens, PDF), en dan alleen het trage versturen als FastAPI-*background task* plannen. Zo krijgt de gebruiker meteen zijn 201-antwoord en kan een mailprobleem het opslaan nooit ongedaan maken.

> **LET OP**
>
> Elke nieuwe manier om een interventie-aanvraag aan te maken, moet ook `queue_new_request_mail()` aanroepen. Anders krijgt de mailinglijst die aanvragen niet. Een wijziging van een bestaande aanvraag stuurt bewust geen mail.

10.14 Databasemigraties met Alembic

Elke wijziging aan de databasestructuur gaat via een migratie in `backend/migrations/versions/`. De werkwijze:

Een migratie maken en uitvoeren (map backend)

```
# 1. Pas het model aan (of voeg een nieuw model toe én importeer het in app/db/base.py)
# 2. Laat Alembic het verschil met de database voorstellen
.venv/Scripts/python -m alembic revision --autogenerate -m "add mailing list to intervention
requests"

# 3. LEES het gegenereerde bestand na en corrigeer waar nodig
# 4. Uitvoeren op je lokale database
.venv/Scripts/python -m alembic upgrade head

# 5. Controleren dat modellen en database nu overeenkomen
```

```
.venv/Scripts/python -m alembic check

# Eén stap terugdraaien (lokaal, om een migratie te testen)
.venv/Scripts/python -m alembic downgrade -1
```

Regels voor migraties:

- **Lees elke automatisch gegenereerde migratie na.** Autogenerate ziet niet alles. Een hernoemde kolom of tabel ziet het bijvoorbeeld als "verwijderen + nieuw aanmaken", wat gegevens wist. Gebruik daarvoor `op.rename_table` / `op.alter_column`, zoals migratie `51a15c8d59d2` alle tabellen hernoemde naar `Landing_…` zonder gegevensverlies.
- **Schrijf altijd een werkende `downgrade()`.**
- **Pas nooit een migratie aan die al op test of productie uitgevoerd is.** Maak een nieuwe migratie die de correctie doet.
- **Gegevenswijzigingen** (bv. een modulenaam aanpassen) mogen in een migratie met `op.execute(...)`, met de omgekeerde wijziging in `downgrade()`.
- **Eén lijn.** Als twee ontwikkelaars tegelijk een migratie maken, ontstaan er twee "heads". Los dat op vóór het mergen (`alembic heads` toont het; `alembic merge` of de `down_revision` van de jongste aanpassen).
- Op de server voert `deploy.sh` de migraties automatisch uit.

10.15 Automatische tests

De backend heeft meer dan 600 testfuncties in `backend/tests/`, met dezelfde mappenstructuur als `app/`. Ze dekken inloggen, tokenrotatie, alle toegangsregels en de endpoints van elke module. Draaien:

Tests draaien (map backend)

```
.venv/Scripts/python -m pytest                          # alles
.venv/Scripts/python -m pytest tests/modules/module_3   # één module
.venv/Scripts/python -m pytest -k mailing -x            # op naam filteren, stoppen bij eerste
fout
```

Hoe het werkt (`tests/conftest.py`):

- Elke test krijgt een **lege, tijdelijke SQLite-database in het geheugen**; Docker of PostgreSQL is niet nodig. Alle tabellen worden aangemaakt vanuit de modellen.
- SQLite negeert vreemde-sleutelregels tenzij je ze aanzet; `conftest.py` voert daarom `PRAGMA foreign_keys=ON` uit, zodat bv. het verwijderen van een gebruiker zich gedraagt zoals op PostgreSQL.
- De fixture `client` is een `TestClient` die de echte app aanspreekt, maar met de testdatabase.
- Elke modulemap heeft een `helpers.py` met hulpjes zoals `create_user`, `grant_access`, `give_requests_role` en `login`.
- Externe diensten worden vervangen met `monkeypatch`, bv. `send_email` door een functie die de mails in een lijst bijhoudt.

backend/tests/modules/module_3/test_mailing_list.py (ingekort)

```
def test_mailing_list_crud(client: TestClient, db_session: Session) -> None:
    _boss(db_session)                          # superbeheerder met toegang tot module-3
    login(client, "boss@example.com")
```

```
    created = client.post(LIST, json={"email": " Jan@Example.COM ", "name": " "})
    assert created.status_code == 201
    assert created.json()["email"] == "jan@example.com"# bijgesneden en in kleine letters
```

> **TIP**
>
> Schrijf bij elke nieuwe functie of bugfix minstens één test: voor de normale werking, en voor de rechten (zonder moduletoegang → 403, zonder rol → 403, met de juiste rol → OK). Kopieer daarvoor een bestaand testbestand van een vergelijkbaar scherm.

11 De frontend in detail

ONTWIKKELAARS

> **KORT GEZEGD**
> De frontend gebruikt de **App Router** van Next.js: de mappenstructuur onder `src/app/` bepaalt de adressen. **Layouts** wikkelen pagina's in (inlogcontrole, menu's, modulekleur); **pagina's** halen op de server de gegevens op en geven die aan een **component** in `src/components/module-N/`, die in de browser de interactie verzorgt en via `src/lib/api.ts` met de backend praat. Alle teksten komen uit `messages/nl.json` en `en.json`.

11.1 Mappen zijn adressen

In de App Router wordt elke map onder `src/app/` een deel van het adres, en een bestand `page.tsx` maakt er een bereikbare pagina van. Drie speciale schrijfwijzen:

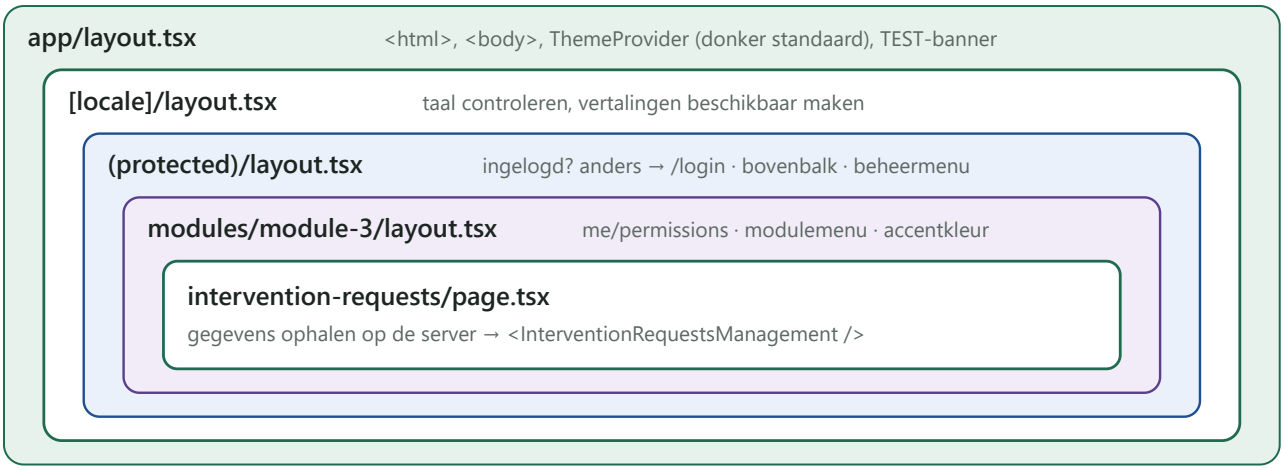
- `[locale]`: een *variabel* stuk. `/nl/…` en `/en/…` komen allebei hier terecht.
- `(protected)`, `(auth)`, `(public)`, `(signed-in)`, `(tabs)`: een *groep* tussen haakjes verschijnt niet in het adres, maar laat toe om een gedeelde layout te geven aan een reeks pagina's.
- `[id]`, `[nummer]`: een variabel stuk, bv. `/m/module-3/request/42`.

Tabel 11.1 – Voorbeelden van map naar adres

Bestand	Adres
`app/[locale]/(protected)/page.tsx`	`/nl`
`app/[locale]/(auth)/login/page.tsx`	`/nl/login`
`app/[locale]/(protected)/modules/module-3/intervention-requests/page.tsx`	`/nl/modules/module-3/intervention-requests`
`app/[locale]/(public)/intervention-request/page.tsx`	`/nl/intervention-request`
`app/[locale]/m/(signed-in)/module-3/request/[id]/page.tsx`	`/nl/m/module-3/request/42`

11.2 De lagen van layouts

Een pagina wordt van buiten naar binnen ingepakt door layouts. Voor een scherm van module 3 zijn dat:



Figuur 11.1 – De geneste layouts rond een scherm van module 3. Elke laag voegt één verantwoordelijkheid toe.

- `app/layout.tsx` is de echte buitenste laag met `<html>` en het thema. Dit staat bewust *boven* de taalmap: stond het erin, dan zou wisselen van taal het hele document opnieuw opbouwen (zie §17.2).
- `[locale]/layout.tsx` weigert onbekende talen (404) en geeft de vertalingen door.
- `(protected)/layout.tsx` vraagt op de server `/api/auth/me`. Niet ingelogd → doorsturen naar de login, nog voordat er iets beschermds naar de browser gaat. Het toont de bovenbalk, het beheermenu (alleen voor de superbeheerder) en levert de open seizoenen aan de seizoenkiezer.
- De **modulelayout** vraagt `me/permissions` op. Bij 403 toont ze "geen toegang"; anders het modulemenu (alleen schermen met *view*) en de accentkleur.
- De **pagina** haalt de gegevens op en geeft ze aan het eigenlijke scherm-component.

11.3 Servercomponenten en clientcomponenten

Next.js kent twee soorten componenten:

- **Servercomponenten** (de standaard; alle layouts en pagina's) draaien alleen op de server. Ze mogen `async` zijn en rechtstreeks gegevens ophalen. Ze kunnen niet reageren op klikken.
- **Clientcomponenten** beginnen met de regel `"use client"`. Ze draaien in de browser, kunnen `useState` en klik-handlers gebruiken, en praten via `lib/api.ts` met de backend. Alle `*-management.tsx`-schermen zijn clientcomponenten.

Het vaste patroon: de **pagina** (server) haalt de begingegevens op en geeft ze als *props* door aan het **scherm** (client), dat daarna zelf wijzigingen doorstuurt en zijn lijst bijwerkt. Zo staat er bij het openen meteen gegevens op het scherm, zonder laadicoon.

11.4 Gegevens ophalen: server en browser

Tabel 11.2 – De drie manieren om de backend aan te spreken

Functie	Waar	Gebruik
`getCurrentUserOnServer()`	`lib/server-auth.ts`	In layouts: wie is ingelogd (of `null`).

`serverApiFetch<T>(pad)`	`lib/server-api.ts`	In pagina's en layouts: gegevens ophalen op de server. Stuurt de cookies van de gebruiker door; gooit `ServerApiError` met de statuscode.
`apiFetch` en de functies in `api.ts`	`lib/api.ts`	In clientcomponenten: één functie per endpoint (`createInterventionRequest`, `listMailingRecipients`, …). Gebruikt `credentials: "include"` en `fetchWithRefresh`; gooit `ApiError` met de melding van de backend.

De types van alle gegevens staan in `lib/types.ts` en volgen de Pydantic-schema's van de backend. Verandert een schema in de backend, pas dan ook het type aan.

> **LET OP**
>
> Een nieuwe `fetch` naar de backend in `api.ts` moet via `fetchWithRefresh` lopen (of via `apiFetch`, dat het al doet). Een kale `fetch` vernieuwt de sessie niet, en de gebruiker krijgt dan na 15 minuten onterecht een fout.

11.5 Componenten

- `components/ui/`: de onderdelen van shadcn/ui (knop, dialoog, tabel, keuzelijst, …). Deze bestanden zijn **gegenereerd** en blijven zoals gegenereerd; stijl pas je aan waar je ze gebruikt.
- `components/shared/`: bovenbalk, beheermenu, gebruikersmenu, taal- en themakiezer, seizoenkiezer en `SeasonProvider`, de "binnenkort"-pagina, wachtwoord wijzigen.
- `components/module-N/`: de schermen van een module. Per module: een `*-sidebar.tsx` (modulemenu), `roles-management.tsx` en `users-management.tsx` (Access Rights), en een `*-management.tsx` per lijstscherm.

Een typisch lijstscherm (`*-management.tsx`) bestaat uit: een titel, een knop "Toevoegen", een tabel met vastgezette kop en Acties-kolom (zie §15.5), een dialoog met het formulier voor toevoegen en wijzigen, een bevestigingsdialoog voor verwijderen, en `toast`-meldingen voor succes of fout. Knoppen verschijnen alleen als de gebruiker het recht heeft.

11.6 Vertalingen

Alle schermteksten staan in `frontend/messages/nl.json` en `en.json`, gegroepeerd per **namespace**: `common`, `login`, `admin`, `errors`, `phone`, en één per module (`tagscan`, `karTracker`, `interventionRequests`, `altsienSelect`, `masterdata`, `mobileApp`). Telefoonteksten van een module staan onder `<module>.phone`.

Een tekst gebruiken

```
// In een clientcomponent
const t = useTranslations("interventionRequests");
toast.success(t("requestCreated"));

// In een servercomponent
const tErrors = await getTranslations("errors");
return <p>{tErrors("forbidden")}</p>;
```

- Elke sleutel moet in **beide** bestanden staan, op dezelfde plaats. Beide bestanden hebben nu exact evenveel regels; houd dat zo.

- Hergebruik een bestaande tekst als die exact past, in plaats van een tweede sleutel met dezelfde tekst te maken.
- Links binnen de app maak je met `Link` en `redirect` uit `@/i18n/navigation`; die voegen de taal automatisch toe.

11.7 Thema en modulekleuren

`lib/module-theme.ts` geeft elke module een accentkleur (Tailwind-kleur), een tegelstijl, een badgestijl en eventueel een pictogram. `getModuleAccentStyle("module-3")` geeft een inline stijl die binnen die module de "primary"-kleur (actief menu-item, standaardknoppen) vervangt door de modulekleur. Een nieuwe module krijgt hier haar kleur.

11.8 Eigenaardigheden van shadcn op Base UI

shadcn/ui genereert in dit project onderdelen op basis van **Base UI**, niet van het bekendere Radix. Online voorbeelden zijn meestal voor Radix geschreven, en twee verschillen zijn verraderlijk:

Tabel 11.3 – Radix-gewoonten die hier niet werken

Radix (werkt hier niet)	Base UI (zo wel)	Waarom belangrijk
`<DialogTrigger asChild><Button/></DialogTrigger>`	`<DialogTrigger render={<Button/>} />`	`asChild` bestaat niet en geeft een typefout.
`<DropdownMenuItem onSelect={…}>`	`<DropdownMenuItem onClick={…}>`	`onSelect` geeft *geen* fout maar wordt nooit uitgevoerd. Heeft al debugtijd gekost.

11.9 Configuratie van de frontend

- `lib/config.ts`: `API_BASE_URL` (voor de browser), `SERVER_API_BASE_URL` (voor servercode; bij de lokale proxy rechtstreeks naar de backend), `SITE_URL` en `ENVIRONMENT_LABEL`.
- `next.config.ts`: `output: "standalone"` (kleine Docker-image), `allowedDevOrigins` (telefoon via tunnel of wifi tijdens `npm run dev`), en een *rewrite* van `/api/*` naar `DEV_API_PROXY_TARGET` als die ingesteld is.
- `src/proxy.ts`: voegt de taal toe aan elk adres (`/` → `/nl`) en vernieuwt de sessie (zie §7.2).

12 De telefoon-app (PWA)

ONTWIKKELAARS

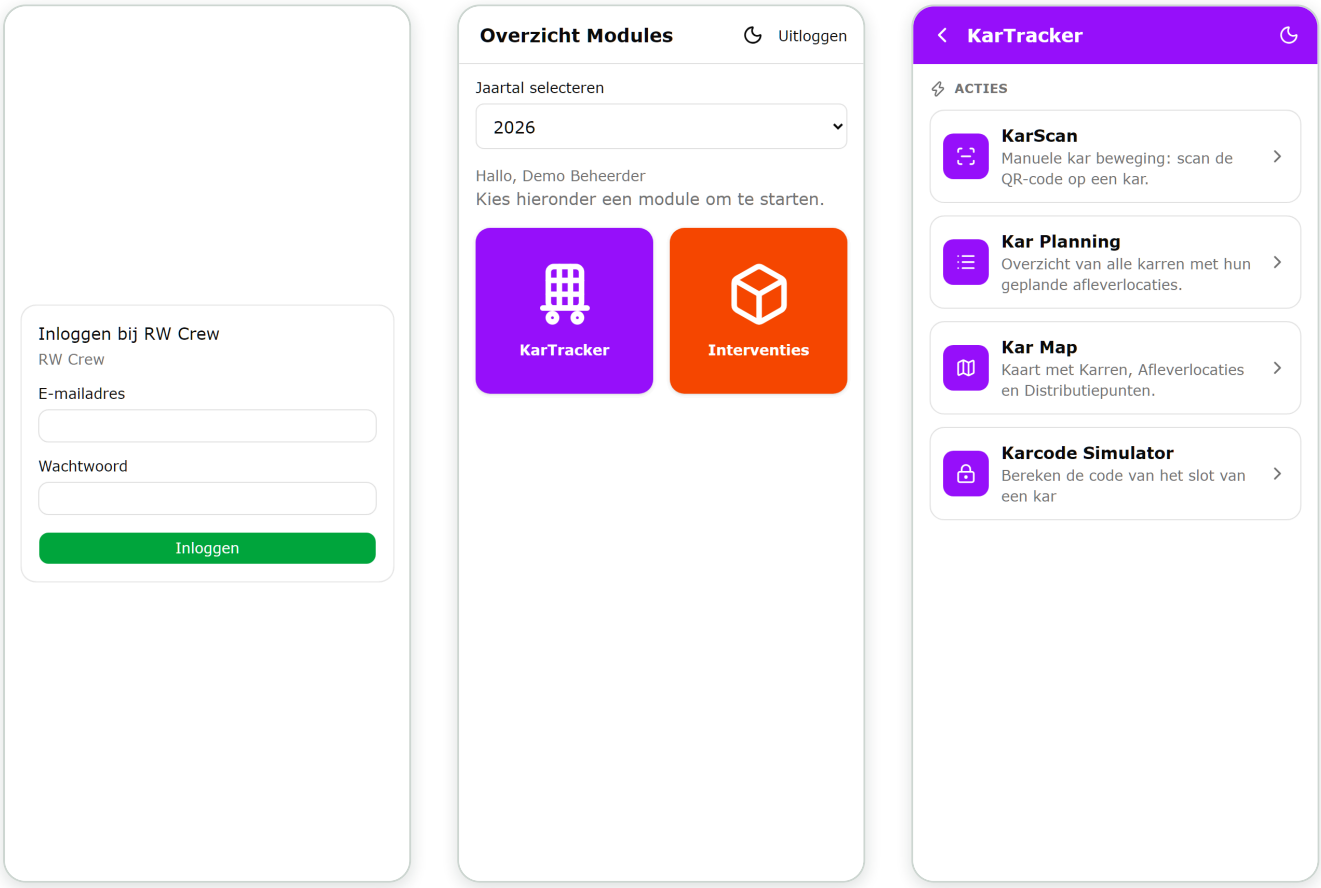
> **KORT GEZEGD**
> De telefoon-app is een **Progressive Web App**: een apart deel van de website onder `/m`, met alleen de telefoonschermen (inloggen, moduletegels, KarTracker en Interventie Aanvragen). Je installeert ze met "Zet op beginscherm". Er is geen App Store, geen aparte API en geen aparte release: dezelfde endpoints, dezelfde login en dezelfde rechten als de website. Ook de telefooncode is per module georganiseerd.

12.1 Waarom een PWA

Tot oktober 2026 bestond er een aparte native app (React Native/Expo) met een eigen login via tokens. Die is volledig verwijderd en vervangen door de PWA, omdat:

- er maar **één codebasis** te onderhouden is, in dezelfde technologie als de website;
- een deployment van de website **meteen** ook de app bijwerkt, zonder goedkeuring van een app store;
- de app dezelfde cookies, endpoints en rechten gebruikt, dus geen dubbele beveiliging nodig heeft.

Alleen oude rijen in de inloghistoriek verwijzen nog naar de native app (bron `mobile`, getoond als "Native app (old)"). Die waarde wordt nooit meer geschreven.



Figuur 12.1 – De telefoon-app: inloggen, de moduletegels met de jaarkeuze, en het menu van KarTracker.

12.2 Installeerbaar maken: het manifest

`frontend/src/app/manifest.ts` beschrijft de app voor de telefoon: naam ("RWCrew", of "RWCrew Test" op test, zodat je beide naast elkaar kunt installeren), iconen (ook een *maskable* versie voor Android), donkergroene achtergrond, `display: "standalone"` (zonder adresbalk) en `start_url: "/m"`. Next.js serveert dat als `/manifest.webmanifest`.

12.3 Structuur

Telefoon-app: routes en componenten

```
src/app/[locale]/m/
├── layout.tsx                    telefoonomhulsel
├── login/page.tsx                inlogpagina (stuurt client = "pwa")
└── (signed-in)/
    ├── layout.tsx                inlogcontrole → anders /m/login; open seizoenen
    ├── page.tsx                  tegels van de telefoonmodules
    ├── module-2/                 KarTracker
    │   ├── layout.tsx            rechten laden; null → PhoneForbidden
    │   ├── page.tsx, scan/, planning/, map/, kar/[nummer]/
    └── module-3/                 Interventie Aanvragen
        ├── layout.tsx
        ├── (tabs)/layout.tsx, page.tsx (Akties), kpi/
        └── request/new/, request/[id]/

src/components/phone/
├── shared/      phone-header, phone-body, phone-tile-grid, phone-menu-row, phone-bottom-tabs,
│                phone-select, phone-season-select, phone-notice, phone-forbidden, phone-
modules.ts
├── module-2/    kar-scan, kar-planning-list, kar-map-phone, …, kartracker-phone-rights.ts/-
routes.ts
└── module-3/    requests-list, request-form, kpi-overview, …, intervention-requests-phone-
rights.ts/-routes.ts
```

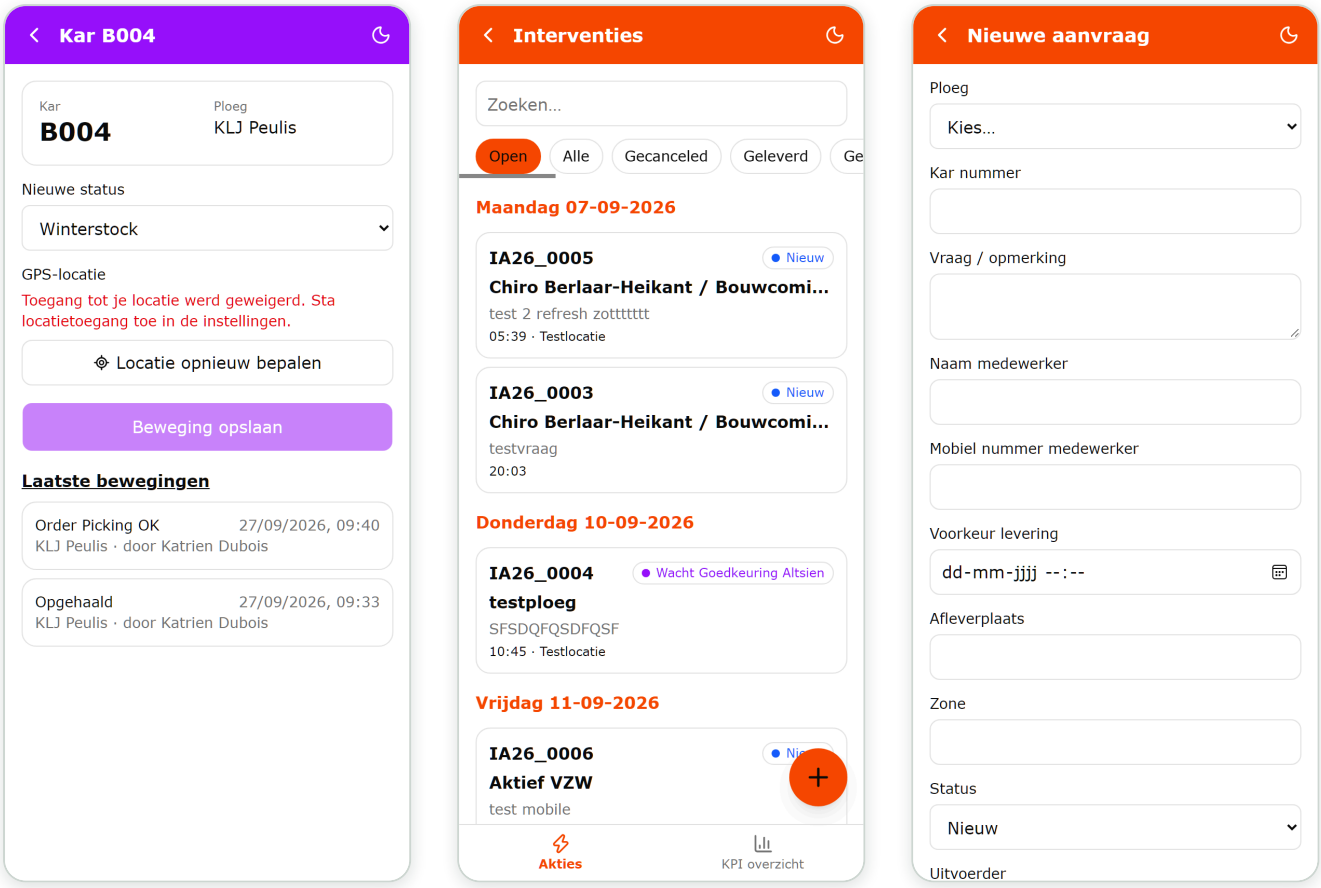
Pagina's blijven **dun**: ze halen op de server gegevens en rechten op en tonen één component. Op de telefoon is er geen zijbalk of bovenbalk van de website, geen verwijderknop en geen PDF.

12.4 Rechten en routes per module

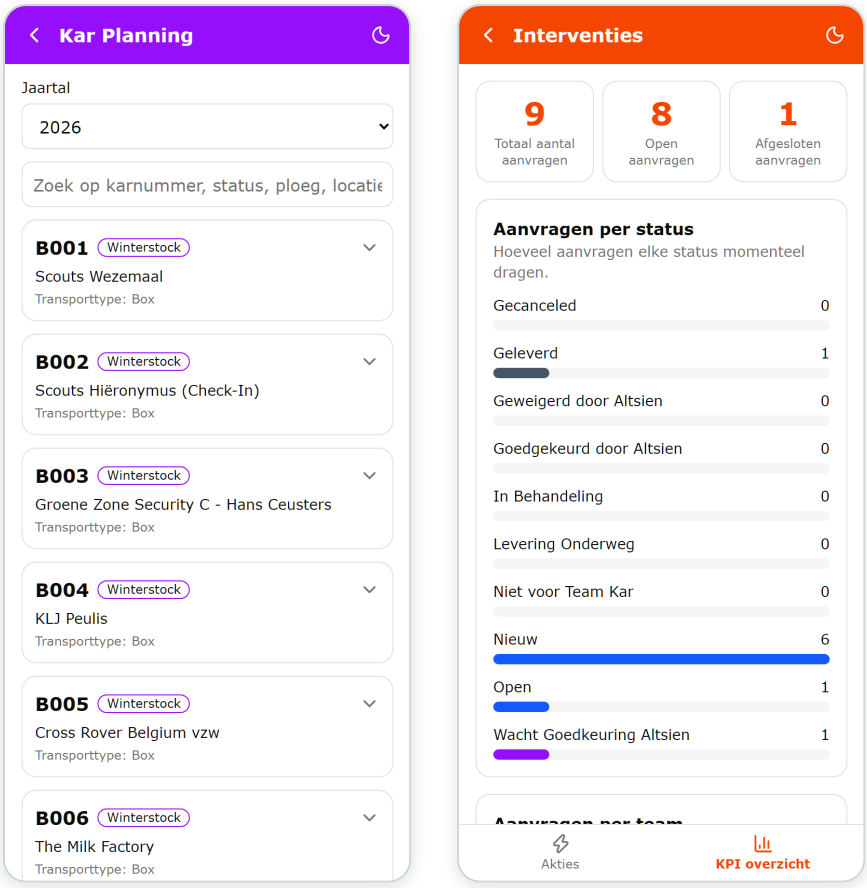
Elke telefoonmodule heeft twee kleine bestanden:

- `*-phone-rights.ts` leest `me/permissions` en zet dat om naar wat de telefoon nodig heeft (bv. `canViewRequests`, `canCreateRequests`, `canEditRequests`). Geeft `null` als de gebruiker geen moduletoegang heeft; de modulelayout toont dan `PhoneForbidden`.
- `*-phone-routes.ts` bevat alle adressen van de module op de telefoon, zodat geen enkel component zelf adressen in elkaar knutselt.

Endpoints die speciaal voor de telefoon bijkwamen, staan gewoon in de router van de module: module 3 `GET /lookups` en `GET /intervention-requests/{id}`, module 2 `GET /seasons`, en `me/permissions` geeft nu ook de lijsten voor aanmaken, wijzigen en verwijderen.



Figuur 12.2 – Telefoonschermen: een kar en haar bewegingen (KarScan), de lijst van interventie-aanvragen (Akties) en een nieuwe aanvraag.



Figuur 12.3 – Kar Planning en het KPI-overzicht van Interventie Aanvragen op de telefoon.

12.5 Code die website en telefoon delen

Code die beide gebruiken, blijft in de map van de module, niet in `phone/`: `components/module-2/qr-scanner.tsx` (camera en QR-herkenning met `@zxing/browser`), `components/module-2/kar-map-rows.tsx` (pins en pop-ups van de Kar Map) en `components/module-3/request-dates.ts` (datumhulpjes).

12.6 Camera, GPS en HTTPS

Browsers geven de camera en GPS alleen vrij op HTTPS (of op `localhost`). Op test en productie is dat geregeld. Lokaal gebruik je de Cloudflare-tunnel (zie §5.2.3). Via het wifi-adres (`http://192.168…`) werkt alles behalve camera en GPS.

12.7 Website of telefoon in de inloghistoriek

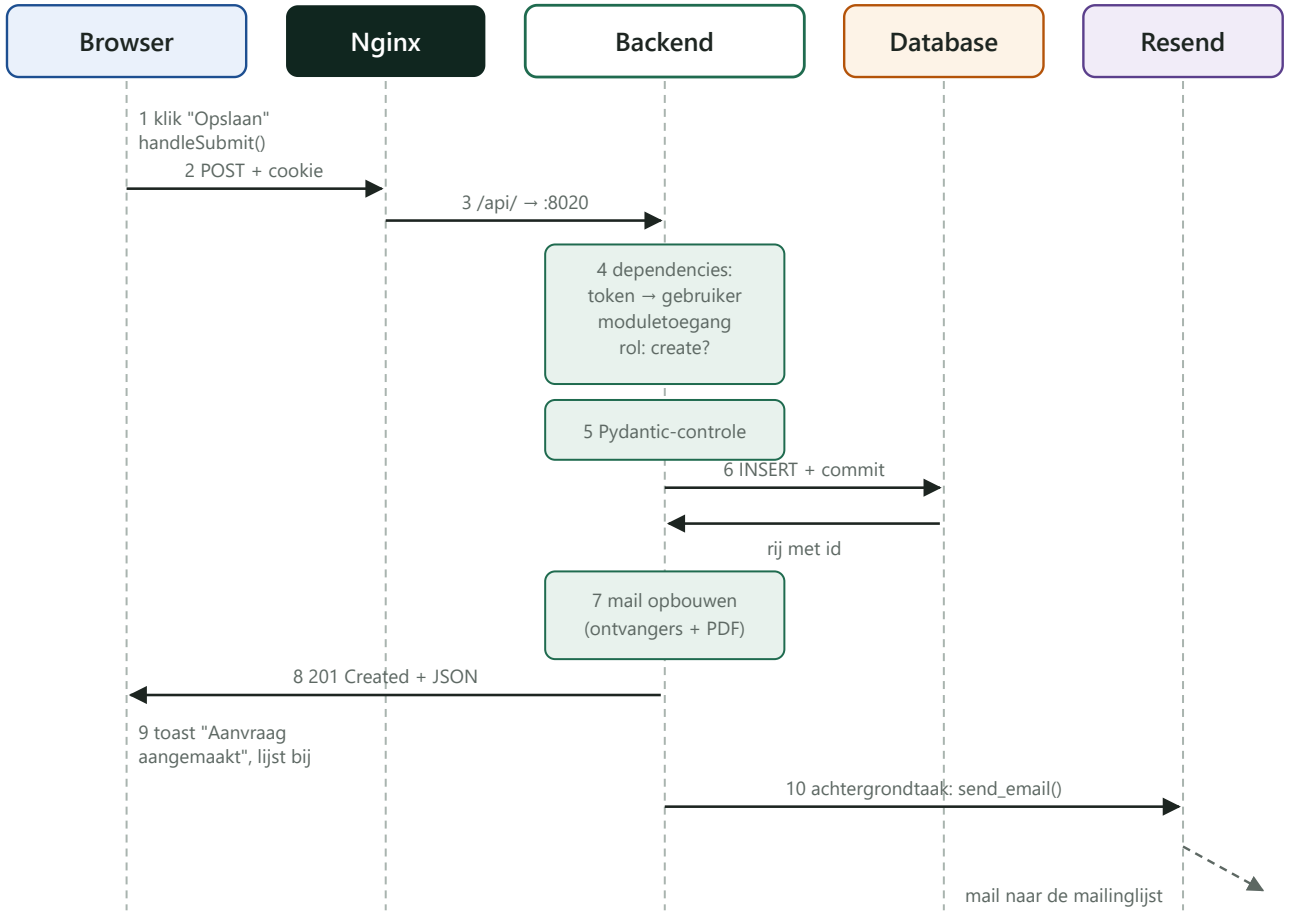
Het loginformulier stuurt `client` mee: `"web"` vanaf `/login`, `"pwa"` vanaf `/m/login`. Dat komt in `Landing_login_history.source`. Het toesteltype (desktop, mobile, tablet) wordt afgeleid uit de User-Agent door `backend/app/shared/device.py`; de ruwe User-Agent wordt bewaard zodat later opnieuw ingedeeld kan worden. Een iPad met iPadOS 13 of later doet zich voor als Mac en telt dus als "desktop".

13 Eén verzoek van begin tot eind

ONTWIKKELAARS

> **KORT GEZEGD**
> Dit hoofdstuk volgt één concrete handeling door het hele systeem: een medewerker maakt in Interventie Aanvragen een nieuwe aanvraag aan. Je ziet welke bestanden en functies achtereenvolgens aan de beurt komen, van de klik in de browser tot de rij in de database en de mail in de mailbox. Wie dit pad begrijpt, begrijpt hoe elk ander scherm werkt.

13.1 Overzicht



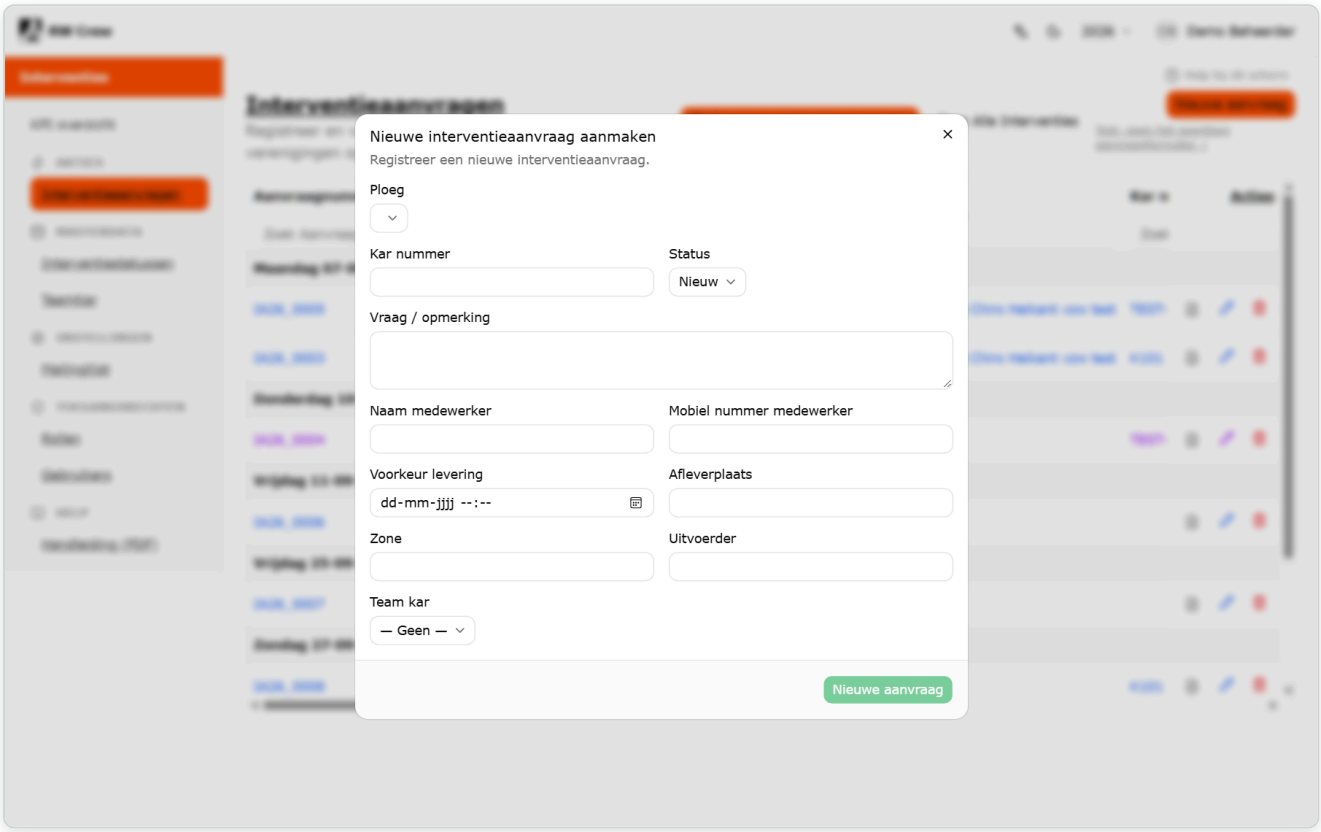
Figuur 13.1 – Het pad van "nieuwe interventie-aanvraag opslaan" door het systeem. De mail vertrekt pas nadat de gebruiker zijn antwoord kreeg.

13.2 Stap 1–2: in de browser

De medewerker vult in het scherm *Intervention Requests* het formulier in en klikt op "Opslaan". Dat scherm is het clientcomponent `components/module-3/intervention-requests-management.tsx`; de klik roept `handleSubmit()` aan:

frontend/src/components/module-3/intervention-requests-management.tsx (ingekort)

```
async function handleSubmit() {
  ...
  const saved = isEditing
    ? await updateInterventionRequest(editing.id, payload)
    : await createInterventionRequest(payload);
  toast.success(isEditing ? t("requestUpdated") : t("requestCreated"));
  ...                                   // lijst bijwerken, dialoog sluiten
  // bij een fout: toast.error(message) met de melding van de backend
}
```



Figuur 13.2 – Het dialoogvenster 'Nieuwe interventieaanvraag aanmaken' waarin de medewerker het formulier invult.

`createInterventionRequest` staat in `lib/api.ts` en roept `apiFetch` aan met `POST` en de gegevens als JSON. `apiFetch` gebruikt `credentials: "include"`, zodat de browser de inlogcookies meestuurt, en loopt via `fetchWithRefresh`: krijgt het verzoek 401 omdat het toegangstoken net verlopen is, dan vernieuwt die functie één keer de sessie en herhaalt het verzoek. De gebruiker merkt daar niets van.

13.3 Stap 3–5: Nginx en de controles in de backend

Nginx ziet een adres dat met `/api/` begint en stuurt het door naar de backend-container. FastAPI vindt het endpoint `create_intervention_request` in `modules/module_3/router.py` en voert eerst de dependencies uit:

1. `get_db` opent een databasesessie;
2. `require_screen_permission("interventionrequests.requests", "create")` roept `require_module_access` aan, die op zijn beurt `get_current_user` gebruikt: cookie `rwcrew_access_token` lezen, handtekening en vervaldatum controleren (`decode_token`), gebruiker laden en nagaan of die actief is (anders 401);

3. moduletoegang voor `module-3` controleren (anders 403);
4. `user_can(...)` : heeft de rol van deze gebruiker *create* op dat scherm (anders 403)?

Daarna controleert FastAPI de JSON tegen het schema `InterventionRequestCreateRequest` : verplichte velden, lengtes, en de regel dat precies één van `team_id` of `team_name` ingevuld is. Bij een fout volgt 422, zonder dat de functie zelf loopt.

13.4 Stap 6–8: opslaan en antwoorden

Pas nu loopt de functie zelf. Ze controleert of de gekozen status, ploeg en TeamKar-medewerker bestaan (anders 404), laat `generate_request_number` (in `module_3/service.py`) het volgende nummer bepalen (bv. `IA26_0042`), maakt het `InterventionRequest` -object, en doet `db.add` , `db.commit` en `db.refresh` . SQLAlchemy vertaalt dat naar een `INSERT INTO "InterventionRequests_request" …` .

Dan roept ze `queue_new_request_mail(db, new_request, background_tasks)` aan. Die bouwt meteen de mail op (actieve adressen uit de mailinglijst, ploegnaam, de leveringsbon als PDF via fpdf2) en plant alleen het versturen als achtergrondtaak. Is de mailinglijst leeg, dan wordt er niets gepland. Mislukt het opbouwen, dan wordt dat gelogd en blijft de aanvraag gewoon bewaard.

FastAPI zet het object om naar JSON volgens `InterventionRequestResponse` en stuurt **201 Created** terug.

13.5 Stap 9–10: terug in de browser, en de mail

`apiFetch` geeft de JSON terug aan `handleSubmit` , dat een groene melding toont en de nieuwe aanvraag in de tabel zet. Ondertussen voert FastAPI, nadat het antwoord vertrokken is, de achtergrondtaak uit: `send_email()` in `core/mail.py` stuurt de mail naar de HTTP-API van Resend, die hem aflevert. Een fout daar komt alleen in de log van de backend.

13.6 Varianten van hetzelfde pad

Tabel 13.1 – Dezelfde handeling via andere ingangen

Ingang	Wat is anders
Telefoon-app	Het formulier is `components/phone/module-3/request-form.tsx` ; daarna exact hetzelfde endpoint, dezelfde controles en dezelfde mail.
Publiek formulier	Endpoint in `module_3/public_router.py` , zonder login. In de plaats van de rechtencontrole komt een snelheidslimiet per IP-adres. De status wordt automatisch "Nieuw". Daarna dezelfde opslag en dezelfde mail.
Pagina openen	Bij het openen van het scherm haalt `intervention-requests/page.tsx` op de server met `serverApiFetch` de lijst, statussen, ploegen en TeamKar-leden op, en geeft ze aan het clientcomponent. Was het toegangstoken verlopen, dan heeft `proxy.ts` de sessie al vóór het renderen vernieuwd.

> **TIP**
>
> Bij een probleem volg je hetzelfde pad met de ontwikkelaarstools van de browser (`F12` > Network): welk verzoek vertrok, met welke gegevens, welke statuscode kwam terug, en welke `detail` -melding gaf de backend? Een 403 wijst naar de rechten, een 422 naar de invoer, een 500 naar de backend-log (`docker compose … logs backend`).

14 Recepten: veelvoorkomende uitbreidingen

ONTWIKKELAARS

> **KORT GEZEGD**
> Stap-voor-stap-lijsten voor de wijzigingen die het vaakst voorkomen. Volg ze in volgorde; ze verwijzen telkens naar een bestaand voorbeeld dat je kunt kopiëren. Werk altijd op een eigen branch en sluit af met tests en een controle op test (zie hoofdstuk 16).

14.1 Een nieuw scherm in een bestaande module

Voorbeeld: een scherm "Leveranciers" in module 9 (MasterData).

Backend

1. **Model**: `app/db/models/supplier.py` met `__tablename__ = "MasterData_supplier"`; importeer het in `app/db/base.py`.
2. **Migratie**: `alembic revision --autogenerate -m "add masterdata supplier"`, nalezen, `alembic upgrade head` (zie §10.14).
3. **Schema's**: `SupplierResponse`, `SupplierInput` in `app/schemas/masterdata.py`.
4. **Scherm registreren**: `ScreenDefinition(key="masterdata.suppliers", label="Suppliers", sort_order=…)` in `module_9/screens.py`.
5. **Endpoints** in `module_9/router.py`: `GET/POST /suppliers`, `PUT/DELETE /suppliers/{id}`, elk met `require_screen_permission("masterdata.suppliers", "view" | "create" | "edit" | "delete")`.
6. **Tests** in `tests/modules/module_9/test_suppliers.py`: CRUD én rechten. Kopieer een bestaand testbestand van een vergelijkbaar scherm.

Frontend

1. **Types** in `lib/types.ts` en **API-functies** in `lib/api.ts` (via `apiFetch`).
2. **Scherm**: `components/module-9/supplier-management.tsx`. Kopieer een vergelijkbaar lijstscherm (bv. `warehouse-management.tsx`, of gebruik `lookup-management.tsx` voor een eenvoudige opzoeklijst), met vette en onderstreepte titel en kolomkoppen en de vaste tabelkop en Acties-kolom (zie hoofdstuk 15).
3. **Pagina**: `app/[locale]/(protected)/modules/module-9/suppliers/page.tsx`: gegevens ophalen met `serverApiFetch`, bij 403 de melding "forbidden" tonen, anders het scherm.
4. **Menu**: een link toevoegen in `components/module-9/masterdata-sidebar.tsx`, zichtbaar als `viewableScreenKeys` het scherm bevat.
5. **Teksten** onder `masterdata.*` in `nl.json` én `en.json`.

Herstart de backend. Het nieuwe scherm verschijnt als lege rij in elke rechtenmatrix van MasterData; geef je eigen rol de rechten om het te testen.

14.2 Een kolom of tabel toevoegen of wijzigen

1. Pas het model aan (nieuw veld met type, `nullable` en eventueel `default` / `server_default`).
2. Een nieuwe verplichte kolom in een tabel met bestaande rijen heeft een standaardwaarde nodig (`server_default`), of een migratie in drie stappen: kolom toevoegen als nullable, bestaande rijen vullen, dan verplicht maken.
3. Genereer en lees de migratie na; test `upgrade` én `downgrade` lokaal.
4. Pas schema's, frontend-types en schermen aan.
5. Draai alle tests.

14.3 Een nieuwe module bouwen (bv. module 7 echt maken)

Zo werden Atelier (module 5) en Walkies (module 6) gebouwd; kopieer bij twijfel van Walkies, de kleinste volledige module.

1. **Naam**: voeg de naam toe aan `_MODULE_NAMES` in `app/modules/registry.py`, en maak een migratie die `UPDATE "Landing_modules" SET name = '…' WHERE key = 'module-7'` uitvoert (met de omgekeerde wijziging in `downgrade()`).
2. **Rechtensysteem**: volg `docs/module-custom-roles-pattern.md` stap voor stap: vier modellen (`<Naam>_screens`, `_roles`, `_role_permissions`, `_user_roles`), migratie, schema's, `screens.py`, `deps.py`, `router.py`. Kopieer van de module die het meest lijkt op wat je bouwt (module 3 is het meest overzichtelijke voorbeeld; module 6 Walkies is de kleinste en meest recente).
3. **main.py**: importeer de nieuwe router en `sync_screens`, en roep die in `lifespan` aan.
4. **Tests**: kopieer `tests/modules/module_3/test_roles.py` en `helpers.py` en pas sleutels en namen aan.
5. **Frontend**: types en API-functies; `components/module-7/` met sidebar, `roles-management.tsx`, `users-management.tsx`; `app/[locale]/(protected)/modules/module-7/layout.tsx` (rechten ophalen, menu, accentkleur) en de pagina's onder `access-rights/roles` en `access-rights/users`.
6. **Uitstraling**: kleur en eventueel pictogram in `lib/module-theme.ts`.
7. **Teksten**: een nieuwe namespace in beide vertaalbestanden.
8. **Documentatie**: `CLAUDE.md`, deze handleiding (tabel 3.1 en bijlage B).

Daarna geeft de superbeheerder zichzelf moduletoegang, maakt een eerste rol "Beheerder" met alle rechten aan en kent die toe aan de toekomstige modulebeheerder.

14.4 Een stap toevoegen aan de Ploeg Wizard

1. Een `StepDefinition` toevoegen in `backend/app/modules/module_8/steps.py` (stabiele `key`, volgorde, eventueel een `completion_check` die zegt wanneer de stap "klaar" mag worden gezet).
2. Een component in `frontend/src/components/module-8/wizard/steps/` en het registreren in de `index.ts` van die map. Onbekende sleutels tonen een plaatshouder, dus er breekt niets zolang dit ontbreekt.
3. Titel en beschrijving onder `altsienSelect.steps.<key>` in beide vertaalbestanden.

4. Optioneel een eigen sectie in `module_8/ploegfiche_pdf.py`.
5. Optioneel "kopiëren van vorig seizoen": een paar `list_lines` / `copy_lines` in `module_8/previous_season.py`, geregistreerd in `COPYABLE_STEPS`.

De voortgang wordt per stapsleutel bewaard; er is geen migratie nodig. Zo werd de stap "products" van plaatshouder een echte stap: ze schrijft via `module_4/requirement_service.py` de benodigdheden per kar van StockMaster, met eigen endpoints in module 8 zodat een kernlid geen StockMaster-rechten nodig heeft, en een eigen "kopiëren van vorig seizoen" per kar.

14.5 Een module toevoegen aan de telefoon-app

1. De modulesleutel toevoegen aan `PHONE_MODULE_KEYS` in `components/phone/shared/phone-modules.ts`.
2. `app/[locale]/m/(signed-in)/module-N/layout.tsx` maken die de rechten laadt en bij `null` `PhoneForbidden` toont.
3. Schermen in `components/phone/module-N/`, met een `*-phone-rights.ts` en een `*-phone-routes.ts`. Gebruik de gedeelde onderdelen uit `phone/shared/`.
4. Teksten onder `<moduleNamespace>.phone.*`; bestaande teksten van het webscherm hergebruiken waar ze passen.
5. Testen op een echte telefoon via de tunnel (camera/GPS) en via wifi.

14.6 Een schermtekst toevoegen of wijzigen

Zoek de sleutel in `frontend/messages/nl.json` (zoeken op de huidige tekst), pas hem aan in `nl.json` én `en.json`. Een nieuwe tekst krijgt een sleutel in de namespace van de module, op dezelfde plaats in beide bestanden. Een frontend-herstart is lokaal meestal niet nodig.

14.7 Een nieuwe instelling

Zie §6.5: `config.py`, de `environment:`-lijst in `docker-compose.prod.yml`, de `.env.*.example`-bestanden, en de echte waarde op de server.

14.8 Een Excel-import toevoegen

Kopieer een bestaand importbestand dat qua vorm lijkt (bv. `module_9/festival_import.py` of `module_2/kar_import.py`). Het patroon: een functie die het sjabloon bouwt, een functie die de geüploade werkmap rij per rij verwerkt in een eigen SAVEPOINT en per rij een resultaat teruggeeft, en eventueel een export. Verwijzingen naar andere tabellen worden in het bestand *bij naam* gegeven en bij het importeren opgezocht. Voeg endpoints toe voor sjabloon, import en export, en de knoppen op het scherm "Data Upload/Download" van de module.

14.9 Een nieuwe mail toevoegen

Volg het patroon van `module_3/notifications.py`: een functie die binnen de databasesessie een `OutgoingMail` opbouwt (of `None` als er niets te sturen valt), en een functie die het versturen als `BackgroundTasks`-taak plant. Gebruik `settings.mail_subject_prefix` voor het onderwerp en

`settings.app_public_url` voor links; escape alle gebruikersinvoer in de HTML. Vervang `send_email` in de tests met `monkeypatch`.

15 Codeerafspraken

ONTWIKKELAARS

> **KORT GEZEGD**
> RWCrew heeft een paar vaste afspraken die de code leesbaar en voorspelbaar houden: commentaar in gewone taal bij elk logisch blok, alles per module, vaste naamgeving met voorvoegsels, en een vaste uitstraling voor lijstschermen (vette en onderstreepte titels en kolomkoppen, tabellen met vaste kop en Acties-kolom). Nieuwe code volgt deze afspraken vanaf de eerste versie.

15.1 Commentaar in gewone taal

Elk logisch blok of statement krijgt commentaar in gewone (Engelse) taal dat uitlegt *wat* het doet en vooral *waarom*. Niet letterlijk elke regel, en niet alleen een docstring bovenaan. Bovenaan elk bestand staat waarom het bestaat. Kijk naar eender welk bestand onder `backend/app/` of `frontend/src/` voor de gewenste dichtheid. Gegenereerde bestanden (Alembic-migraties, `src/components/ui/*`) blijven zoals gegenereerd.

De reden: het project moet begrijpelijk zijn voor wie nieuw is in de code, of zelfs nieuw in programmeren. Commentaar dat het *waarom* vastlegt, voorkomt dat iemand later een bewuste keuze terugdraait.

15.2 Alles per module

Nieuwe code komt in de mappen van de module waartoe ze behoort: `backend/app/modules/module_N/`, `frontend/src/app/[locale]/(protected)/modules/module-N/`, `frontend/src/components/module-N/`, `frontend/src/components/phone/module-N/`, en een eigen vertaalnamespace. Alleen wat echt door meerdere modules gedeeld wordt, komt in `shared/`. Gedeelde logica rond gegevens van één module staat bij die module als service (zie §10.9).

15.3 Naamgeving

Tabel 15.1 – Naamgevingsregels

Wat	Regel	Voorbeeld
Databasetabel	`<Module>_<naam>`	`InterventionRequests_status`, `Landing_users`
Modelklasse	PascalCase met modulevoorvoegsel	`InterventionRequestsRole`
Modelbestand	snake_case, één per tabel	`intervention_requests_role.py`
Modulesleutel (adres)	`module-N` (verandert nooit)	`module-3`
Python-pakket	`module_N`	`app/modules/module_3/`
Schermsleutel	`<module>.<scherm>`, kleine letters	`interventionrequests.mailinglist`
Vertaalnamespace	camelCase per module	`interventionRequests`, `karTracker`
React-componentbestand	kebab-case	`mailing-list-management.tsx`
React-component	PascalCase	`MailingListManagement`
Lijstscherm	`<ding>-management.tsx`	`festival-management.tsx`

> **LET OP**
> Een **schermsleutel**, **stapsleutel** of **modulesleutel** verander je nooit meer zodra ze bestaat: ze staat in de database (rechten, voortgang, toegang). Een *label* (wat de gebruiker ziet) mag je vrij aanpassen.

15.4 Lijstschermen: titels en kolomkoppen

Op elk lijst- of beheerscherm (`*-management.tsx`) zijn de paginatitel en alle kolomkoppen **vet en onderstreept**, voor een duidelijke scheiding tussen kop en gegevens:

- de `<h1>`-titel: `font-bold … underline` (niet `font-semibold`);
- elke `<TableHead>` : `font-bold underline` toegevoegd aan de bestaande klassen (uitlijning zoals `text-right` blijft).

Pas dat toe waar je de component gebruikt, niet in `ui/table.tsx` zelf (gegenereerd bestand).

15.5 Tabellen met vaste kop en Acties-kolom

Elke lijsttabel houdt bij het scrollen de **kopregel** en de **Acties-kolom** in beeld, en scrolt in beide richtingen binnen een kader van maximaal 65% van de schermhoogte, in plaats van de hele pagina te laten groeien. Het referentievoorbeeld is `components/module-3/intervention-requests-management.tsx` . Het patroon bestaat uit vier delen, altijd samen:

Het tabelpatroon (vereenvoudigd)

```
<div className="rounded-md border [&>div]:max-h-[65vh] [&>div]:overflow-y-auto">   {/* 1 */}
  <Table>
    <TableHeader>
      <TableRow>
        <TableHead className="sticky top-0 z-20 bg-background font-bold
underline">Naam</TableHead>   {/* 2 */}
        <TableHead className="sticky top-0 right-0 z-30 bg-background font-bold
underline">Acties</TableHead>  {/* 3 */}
      </TableRow>
    </TableHeader>
    <TableBody>
      <TableRow className="group">                                              {/* 4 */}
        <TableCell>…</TableCell>
        <TableCell className="sticky right-0 z-10 bg-background group-hover:bg-muted/50">…
</TableCell>
      </TableRow>
    </TableBody>
  </Table>
</div>
```

1. De omhullende `div` beperkt de hoogte en scrolt verticaal. Zet hier géén `overflow-x-auto` : de tabel heeft al een eigen horizontaal scrollende container.
2. Elke kopcel is `sticky top-0 z-20 bg-background` . Een tweede kopregel (bv. filters) gebruikt `top-10` .
3. De kopcel van Acties is ook `right-0` en `z-30` , zodat de hoek boven de andere vaste cellen blijft.
4. Elke rij heeft `group` , en de Acties-cel volgt met `group-hover:bg-muted/50` de hoverkleur van de rij.

`bg-background` is verplicht op elke vaste cel; anders schuift de inhoud er zichtbaar onderdoor. Een tabel zonder Acties-kolom krijgt alleen deel 1 en 2. Het gedeelde `module-9/lookup-management.tsx` volgt het patroon al.

15.6 Belgische tijd overal

Elke datum en elk tijdstip in RWCrew (schermen, telefoon-app, PDF's, mails, dashboards, Excel-exports) is in **Belgische tijd** (Europe/Brussels), nooit UTC. De database bewaart tijdstippen met tijdzone; de omzetting gebeurt op één plaats:

- **frontend**: alleen via `frontend/src/lib/date-time.ts`. Nooit `slice()` op een ISO-tekst en nooit `toLocaleString()`: dat toont UTC of de tijdzone van de pc;
- **backend**: via `app/core/timezone.py` (`to_belgian`, `belgian_now`, `belgian_today`), nooit `datetime.now()` of `date.today()` voor iets wat de gebruiker ziet;
- databasesessies draaien in Europe/Brussels (`app/core/database.py`), zodat een datum en uur uit een formulier zonder tijdzone als Belgische tijd gelezen worden.

De aanleiding: op Tag Headerdata stond het tijdstip eerst in UTC, één tot twee uur verschoven.

15.7 Afspraken in de backend

- **Rechten in elk endpoint** via `require_screen_permission`; nooit vertrouwen op een verborgen knop.
- **Velden die de server bepaalt** (nummers, tijdstippen, aanmaker) staan niet in het create-schema.
- **Instellingen** altijd via `settings`, nooit hardgecodeerd.
- **Nevenwerk faalt veilig**: mails en andere bijkomende acties mogen de hoofdbewerking nooit ongedaan maken.
- **Leesbare foutmeldingen**: `HTTPException(status_code=…, detail="…")`; de frontend toont `detail` rechtstreeks.
- **Pure Python-pakketten** (zie §8.6).
- **Tests** voor elke nieuwe functie en elke regel in de rechten.

15.8 Afspraken in de frontend

- **Pagina's zijn dun**: gegevens ophalen op de server, één component tonen. Logica en interactie in het component.
- **Backend alleen via** `lib/api.ts` (browser) of `lib/server-api.ts` (server); geen losse `fetch`-aanroepen in componenten.
- **Geen hardgecodeerde schermteksten**: alles via de vertaalbestanden.
- **Knoppen volgens de rechten**: toon "Toevoegen", "Wijzigen", "Verwijderen" alleen bij het overeenkomstige recht.
- **Base UI**: `render={…}` in plaats van `asChild`, `onClick` in plaats van `onSelect`.
- **Lijstschermen**: vet en onderstreept, vaste kop en Acties-kolom.
- Voor een commit: `npm run lint` en, bij grotere wijzigingen, `npm run build`.

15.9 Commits en branches

- Kleine, samenhangende commits met een beschrijvende boodschap in de gebiedende wijs: "Add mailing list to intervention requests", "Fix season lock for organisation role".
- Eén onderwerp per branch: `feature/leveranciers`, `fix/karblad-qr`.
- Nooit geheimen committen. Controleer voor elke commit wat je toevoegt (`git status`, `git diff --staged`).

16 Samenwerken als team

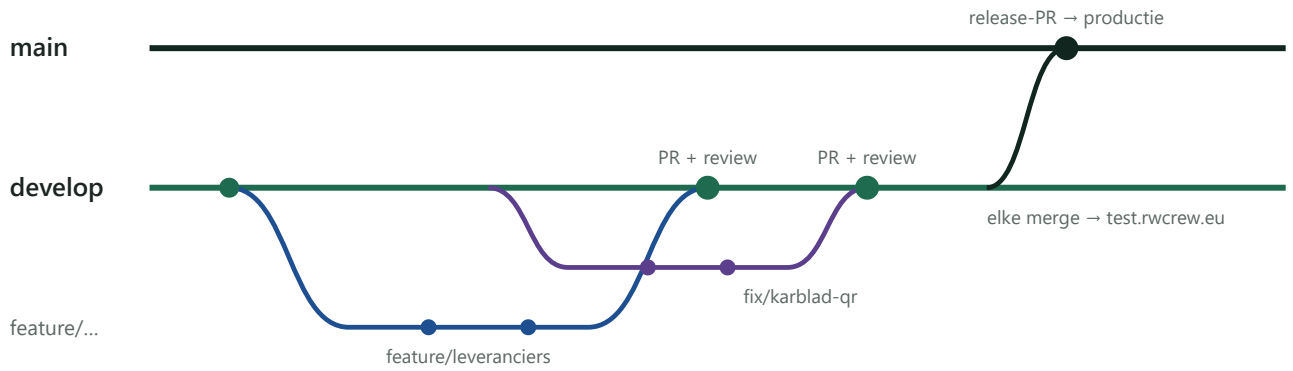
IEDEREEN

> **KORT GEZEGD**
> Tot nu toe werd rechtstreeks naar `develop` en `main` gepusht, wat meteen deployt. Met meerdere mensen is dat te riskant. Dit hoofdstuk beschrijft een **aanbevolen werkwijze**: werken op aparte branches, wijzigingen via pull requests laten nalezen, `main` en `develop` beschermen, een gezamenlijke "definition of done", en een duidelijk beheer van wie toegang heeft tot welke systemen. Het gaat om voorstellen: spreek ze in het team af en pas aan waar nodig.

16.1 De huidige situatie

RWCrew werd tot nu toe door één persoon ontwikkeld. Wijzigingen gingen rechtstreeks naar `develop` (en zo naar test), en daarna via een merge naar `main` (en zo naar productie). Er zijn geen beschermde branches, geen verplichte review en geen automatische tests bij een push. Dat werkte omdat er maar één persoon was die alles overzag.

16.2 Aanbevolen branchmodel



Figuur 16.1 – Aanbevolen branchmodel: werk op korte feature-branches, merge via pull requests in `develop`, en breng `develop` via een release-PR naar `main`.

1. Start elke wijziging vanaf een bijgewerkte `develop`: `git switch develop && git pull && git switch -c feature/<naam>`.
2. Werk en commit op je branch; push hem naar GitHub (`git push -u origin feature/<naam>`).
3. Open een **pull request naar** `develop` met een korte beschrijving: wat, waarom, hoe testen, eventuele migraties of nieuwe instellingen.
4. Een collega leest na (zie §16.5). Na goedkeuring merge je; GitHub Actions zet het op test.
5. Laat de gebruiker(s) die erom vroegen het op `test.rwcrew.eu` uitproberen.
6. Voor een release: een pull request van `develop` naar `main`, nagelezen door een tweede persoon, op een rustig moment gemerged (zie §6.8).

7. Een dringende oplossing in productie (*hotfix*): branch vanaf `main`, PR naar `main`, en daarna dezelfde wijziging ook in `develop` mergen zodat de branches niet uit elkaar lopen.

16.3 Branches beschermen op GitHub

Stel in GitHub (Settings > Branches > Branch protection rules, of "Rulesets") voor `main` en `develop` in:

- wijzigingen alleen via een pull request;
- minstens één goedkeuring van iemand anders dan de auteur;
- geen *force push* en niet verwijderen;
- zodra er een testworkflow is (zie hieronder): de tests moeten slagen voor een merge.

16.4 Aanbevolen: automatische tests bij elke pull request

De bestaande workflows deployen alleen; ze testen niets. Een aanbevolen aanvulling is een workflow `.github/workflows/ci.yml` die bij elke pull request de backend-tests (`pytest`, met Python 3.14) en de frontend-controles (`npm ci`, `npm run lint`, `npm run build`) uitvoert. Zo vangt GitHub een gebroken test of een build-fout op voordat iemand mergt. De tests hebben geen database nodig (SQLite in het geheugen), dus de workflow is eenvoudig.

16.5 Code review

Bij het nalezen van een pull request let je op:

- ☐ Doet de wijziging wat de beschrijving zegt, en niet meer?
- ☐ Controleert elk nieuw endpoint de rechten (`require_screen_permission`)?
- ☐ Zijn er tests voor de nieuwe werking en de rechten, en slagen ze?
- ☐ Is een migratie nagelezen, met een werkende `downgrade()`, en zonder gegevensverlies?
- ☐ Staan nieuwe teksten in beide vertaalbestanden?
- ☐ Volgen nieuwe lijstschermen de tabel- en typografieafspraken?
- ☐ Is het commentaar duidelijk, vooral het *waarom*?
- ☐ Zijn nieuwe instellingen toegevoegd aan `docker-compose.prod.yml` en de voorbeeldbestanden?
- ☐ Zijn `CLAUDE.md` en deze handleiding bijgewerkt waar nodig?
- ☐ Staan er geen geheimen, testwachtwoorden of debugcode in?

16.6 Definition of done

Een wijziging is pas **klaar** als:

1. de code de afspraken uit hoofdstuk 15 volgt;
2. alle backend-tests slagen en er nieuwe tests zijn voor de nieuwe werking;
3. de frontend zonder fouten bouwt en lint;
4. een collega de pull request heeft goedgekeurd;
5. de wijziging op `test.rwcrew.eu` gecontroleerd is, door de aanvrager waar mogelijk;
6. de documentatie (`CLAUDE.md`, deze handleiding, eventueel `docs/module-custom-roles-pattern.md`) klopt.

16.7 Rollen in het team

Tabel 16.1 – Voorgestelde rollen (één persoon kan meerdere rollen hebben)

Rol	Verantwoordelijk voor
Product owner	Beslist wat er gebouwd wordt en in welke volgorde; aanspreekpunt voor de gebruikers.
Ontwikkelaar	Bouwt en test wijzigingen, leest pull requests van anderen na.
Release-verantwoordelijke	Beslist wanneer `develop` naar `main` gaat, volgt de checklist, controleert na de deployment.
Serverbeheerder	VPS, Nginx, certificaten, back-ups, `.env`-bestanden, toegang tot Hostinger en Cloudflare.
Functioneel beheerder	Gebruikers, moduletoegang, rollen, stamgegevens (zie hoofdstuk 18).

Houd taken en vragen bij op één plaats, bijvoorbeeld **GitHub Issues** met een label per module, zodat een pull request naar het issue kan verwijzen ("Closes #12").

16.8 Toegangen en geheimen beheren

Tabel 16.2 – Systemen waarvoor toegang beheerd moet worden

Systeem	Geeft toegang tot	Wie
GitHub (repository-beheerder)	Code, secrets, workflows, branchbescherming	Minstens twee beheerders
GitHub (schrijfrechten)	Branches pushen, pull requests	Ontwikkelaars
SSH naar de VPS	Alles op de server, inclusief `.env` en databases	Serverbeheerder(s)
Hostinger-account	De server zelf, snapshots, facturatie	Serverbeheerder(s)
Cloudflare-account	DNS van `rwcrew.eu`	Serverbeheerder(s)
Resend-account	Uitgaande mail, API-sleutels	Serverbeheerder(s)
Superbeheerder in RWCrew	Gebruikers en moduletoegang op test/productie	Functioneel beheerder(s)
`rwcrew_readonly` (pgAdmin)	Alle gegevens lezen	Ontwikkelaars die moeten debuggen

- Bewaar gedeelde wachtwoorden en sleutels in een **wachtwoordbeheerder** met een gedeelde kluis, nooit in mails, chat of Git.
- Zorg dat **minstens twee mensen** elk kritiek account kunnen beheren, zodat het project niet stilvalt als iemand onbereikbaar is.
- Gebruik persoonlijke accounts en SSH-sleutels per persoon waar mogelijk, zodat je iemand kunt uitschakelen zonder anderen te storen.
- **Bij vertrek** van een teamlid: GitHub-toegang intrekken, zijn SSH-sleutel van de server verwijderen, gedeelde wachtwoorden wijzigen, en bij twijfel `JWT_SECRET` en `RESEND_API_KEY` vernieuwen (een nieuwe `JWT_SECRET` logt iedereen uit).

17 Bekende valkuilen

ONTWIKKELAARS

> **KORT GEZEGD**
> Een verzameling problemen die tijdens de ontwikkeling echt voorgekomen zijn, met hun symptoom, oorzaak en oplossing. Herken je een symptoom, kijk dan eerst hier. Ontdek je een nieuwe valkuil, voeg die dan toe aan dit hoofdstuk en aan `CLAUDE.md`.

17.1 De backend reageert met oude code (`--reload` op Windows)

Symptoom: endpoints geven willekeurig 404, 405 of 500, of gedragen zich alsof een oplossing nooit toegepast is, terwijl de tests (die altijd de actuele code gebruiken) slagen.

Oorzaak: `uvicorn --reload` start op Windows de eigenlijke server als apart kindproces via `multiprocessing`. Stop je alleen het ouderproces (het proces dat `Get-NetTCPConnection` als eigenaar van de poort toont), dan blijft het kind als wees de poort bezetten met de oude code. Verzoeken gaan dan willekeurig naar de oude of de nieuwe server.

Oplossing: start de backend zonder `--reload` en herstart met de hand. Zoek en stop achtergebleven processen:

PowerShell: achtergebleven backendprocessen vinden

```
Get-CimInstance Win32_Process -Filter "Name = 'python.exe'" |
  Where-Object { $_.CommandLine -match 'uvicorn|multiprocessing' } |
  Select-Object ProcessId, CommandLine

Stop-Process -Id <ProcessId> -Force      # voor elk gevonden proces
netstat -ano | findstr :8020              # controle: luistert er nog iets?
```

Blijft een poort "behekst", wijk dan uit naar een nieuwe poort en pas `frontend/.env.local(.example)`, `frontend/src/lib/config.ts`, `backend/README.md` en `CLAUDE.md` aan.

17.2 Taal wisselen laat de hele pagina flitsen

Oorzaak: de `<html>` / `<body>`-laag en de `ThemeProvider` moeten in `src/app/layout.tsx` staan, *boven* `[locale]`. Staan ze in de taallayout, dan wordt bij elke taalwissel het hele document opnieuw opgebouwd. React 19 waarschuwt daarnaast over het inline `<script>` van next-themes; `theme-provider.tsx` omzeilt dat door in de browser `scriptProps` met `type: "application/json"` mee te geven. Laat dat staan tot next-themes het zelf oplost.

17.3 Overige valkuilen

Tabel 17.1 – Symptoom, oorzaak en oplossing van bekende problemen

Symptoom	Oorzaak	Oplossing

Backend kan lokaal de database niet bereiken	De lokale PostgreSQL draait op poort **5433**, niet 5432 (5432 is bezet door het andere RWCrew.be-project).	`DATABASE_URL` met poort 5433; `docker compose up -d db`.
Inloggen lijkt te lukken, maar je komt terug op het loginscherm	Frontend-adres en `NEXT_PUBLIC_API_URL` gebruiken een andere hostnaam (`localhost` vs `127.0.0.1`, of een tunneladres): de cookie hoort bij een andere site.	Zelfde hostnaam, of de proxy-instelling met lege `NEXT_PUBLIC_API_URL` (zie §5.2.3).
Sporadisch uitgelogd bij meerdere tabbladen	Twee gelijktijdige vernieuwingen met hetzelfde (geroteerde) token.	Single-flight in `proxy.ts` / `session-refresh.ts` en `api.ts` behouden.
Rotatie van tokens faalt "willekeurig" in tests	Zonder `jti` zijn twee tokens in dezelfde seconde identiek.	De `jti` in `security.py` laten staan.
Verwijderen in tests laat wezen achter	SQLite negeert `ON DELETE CASCADE` zonder `PRAGMA foreign_keys=ON`.	Staat in `tests/conftest.py`; niet weghalen.
Menu-item doet niets	`onSelect` op een Base UI-menu-item wordt nooit uitgevoerd.	`onClick` gebruiken; zoek op `onSelect` in `src/components/`.
Typefout "asChild does not exist"	Base UI kent geen `asChild`.	`render={<Button … />}`.
Nieuwe instelling werkt lokaal, niet op de server	Ontbreekt in de `environment:`-lijst van `docker-compose.prod.yml`.	Zie §6.5.
Gewijzigde `NEXT_PUBLIC_…`-waarde heeft geen effect	Wordt bij het bouwen ingebakken.	Frontend opnieuw bouwen (deploy), niet alleen herstarten.
Raspberry Pi krijgt "413" bij grote CSV's	Nginx-limiet van 1 MB; na certbot ontbreekt `client_max_body_size` in de 443-blokken.	Regel en `limit_req` ook in de 443-blokken zetten.
Half geschreven CSV-bestanden in Scans	De scanner-app schrijft rechtstreeks naar `.csv`.	Eerst `naam.csv.tmp` schrijven en dan hernoemen; unieke bestandsnamen. Zelfde naam + zelfde inhoud = duplicaat; zelfde naam + andere inhoud = bewaard als `naam__<hash>.csv`.
API-sleutel van een scanner wordt geweigerd	Sleutel aangemaakt op een andere omgeving.	Een sleutel per omgeving aanmaken (test en productie apart).
Superbeheerder ziet een module niet	Superbeheerder krijgt geen automatische moduletoegang (bewust).	Zichzelf het vinkje geven in Manage Access.
Nieuw scherm verschijnt niet in de rechtenmatrix	`sync_screens()` loopt alleen bij het opstarten.	Backend herstarten.
Kolom of tabel kwijt na een migratie	Autogenerate zag een hernoeming als verwijderen + aanmaken.	Migraties altijd nalezen; `rename_table` / `alter_column` gebruiken.
Module 10 toont "niet geconfigureerd"	`APP_PUBLIC_URL` niet ingesteld.	Instelling invullen op de server (en in `docker-compose.prod.yml`).
iPad telt als pc in de inloghistoriek	iPadOS 13+ doet zich voor als Mac.	Bekende beperking; de bron (`pwa`) toont wel dat het de telefoon-app was.
Scans-bestanden worden twee keer verwerkt of de automatische scan loopt dubbel	Meerdere backendprocessen (workers) starten elk hun eigen automatische scan.	Eén uvicorn-proces per omgeving houden, of de taak naar één worker of cron verhuizen; noodstop met `TAGSCAN_AUTO_SCAN_WORKER=false`.

CSV met `Mode` = andere omgeving komt niet aan	`TAGSCAN_ENVIRONMENT`, `TAGSCAN_FORWARD_URL` of `TAGSCAN_FORWARD_API_KEY` ontbreekt, of de sleutel is niet op de andere omgeving gemaakt.	Alle drie instellen in elke `.env` (en `docker-compose.prod.yml`). Een doorgestuurd bestand krijgt de header `X-Scans-Forwarded: 1` en wordt nooit teruggestuurd.
Tijdstippen één of twee uur verschoven	Een tijd in UTC getoond (bv. `slice()` op een ISO-tekst).	`lib/date-time.ts` en `app/core/timezone.py` gebruiken (zie §15.6).
Foutmelding in StockMaster toont een code als `insufficient_stock|…`	Nieuwe `StockError`-code zonder vertaling.	Vertaling toevoegen onder `stockMaster.errors.<code>` in `nl.json` en `en.json`.
Verwarring met een "RWCrew"-project in `c:\Fre Software\RWCrew.be`	Ander, los project met dezelfde naam.	Negeren; het heeft niets met deze repository te maken.

DEEL D

Dagelijks beheer

Taken die je uitvoert zonder te programmeren: gebruikers en rechten, seizoenen en stamgegevens, scanners, de telefoon-app, controleren of alles draait en wat te doen bij een storing.

Doelgroep: functionele beheerders en serverbeheerders.

18 Beheer zonder te programmeren

BEHEERDERS

> KORT GEZEGD
> Bijna alle beheer gebeurt in RWCrew zelf. De **superbeheerder** maakt gebruikers aan en beslist wie welke module mag openen (*Manage Access*). Elke **modulebeheerder** bepaalt in zijn module met rollen wat die gebruikers mogen (*Access Rights*). Seizoenen en andere stamgegevens beheer je in **MasterData**. Voor de server zijn er een paar eenvoudige controles: de gezondheids-URL, het tabblad Actions op GitHub, en bij twijfel de logs.

18.1 De superbeheerder

De superbeheerder is een gewone gebruiker met de vlag *Super admin*. Hij ziet links een beheermenu met:

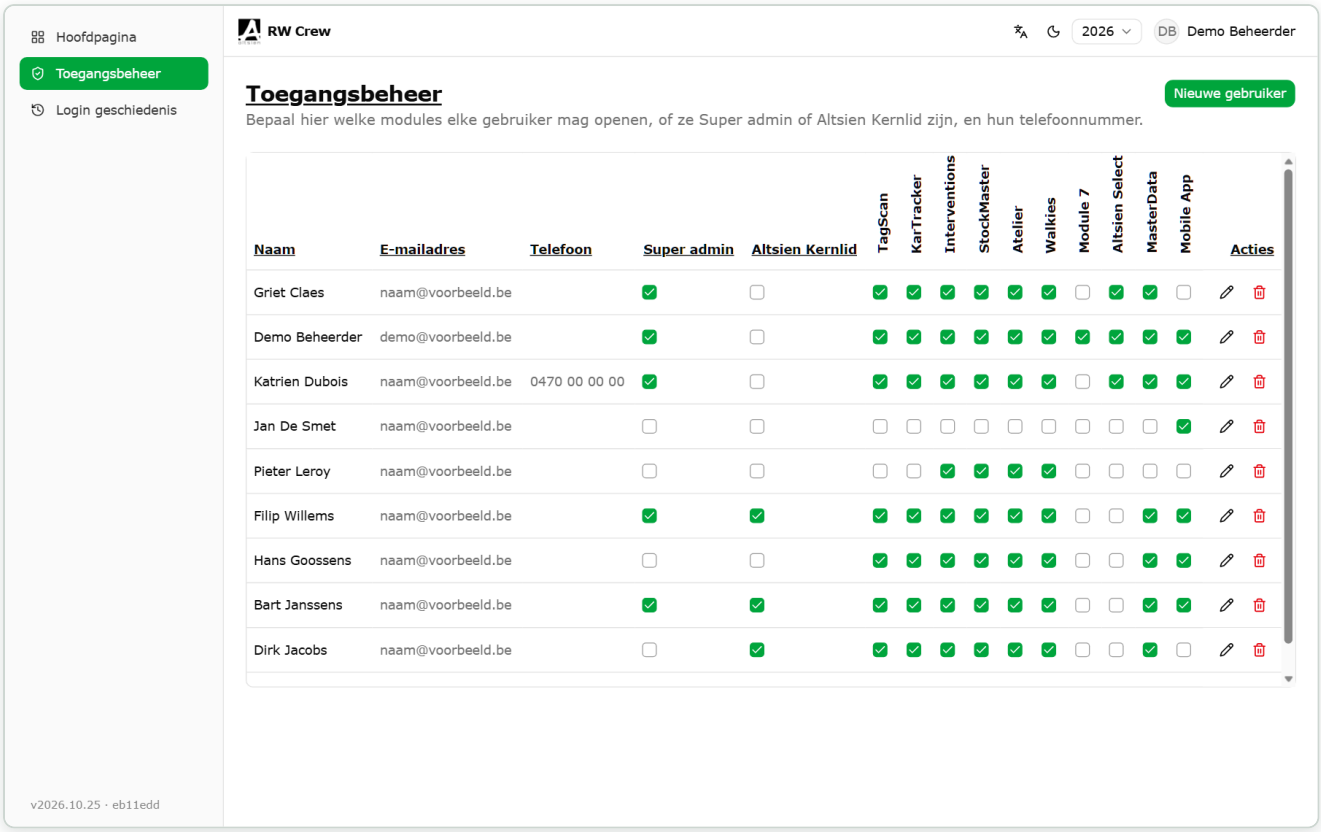
- **Manage Access**: gebruikers aanmaken, gegevens en vlaggen aanpassen, wachtwoorden resetten, verwijderen, en per gebruiker de modules aanvinken die hij mag openen;
- **Login History**: alle inlogpogingen.

De eerste superbeheerder wordt bij de installatie aangemaakt met het seed-script. Daarna kan elke superbeheerder anderen superbeheerder maken. Zorg dat er altijd **minstens twee** zijn. Onthoud: superbeheerder zijn geeft geen automatische toegang tot modules (zie §7.3.1).

18.2 Een nieuwe gebruiker toevoegen

1. Open **Manage Access** en kies "gebruiker toevoegen". Vul e-mailadres, weergavenaam, een tijdelijk wachtwoord (minstens 8 tekens) en eventueel een telefoonnummer in. Vink *Altsien Kernlid* aan als de persoon de Ploeg Wizard moet invullen, en *Super admin* alleen als dat echt nodig is.
2. Vink in het raster de **modules** aan die de gebruiker mag openen.
3. Vraag de **beheerder van elke module** (of doe het zelf als superbeheerder met moduletoegang) om in die module onder *Access Rights* > *Users* een **rol** toe te kennen. Zonder rol ziet de gebruiker de moduletegel wel, maar binnen de module geen schermen.
4. Bezorg de gebruiker het adres (`https://rwcrew.eu`) en het tijdelijke wachtwoord via een veilig kanaal, en vraag hem het wachtwoord meteen te wijzigen (gebruikersmenu rechtsboven > wachtwoord wijzigen).

> TIP
> Een modulebeheerder kan ook zelf vanuit *Access Rights* > *Users* van zijn module een gebruiker aanmaken of toegang geven. Die gebruiker krijgt dan toegang tot die ene module, niet tot andere.



Figuur 18.1 – Manage Access: gebruikers aanmaken en per gebruiker de modules aanvinken. Rechtsboven de knop ‘Nieuwe gebruiker’.

18.3 Toegang intrekken, wachtwoord resetten, verwijderen

Tabel 18.1 – Veelvoorkomende wijzigingen aan gebruikers

Ik wil…	Doe dit
iemand een module afnemen	Manage Access: het vinkje bij die module weghalen. Vanaf dan antwoordt elke pagina van die module met "geen toegang".
iemand minder rechten geven in een module	In die module, Access Rights > Users: een andere rol kiezen, of de rechtenmatrix van zijn rol aanpassen (geldt dan voor iedereen met die rol).
een vergeten wachtwoord oplossen	Manage Access: een nieuw wachtwoord instellen. Alle sessies van die gebruiker worden beëindigd.
iemand definitief verwijderen	Manage Access: verwijderen. Zijn moduletoegang, rollen en sessies verdwijnen mee. Je kunt jezelf niet verwijderen.

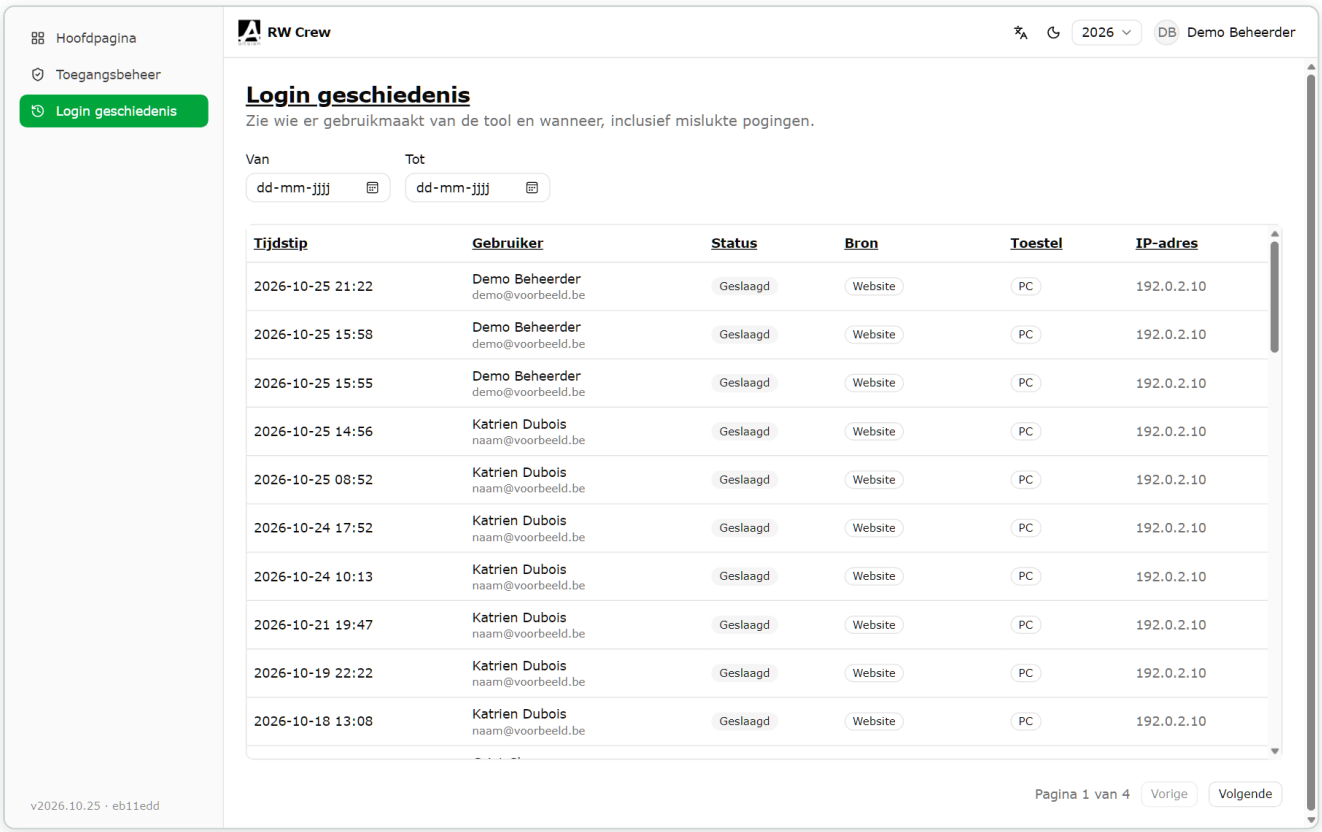
18.4 Rollen beheren binnen een module

1. Open de module en ga naar *Access Rights* > *Roles* (je hebt daarvoor zelf de juiste rechten nodig).
2. Maak een rol met een duidelijke naam die zegt wat iemand doet ("Logistiek", "Kernlid", "Alleen lezen").
3. Vink in de rechtenmatrix per scherm aan wat de rol mag: zien, aanmaken, wijzigen, verwijderen. Bewaar.
4. Ken de rol toe onder *Access Rights* > *Users*.

Tips: geef "verwijderen" spaarzaam; geef de schermen *Roles* en *Users* alleen aan modulebeheerders; en zorg dat elke module minstens twee mensen heeft die rollen kunnen beheren. Een overzicht van alle schermen per module staat in bijlage B.

18.5 De inloghistoriek bekijken

Login History toont per poging: tijdstip, e-mailadres, geslaagd of niet, IP-adres, bron (website, telefoon-app, of "Native app (old)" voor oude rijen) en toesteltype. Gebruik het om te zien wie de applicatie gebruikt, of om verdachte pogingen te herkennen (veel mislukte pogingen op één e-mailadres of van één IP-adres).



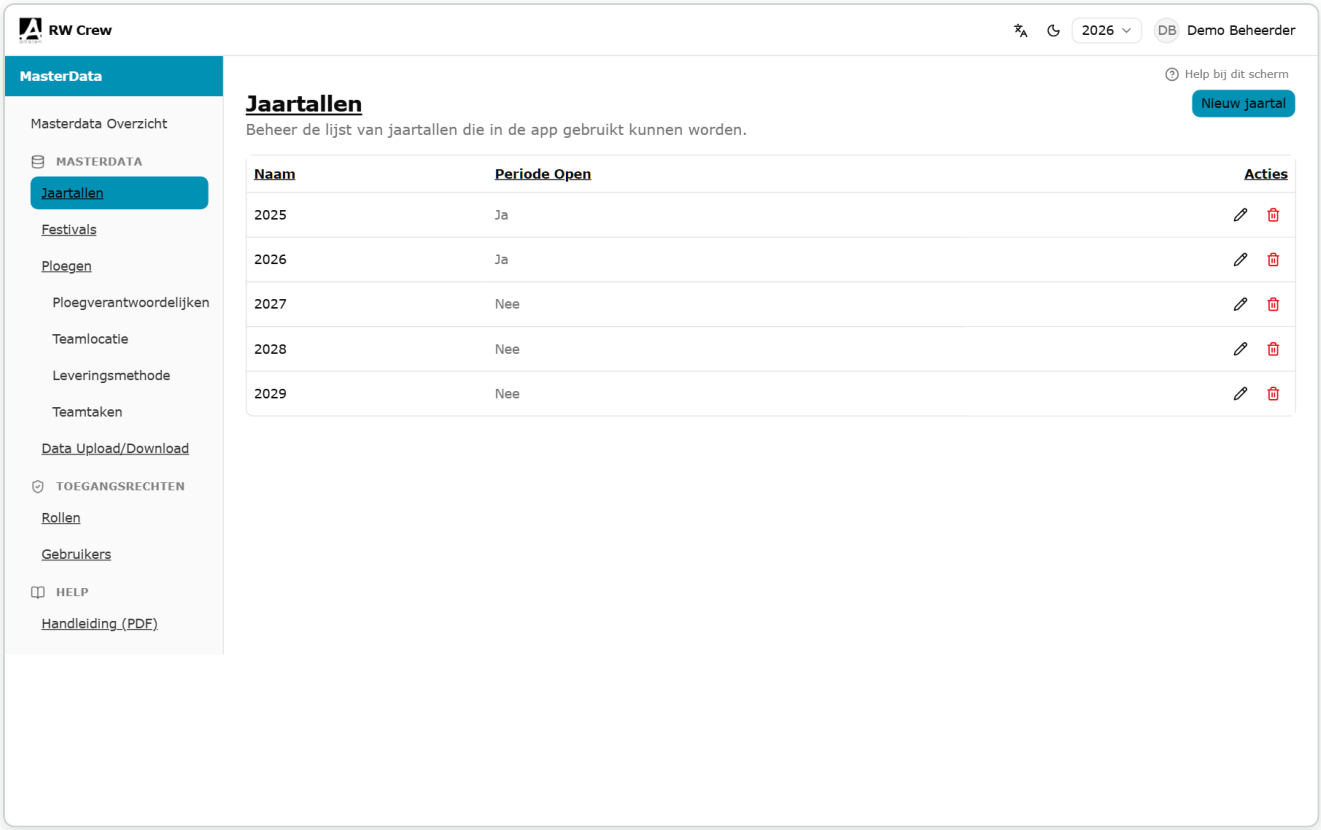
Figuur 18.2 – Login History: elke inlogpoging met tijdstip, gebruiker, resultaat, bron (website of telefoon-app), toestel en IP-adres.

18.6 Seizoenen en stamgegevens

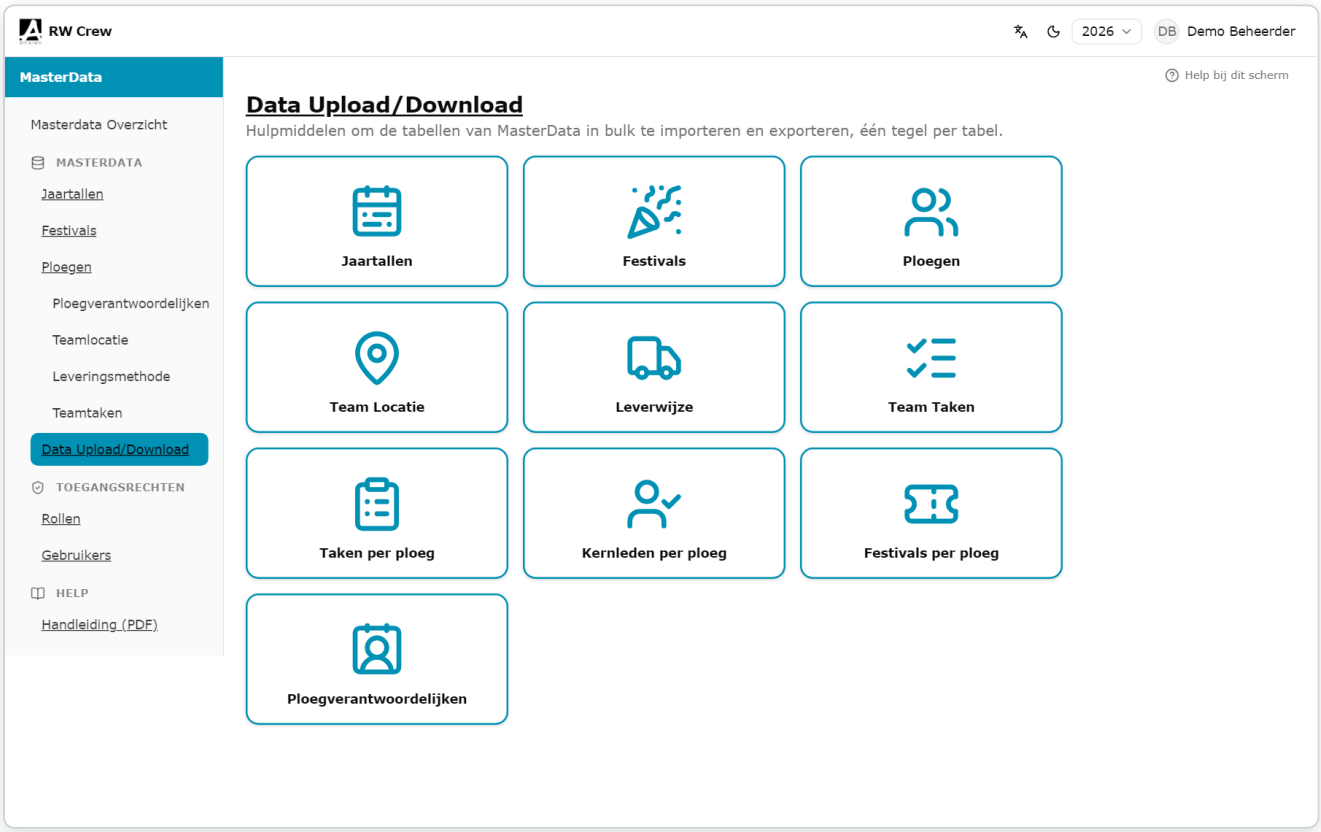
Stamgegevens beheer je in **MasterData** (module 9). De belangrijkste jaarlijkse taak is het **nieuwe seizoen**:

1. MasterData > *Years*: een nieuw seizoen toevoegen (bv. "2027"). Namen worden alfabetisch gesorteerd, en "vorig seizoen" is het seizoen dat net vóór het huidige sorteert; gebruik dus altijd hetzelfde formaat (vier cijfers).
2. Zet *periode open* aan zodra ploegen hun keuzes mogen maken. Open seizoenen verschijnen in de seizoenkiezer bovenaan.
3. Kijk festivals, ploegen en locaties (MasterData) en de producten (StockMaster > Masterdata) na voor het nieuwe jaar. Grote aantallen gaan het snelst via *Data Upload/Download*: sjabloon downloaden, invullen, uploaden. Je krijgt per rij te zien of het gelukt is.
4. In Altsien Select kunnen kernleden bij verschillende stappen de keuzes van vorig seizoen overnemen.

5 Na de inschrijvingsperiode: *periode open* uitzetten. Vanaf dan kunnen alleen gebruikers met het recht "organisatie" in Altsien Select nog wijzigen.



Figuur 18.3 – MasterData · Jaartallen: de seizoenen en of hun periode open is.

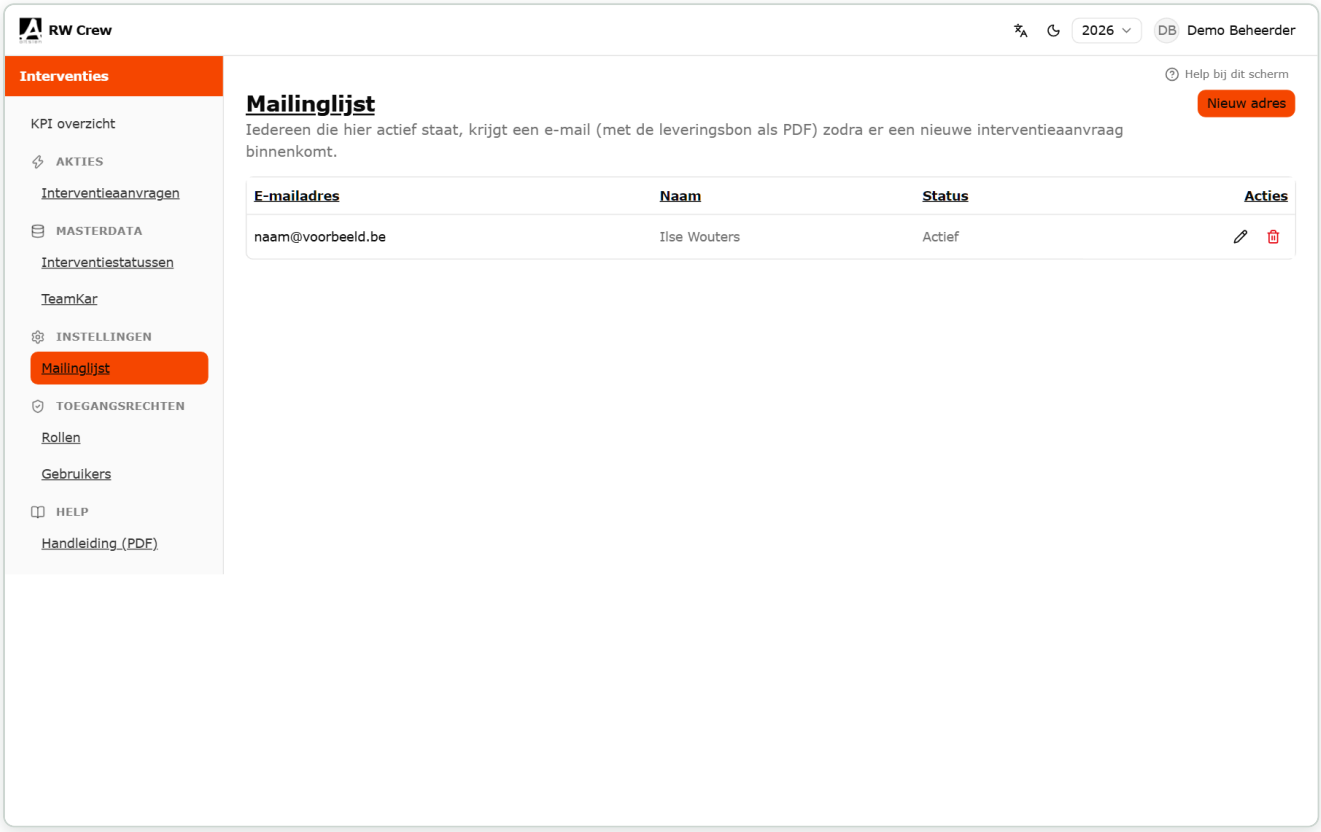


Figuur 18.4 – MasterData · Data Upload/Download: per tabel een sjabloon downloaden, invullen en weer uploaden.

Altsien-kernleden aan ploegen koppelen: zet de vlag *Altsien Kernlid* bij de gebruiker (Manage Access), en koppel hem in MasterData > *Teams* aan zijn ploeg(en). Hij ziet in Altsien Select alleen die ploegen.

18.7 De mailinglijst van Interventie Aanvragen

In Interventie Aanvragen > *Instellingen* > *Mailinglijst* beheer je wie een mail krijgt bij elke nieuwe aanvraag. Je kunt een adres tijdelijk *inactief* zetten zonder het te verwijderen. Op test begint het onderwerp met "[TEST]". Komen er geen mails aan, zie dan tabel 18.3.



Figuur 18.5 – Interventie Aanvragen · Mailinglijst: wie een mail krijgt bij elke nieuwe aanvraag.

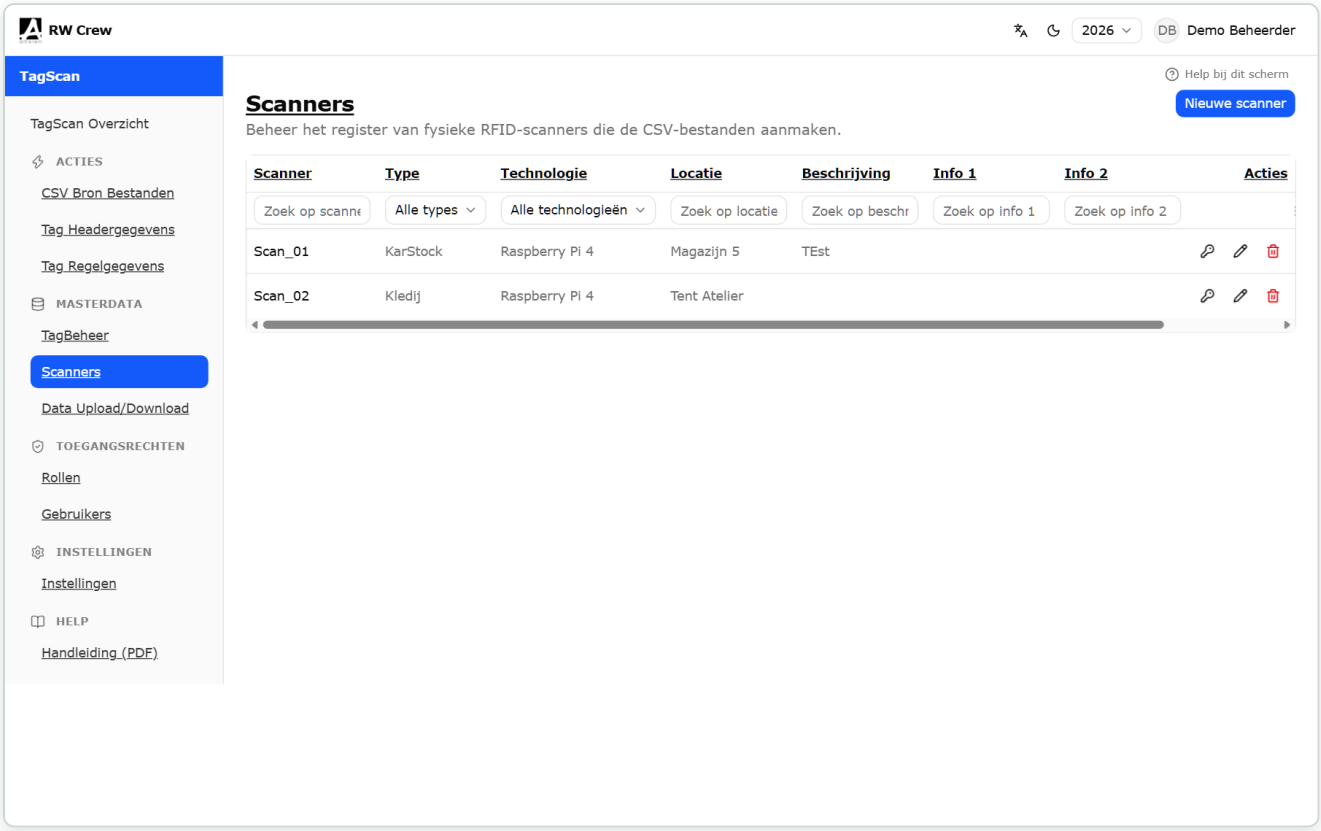
18.8 Een Scans-scanner toevoegen

1. Scans > *Scanners*: de scanner toevoegen en een **API-sleutel genereren**. De sleutel wordt maar één keer getoond; kopieer hem meteen.
2. Zet de sleutel in het configuratiebestand van de watcher op de Raspberry Pi (`/etc/tagscan-watcher/prod.env` voor productie of `dev.env` voor test; zie `scripts/pi-watcher/README.md`) en herstart de watcher.
3. Controleer na een scan of het bestand onder *CSV Source Files* verschijnt, en druk op **Scan** om het in te lezen.

Een sleutel werkt alleen op de omgeving waar hij gemaakt is. Kwijt of gelekt? Genereer een nieuwe; de oude werkt dan niet meer. De volledige installatie van een Pi staat in `docs/tagscan-raspi-setup-guide.pdf`.

Automatisch scannen en acties verwerken. Op het scherm *Instellingen* van Scans zet je het automatisch scannen aan of uit en kies je het interval (standaard 60 seconden). Staat er een oranje balk boven de Scans-schermen, dan wachten er regels met een actie op verwerking: open de balk, kies per bestand het product en

klik op verwerken, of annuleer een foute regel met een reden.



Figuur 18.6 – Scans · Scanners: de geregistreerde scanners; via het sleutelpictogram maak je een nieuwe API-sleutel aan.

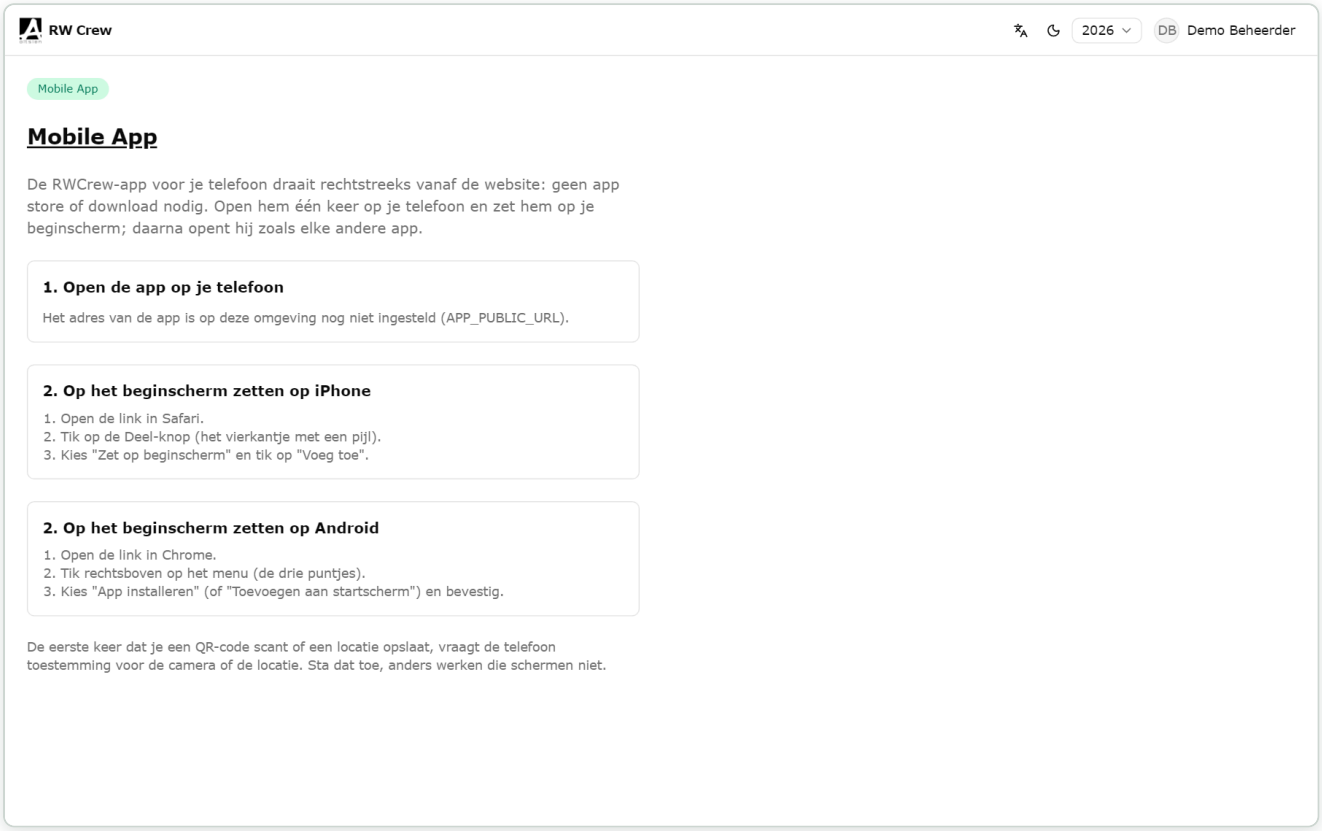
18.9 De polsbandscans van Atelier koppelen

Het polsbandbedrijf stuurt elke scan aan de Atelier-balie naar `https://<adres>/api/public/wristband-scan`. Zo zet je de koppeling op (de echte koppeling kan pas vanaf mei 2027 getest worden):

1. Maak op Scans > *Instellingen* met **Genereer token** de token aan die in elk bericht moet staan (test en productie elk hun eigen). Hij wordt maar één keer getoond; bewaard wordt enkel een hash in `Tagscan_settings`.
2. Geef het bedrijf het webhookadres en de PDF *Uitleg koppeling polsbandbedrijf* van dat scherm, en bezorg de token apart en beveiligd.
3. Na de eerste scan verschijnt elke scanner vanzelf in de lijst *Scanners*; geef hem een naam (bv. "Balie 1"). Aan elke balie kies je bovenaan welke scanner daar staat.
4. Kijk bij *Ontvangen berichten* na wat er echt binnenkomt: geweigerde berichten (verkeerde token, geen JSON) en berichten waarvan de ploegnaam niet overeenkomt met een ploeg uit MasterData zijn daar te zien, met het bericht zoals het binnenkwam.
5. Zonder het bedrijf test je met *Instellingen* > *Test-scan*: een formulier of een JSON-bestand gaat door dezelfde verwerking, zonder token, en verschijnt live aan de balie van die scanner.

18.10 De telefoon-app laten installeren

Geef de gebruiker toegang tot module 10 (*Mobile App*). Die pagina toont het adres en een QR-code. De gebruiker scant de code, opent het adres in Safari (iPhone) of Chrome (Android) en kiest "Zet op beginscherm" (iPhone: deelknop) of "App installeren" (Android). De app opent daarna schermvullend op `/m`. Een update is niet nodig: de app is altijd de nieuwste versie van de website.



Figuur 18.7 – Mobile App (module 10): het adres van de telefoon-app met de installatiestappen. Lokaal staat er 'niet geconfigureerd' omdat `APP_PUBLIC_URL` daar leeg is; op test en productie staat hier de QR-code.

18.11 Controleren of alles draait

Tabel 18.2 – Snelle controles

Controle	Verwacht resultaat
`https://rwcrew.eu/api/health` in een browser	`{"status":"ok","version":"…"}` : de backend draait, met de versie
Versielabel onderaan het beheermenu	Dezelfde commit als de laatste op GitHub (zie §6.6)
`https://rwcrew.eu/nl/login`	Het loginscherm: frontend en Nginx draaien
GitHub > tabblad *Actions*	De laatste deployments zijn groen
Op de server: `docker compose -f docker-compose.prod.yml ps`	Drie containers met status "running" / "Up"
Op de server: `… logs --tail=200 backend`	Geen herhaalde fouten (`Traceback` , `ERROR`)

Vervang `rwcrew.eu` door `test.rwcrew.eu` voor de testomgeving.

18.12 Gegevens bekijken

Voor vragen als "hoeveel aanvragen kwamen er gisteren binnen?" is de applicatie zelf (KPI-schermen, exports) de eerste keuze. Wie meer nodig heeft en toegang kreeg, kan met pgAdmin en de read-only gebruiker de database bekijken (zie §5.5). Die gebruiker kan niets wijzigen; wijzigingen aan gegevens gebeuren altijd via de applicatie.

18.13 Wat te doen bij een storing

Tabel 18.3 – Eerste hulp bij storingen

Wat zie je	Eerst controleren	Mogelijke oorzaak en actie
Site helemaal onbereikbaar	Andere omgeving bereikbaar? `/api/health` ? Hostinger-status?	Server of Nginx plat, certificaat verlopen, DNS. Serverbeheerder: `docker compose ps` , `sudo systemctl status nginx` , `sudo certbot certificates` .
Foutpagina na een deployment	GitHub Actions rood?	Build of migratie mislukt: foutmelding in Actions lezen; zo nodig de laatste commit terugdraaien (§6.7).
Niemand kan inloggen	Login History: pogingen zichtbaar? Backend-logs?	Backend of database plat; of `JWT_SECRET` / `CORS_ALLOWED_ORIGINS` verkeerd na een wijziging.
Eén gebruiker kan niet inloggen	Login History voor zijn e-mailadres	Fout wachtwoord → resetten. Account verwijderd of inactief.
"Geen toegang" in een module	Manage Access: vinkje? Access Rights: rol?	Moduletoegang of rol ontbreekt, of de rol heeft geen "zien" op dat scherm.
Geen mails bij nieuwe aanvragen	Mailinglijst: actieve adressen? Spam? Backend-log ("skip" of fout van Resend)?	`RESEND_API_KEY` / `MAIL_FROM` leeg of fout; domein niet geverifieerd bij Resend; instelling niet in `docker-compose.prod.yml` .
Scans-bestanden komen niet binnen	Draait de watcher op de Pi? Netwerk? Juiste sleutel en omgeving?	Sleutel van de andere omgeving; Nginx 413 bij grote bestanden; watcher gestopt.
Camera of GPS werkt niet op de telefoon	Adres begint met `https://` ? Toestemming in de browser?	Alleen HTTPS geeft toegang; toestemming voor de site opnieuw toestaan in de browserinstellingen.
Foutmelding met "500"	Backend-log op het tijdstip van de fout	Fout in de code: noteer wat je deed, maak een GitHub Issue met de logregels, en verwittig een ontwikkelaar.

> **LET OP**
>
> Wijzig bij een storing **nooit** met de hand code op de server en voer geen SQL uit die gegevens wijzigt. Zo'n wijziging wordt bij de volgende deployment overschreven of blokkeert ze (`git pull --ff-only`), en maakt het probleem moeilijker te volgen. Los het op via een commit, of vraag hulp.

BIJLAGEN

Naslag

De woordenlijst, een overzicht van alle schermen en hun rechtensleutels per module, en hoe je deze handleiding zelf bijwerkt.

Bijlage A Woordenlijst

Alembic

> Tool die databasemigraties maakt en uitvoert. Zie §10.14.

API (Application Programming Interface)

> De vaste set adressen waarop de backend vragen beantwoordt. Zie §2.4.

API-sleutel

> Een geheime tekenreeks waarmee een apparaat (hier: de Raspberry Pi) zich bij de API identificeert, zonder gebruiker of wachtwoord.

App Router

> De manier waarop Next.js adressen koppelt aan mappen onder `src/app/`.

Background task (achtergrondtaak)

> Werk dat FastAPI uitvoert nadat het antwoord al vertrokken is, bv. een mail versturen.

Backend

> Het deel van de software dat de regels toepast en met de database praat. In RWCrew: FastAPI (Python).

Base UI

> Bibliotheek met toegankelijke basisonderdelen waarop de shadcn-componenten hier gebouwd zijn.

Boeking (StockMaster)

> Eén handeling die voorraad verplaatst (inboeken, kar laden, …), met een nummer zoals `SM-2026-00002`. Wordt nooit gewist; een fout herstel je met een tegenboeking.

Branch

> Een aparte lijn van wijzigingen in Git. `main` = productie, `develop` = test.

Build (bouwen)

> Broncode omzetten naar een uitvoerbare, geoptimaliseerde versie.

certbot / Let's Encrypt

> Gratis HTTPS-certificaten en het programma dat ze aanvraagt en vernieuwt.

CI/CD

> Continuous Integration / Continuous Deployment: automatisch testen en installeren bij elke wijziging. Hier: GitHub Actions.

Clientcomponent

> Een React-component dat in de browser draait (bestand begint met `"use client"`).

Commit

> Een bewaarde momentopname van wijzigingen in Git, met een beschrijving.

Container

> Een verpakt programma met alles wat het nodig heeft, dat overal hetzelfde draait. Zie Docker.

Cookie

> Een stukje tekst dat de browser bewaart en automatisch meestuurt naar dezelfde website; hier de inlogtokens.

CORS

> Cross-Origin Resource Sharing: regels over welke websites een API vanuit de browser mogen aanroepen.

CRUD

> Create, Read, Update, Delete: aanmaken, lezen, wijzigen, verwijderen. Een "CRUD-scherm" is een lijst met die vier acties.

CSV

> Eenvoudig tekstbestand met gegevens gescheiden door komma's of puntkomma's; zo leveren de RFID-scanners hun scans aan.

Database

> Het programma dat gegevens blijvend in tabellen bewaart. Hier: PostgreSQL.

Dependency (FastAPI)

> Een functie die automatisch vóór een endpoint loopt, bv. om de gebruiker en zijn rechten te controleren.

Deployen

> Een nieuwe versie van de software installeren en starten op een server.

DNS

> Het systeem dat domeinnamen (`rwcrew.eu`) vertaalt naar IP-adressen.

Docker / Docker Compose

> Tool om containers te bouwen en te draaien / om meerdere containers samen te beschrijven en te starten.

Endpoint

> Eén adres + methode van de API, bv. `POST /api/auth/login` .

`.env` -bestand

> Bestand met instellingen en geheimen per omgeving; staat nooit in Git.

FastAPI

> Python-raamwerk voor API's, gebruikt voor de backend.

Fixture (pytest)

> Herbruikbare voorbereiding voor tests, bv. een lege testdatabase of een testclient.

Foreign key (vreemde sleutel)

> Een kolom die naar de sleutel van een rij in een andere tabel verwijst.

Frontend

> Het deel van de software dat schermen maakt. In RWCrew: Next.js.

Git / GitHub

> Versiebeheersysteem / de website waar de centrale kopie van de repository staat.

GitHub Actions

> De automatisatie van GitHub die bij een push de deployment uitvoert.

Grootboek (ledger)
> De volledige, onveranderlijke lijst van alle voorraadbewegingen in StockMaster; het saldo per product en plaats kan er altijd uit herberekend worden.

Hash
> Een onomkeerbare "vingerafdruk" van gegevens; wachtwoorden worden alleen als hash bewaard.

Headless
> Een programma (bv. een browser) dat zonder zichtbaar venster draait.

HTTP / HTTPS
> Het protocol van het web / dezelfde, maar versleuteld.

HttpOnly
> Eigenschap van een cookie: JavaScript in de pagina kan ze niet lezen.

JSON
> Leesbaar tekstformaat voor gegevens, gebruikt tussen frontend en backend.

JWT (JSON Web Token)
> Een digitaal ondertekend bewijsje met daarin wie de gebruiker is en tot wanneer het geldig is.

Layout (Next.js)
> Een omhulsel rond een groep pagina's, bv. voor de inlogcontrole of het modulemenu.

Merge
> Wijzigingen van de ene branch samenvoegen in een andere.

Migratie
> Een genummerd script dat de structuur van de database aanpast. Zie §2.5.

Model (SQLAlchemy)
> Een Python-klasse die een databasetabel beschrijft.

Module
> Een afgerond deel van RWCrew met een eigen tegel, menu en rechten (Scans, KarTracker, …).

Moduletoegang
> Het ja/nee-recht om een module te openen, toegekend door de superbeheerder.

Namespace (vertalingen)
> Een groep schermteksten in `nl.json` / `en.json`, meestal één per module.

Next.js
> Raamwerk rond React, gebruikt voor de frontend.

Nginx
> De webserver/reverse proxy vooraan op de server.

ORM
> Object-Relational Mapper: vertaalt tussen Python-objecten en databasetabellen. Hier: SQLAlchemy.

pgAdmin
> Programma om een PostgreSQL-database te bekijken.

PostgreSQL
> De database van RWCrew.

Productie
> De echte omgeving met echte gegevens: `rwcrew.eu`.

Proxy (`proxy.ts`)
> Code in de frontend die vóór elke pagina loopt: taal toevoegen en de sessie vernieuwen.

Pull request (PR)
> Voorstel op GitHub om een branch samen te voegen, met ruimte voor review.

Push / pull
> Commits naar GitHub sturen / van GitHub ophalen.

PWA (Progressive Web App)
> Een website die je als app op je telefoon kunt installeren. Zie hoofdstuk 12.

Pydantic / schema
> Bibliotheek die de vorm van gegevens beschrijft en controleert / zo'n beschrijving.

pytest
> Het testraamwerk van de backend.

React
> Bibliotheek om schermen op te bouwen uit componenten.

Repository (repo)
> De map met alle broncode en haar volledige geschiedenis.

Resend
> Dienst die de uitgaande mails verstuurt.

Reverse proxy
> Een server vooraan die verzoeken aanneemt en doorstuurt naar het juiste programma.

RFID
> Radio Frequency Identification: tags die draadloos uitgelezen worden met een scanner.

Rol
> Een benoemde set rechten binnen één module, met per scherm zien/aanmaken/wijzigen/verwijderen.

Rotatie (token)
> Het vervangen van het vernieuwingstoken bij elk gebruik, waarbij het oude ongeldig wordt.

Router (FastAPI)
> Een groep endpoints met een gemeenschappelijk adresvoorvoegsel, bv. alles van module 3.

SameSite
> Eigenschap van een cookie die bepaalt of ze meegestuurd wordt met verzoeken vanaf andere websites.

SAVEPOINT
> Een tussentijds bewaarpunt in een databasetransactie; laat toe één rij van een import terug te draaien.

Schermsleutel
> De vaste, technische naam van een scherm voor de rechten, bv. `interventionrequests.requests`.

Secret (GitHub)
> Een versleuteld bewaarde waarde in GitHub, bv. de SSH-sleutel voor de deployment.

Seed-script
> Script dat een lege database van de eerste gegevens voorziet (modules, seizoenen, eerste superbeheerder).

Servercomponent
> Een React-component dat alleen op de server draait en rechtstreeks gegevens kan ophalen.

Sessie
> De periode waarin een gebruiker ingelogd blijft: maximaal 6 uur na het ingeven van het wachtwoord.

shadcn/ui
> Verzameling kant-en-klare UI-onderdelen die als broncode in het project staan.

SQL
> De taal waarmee je een relationele database bevraagt.

SQLAlchemy
> De ORM van de backend.

SQLite
> Een kleine database in een bestand of in het geheugen; gebruikt voor de automatische tests.

SSH / SSH-tunnel
> Beveiligde verbinding met een server / een beveiligde doorgang via die verbinding, bv. voor pgAdmin.

Stamgegevens
> Gegevens die alle modules delen en weinig veranderen (seizoenen, ploegen, festivals, producten). In MasterData.

Statuscode
> Getal in een HTTP-antwoord dat zegt hoe het verzoek afliep (200, 403, 500, …). Zie tabel 2.2.

Superbeheerder (super admin)
> Gebruiker die beslist wie welke module mag openen en gebruikers beheert.

Tailwind CSS
> Opmaak via korte klassen in de HTML.

Test (omgeving)
> De oefenomgeving `test.rwcrew.eu`, met de branch `develop`.

Toegangstoken / vernieuwingstoken
> Kortlevend inlogbewijs (15 min.) / langer levend bewijs om een nieuw toegangstoken te halen (tot 6 u).

Tunnel (Cloudflare)
> Een tijdelijk publiek HTTPS-adres dat naar je eigen pc wijst; voor telefoontests met camera en GPS.

TypeScript
> JavaScript met types; de taal van de frontend.

Uvicorn

> De server die de FastAPI-applicatie laat draaien.

Volume (Docker)

> Opslag die buiten een container bewaard blijft, ook als de container vervangen wordt.

VPS

> Virtual Private Server: een gehuurde virtuele server, hier bij Hostinger.

Wheel

> Een voorgecompileerd Python-pakket; zie §8.6.

Bijlage B Schermen en rechtensleutels per module

Deze lijsten komen rechtstreeks uit de `SCREEN_DEFINITIONS` in `backend/app/modules/module_N/screens.py`. Elk scherm verschijnt als rij in de rechtenmatrix van zijn module. Het label tussen haakjes is wat in de matrix staat.

Tabel B.1 – Scans (module 1)

Schermsleutel	Label en functie
`tagscan.dashboard`	(CSV Source Files) ontvangen CSV-bestanden, knop Scan
`tagscan.tag-headerdata`	(Tag Headerdata) één rij per ingelezen bestand, PDF
`tagscan.tag-linedata`	(Tag Linedata) één rij per gescande tag
`tagscan.tag-management`	(TagManagement) register van tags, Excel-import
`tagscan.scanners`	(Scanners) scanners en API-sleutels
`tagscan.settings`	(Settings) instellingen van de module
`tagscan.dataupload`	(Data Upload/Download) Excel
`tagscan.roles`, `tagscan.users`	(Roles, Users) toegangsrechten

Tabel B.2 – KarTracker (module 2)

Schermsleutel	Label en functie
`kartracker.karmanagement`	(KarManagement) karrenregister
`kartracker.karstatuses`	(KarStatussen) mogelijke statussen van een kar
`kartracker.distributiepunten`	(Distributiepunten)
`kartracker.zones`	(Zone)
`kartracker.afleverlocaties`	(Afleverlocatie)
`kartracker.leverdata`	(Delivery Dates) lever- en ophaalmomenten
`kartracker.plankar`	(Plan a kar) afleverlocatie per ploeg/festival/seizoen
`kartracker.karplanning`	(Kar Planning) overzicht en karbladen (PDF); telefoon: Kar Planning
`kartracker.karmap`	(Kar Map) kaart; telefoon: Kar Map
`kartracker.groundplan`	(Grondplan)
`kartracker.actions`	(Manuele kar beweging) bewegingen registreren; telefoon: KarScan
`kartracker.dataupload`	(Data Upload/Download) Excel
`kartracker.leveringsvolgorde`	(Leveringsvolgorde) de levertoer ordenen, met kaart
`kartracker.initieleleverplanning`	(Initiële leverplanning) planningsanalyse + PDF
`kartracker.karverplaatsingen`	(Verplaatsingen tussen festivals) planningsanalyse + PDF
`kartracker.specialelevertoeren`	(Speciale levertoeren) masterdata van de toeren
`kartracker.levertoerplanning`	(Speciale levertoeren-planning) stops per toer en festival + PDF
`kartracker.karcodesimulator`	(Karcode Simulator) code van het slot; ook op de telefoon
`kartracker.settings`	(Instellingen) de karcodeformule: *zien* en *wijzigen*

`kartracker.roles` , `kartracker.users`	(Roles, Users) toegangsrechten

Tabel B.3 – Interventie Aanvragen (module 3)

Schermsleutel	Label en functie
`interventionrequests.requests`	(Intervention Requests) aanvragen, PDF-leveringsbon; telefoon: Akties
`interventionrequests.statuses`	(Intervention Statuses) statuslijst met kleuren
`interventionrequests.teamkar`	(TeamKar) medewerkers van de leverkar
`interventionrequests.mailinglist`	(Mailing List) ontvangers van de mail bij een nieuwe aanvraag
`interventionrequests.roles` , `….users`	(Roles, Users) toegangsrechten

Tabel B.4 – Altsien Select (module 8)

Schermsleutel	Label en functie
`altsienselect.wizard`	(Ploeg Wizard) de stappen per ploeg en seizoen
`altsienselect.ploegfiche`	(Ploegfiche) alle keuzes op één scherm + PDF
`altsienselect.allteams`	(All teams (organisation)) *zien* = alle ploegen; *wijzigen* = ook na het sluiten van het seizoen
`altsienselect.requests`	(Request follow-up) opvolging van speciale vragen
`altsienselect.statuses`	(Request statuses) statussen van speciale vragen (laagste volgorde = startstatus)
`altsienselect.roles` , `….users`	(Roles, Users) toegangsrechten

Tabel B.5 – MasterData (module 9)

Schermsleutel	Label en functie
`masterdata.season`	(Years) seizoenen, periode open/dicht
`masterdata.festival`	(Festivals)
`masterdata.teams`	(Teams) ploegen, koppeling met taken en kernleden
`masterdata.team-location`	(Team Location)
`masterdata.delivery-method`	(Delivery Method)
`masterdata.team-tasks`	(Team Tasks)
`masterdata.team-responsibles`	(Ploegverantwoordelijken) per ploeg en seizoen
`masterdata.dataupload`	(Data Upload/Download) Excel
`masterdata.roles` , `masterdata.users`	(Roles, Users) toegangsrechten

Tabel B.6 – StockMaster (module 4)

Schermsleutel	Label en functie
`stockmaster.kpi`	(KPI) voorraad en karren in één oogopslag
`stockmaster.stock`	(Stock overview) voorraadoverzicht per product, met inboeken en uitboeken
`stockmaster.kars`	(Kars) karren: laden, vertrekt, terug, uitladen
`stockmaster.count`	(Stock count) telling
`stockmaster.requirements`	(Kar needs) benodigdheden per kar en seizoen

`stockmaster.orderneeds`	(To order) te bestellen
`stockmaster.bookings`	(Bookings) alle boekingen, ongedaan maken, boekingsbon
`stockmaster.reasons`	(Reasons) redenen voor uitboeken en telverschillen
`stockmaster.products`	(Products) Masterdata: producten
`stockmaster.product-types`, `stockmaster.warehouses`, `stockmaster.product-categories`, `stockmaster.product-limits`	(Product types, Warehouses, Product categories, Product limits) Masterdata: de lijsten van een product
`stockmaster.dataupload`	(Data Upload/Download) Excel van producten en hun lijsten
`stockmaster.closedseason`	(Book in a closed season) *wijzigen* = ook boeken in een gesloten seizoen
`stockmaster.roles`, `stockmaster.users`	(Roles, Users) toegangsrechten

Tabel B.7 – Atelier (module 5)

Schermsleutel	Label en functie
`atelier.kpi`	(KPI) balie, masterdata, reservaties en stock in cijfers
`atelier.desk`	(Uitgifte & inname) de balie: ophalen en terugbrengen; *wijzigen* = een uitgifte annuleren
`atelier.open`	(Openstaand) kledij die nog niet terug is
`atelier.reservations`	(Reservaties) personen en pakket per ploeg en festival
`atelier.preparation`	(Voorbereiden) dagelijkse behoefte, geschatte maten, picklijst-PDF
`atelier.stock`	(Stock) overzicht per artikel en maat
`atelier.receipt`, `atelier.wash`, `atelier.writeoff`, `atelier.count`	(Ontvangst, Wassen, Afboeken, Telling) de stockboekingen
`atelier.bookings`	(Boekingen) alle boekingen, boekingsbon; *wijzigen* = ongedaan maken
`atelier.orderneeds`	(Te bestellen) behoefte min eigen stock, bestellijst-PDF
`atelier.articles`, `atelier.sizes`	(Artikelen, Maten) masterdata
`atelier.dataupload`	(Data Upload/Download) Excel
`atelier.settings`	(Instellingen) polsband verplicht, scanners, ontvangen berichten
`atelier.scantest`	(Test-scan) polsbandscans naspelen zonder het polsbandbedrijf
`atelier.roles`, `atelier.users`	(Roles, Users) toegangsrechten

Tabel B.8 – Walkies (module 6)

Schermsleutel	Label en functie
`walkies.kpi`	(KPI) startpagina
`walkies.roles`, `walkies.users`	(Roles, Users) toegangsrechten

Modules 7 en 10 hebben geen schermregister; daar gelden de vaste rollen admin/editor/reader (zie §7.3.3).

Bijlage C Deze handleiding bijwerken

De bron van deze handleiding staat in de repository, in `docs/handleiding/`, zodat ze samen met de code kan meegroeien.

Tabel C.1 – Bestanden van de handleiding

Bestand	Inhoud
`docs/handleiding/hoofdstukken/*.html`	De tekst, per deel, in naamvolgorde samengevoegd (`00-voorwerk.html`, `01-inleiding.html`, …)
`docs/handleiding/stijl.css`	De opmaak: A4, kop- en voettekst, kaders, tabellen, diagrammen
`docs/handleiding/build_pdf.py`	Script dat nummert, de inhoudsopgave maakt en de PDF afdrukt
`docs/handleiding/afbeeldingen/*.png`	De schermafbeeldingen (gemaakt met `screenshots.py`)
`docs/handleiding/screenshot_user.py`, `screenshots.py`	Tijdelijke demogebruiker aanmaken en opruimen; de schermafbeeldingen maken met gemaskeerde persoonsgegevens
`docs/handleiding/vendor/paged.polyfill.js`	Paged.js 0.4.3 (MIT-licentie): zet HTML om naar pagina's
`docs/RWCrew-Ontwikkelaarshandleiding-v2.pdf`	Het resultaat (versie 2; versie 1 blijft als `RWCrew-Ontwikkelaarshandleiding.pdf`)

C.1 Werkwijze

1. Pas de tekst aan in het juiste bestand onder `hoofdstukken/`. Het is gewone HTML.
2. Werk de versie en datum bij op het voorblad en in de versiegeschiedenis (`00-voorwerk.html`), en de voettekst in `stijl.css` (`@bottom-left`).
3. Maak de PDF opnieuw, vanuit de hoofdmap van de repository: `python docs/handleiding/build_pdf.py`. Er is alleen Python (3.10 of hoger, geen extra pakketten) en Microsoft Edge of Google Chrome nodig.
4. Controleer de PDF (inhoudsopgave, nieuwe figuren en tabellen) en commit de bronbestanden en de PDF samen.

C.2 Schermafbeeldingen vernieuwen

Na een wijziging aan de schermen maak je de afbeeldingen opnieuw, tegen de **lokale** applicatie (database, backend en frontend moeten draaien, zie §5.2.2):

```
Vanuit de hoofdmap van de repository
backend/.venv/Scripts/python docs/handleiding/screenshot_user.py setup     # tijdelijke demogebruiker
python docs/handleiding/screenshots.py                                     # alle afbeeldingen
python docs/handleiding/screenshots.py 16 20                               # of alleen enkele
backend/.venv/Scripts/python docs/handleiding/screenshot_user.py cleanup   # gebruiker weer weg
python docs/handleiding/build_pdf.py
```

- `setup` maakt de superbeheerder `screenshots@voorbeeld.be` met toegang en een rol met alle rechten in elke module, en exporteert alle namen, e-mailadressen, telefoonnummers en IP-adressen uit de database naar een bestand dat niet in Git komt.

- `screenshots.py` logt in als die gebruiker (licht thema, Belgische datumweergave), opent elk scherm en vervangt vóór de opname alle persoonsgegevens door fictieve waarden. Welke schermen, staat in de lijst bovenaan het script.
- Kijk elke nieuwe afbeelding na voor je ze commit: geen echte namen of adressen, geen foutpagina's.
- Een schermafbeelding in de tekst: `<figure class="diagram" data-soort="scherm" id="fig-scr-…"><img src="afbeeldingen/naam.png" alt=""><figcaption>…</figcaption></figure>`; telefoonschermen naast elkaar met `data-soort="telefoon"` en een `<div class="telefoon-rij">`.

C.3 Opmaakregels voor auteurs

- **Hoofdstuk**: `<h1 class="hoofdstuk" id="h-…">Titel</h1>`, gevolgd door een doelgroeplabel (`<div class="doelgroep iedereen|beheer|ontwikkelaars">`) en een kader "Kort gezegd". Nummers worden automatisch toegekend.
- **Paragraaf**: `<h2 id="s-…">`; subparagraaf `<h3>`. Alleen h1 en h2 komen in de inhoudsopgave.
- **Kaders**: `<div class="kader kort|analogie|tip|letop|dev"><span class="kader-titel"></span>…</div>`.
- **Tabel**: `<table id="tab-…"><caption>Bijschrift</caption>…`; nummer en lijst van tabellen worden automatisch gemaakt.
- **Figuur**: `<figure class="diagram" id="fig-…"><svg …>…</svg><figcaption>…</figcaption></figure>`. Gebruik de SVG-klassen uit `stijl.css` (`box`, `box-blauw`, `pijl`, `t-titel`, …) zodat de stijl gelijk blijft.
- **Codevoorbeeld**: `<figure class="code"><span class="bestand">pad</span><pre>…</pre></figure>`.
- **Kruisverwijzing**: `<a class="ref" href="#id"></a>` wordt automatisch "§4.2", "hoofdstuk 4", "figuur 4.1", "tabel 4.1" of "bijlage A". Het script waarschuwt bij een onbekende id.
- Een nieuw deel: `<section class="deel" id="deel-…" data-toc="…">` zoals de bestaande delen.

> **TIP**
> Wil je het resultaat tijdens het schrijven snel bekijken, voer dan het script uit en open de PDF; een volledige opbouw duurt maar enkele seconden.